

GhostTrack: Diseño e Implementación de un Framework Modular de Pentesting y Evaluación de Seguridad Ofensiva

Izán Jiménez Nuñez

Escuela de Imagen y Sonido CES
Técnico Superior en Desarrollo de Aplicaciones Multiplataforma

Tutor: Mateo Morace Llamas

Trabajo de Fin de Grado — Manual Técnico
Mayo de 2026

Resumen

El presente documento constituye el manual técnico del proyecto GhostTrack , un framework de pentesting y evaluación de seguridad ofensiva desarrollado como Trabajo de Fin de Grado. GhostTrack integra cuarenta módulos funcionales organizados en dos bloques: veinte módulos básicos que cubren las fases de reconocimiento, enumeración, análisis de red y detección de vulnerabilidades web, y veinte módulos Red Team orientados a técnicas avanzadas de evaluación ofensiva en entornos corporativos Windows y Active Directory. El framework está desarrollado íntegramente en Python 3.8+ y sigue una arquitectura modular en capas que facilita su extensión mediante scripts personalizados. El documento describe en detalle la arquitectura del sistema, el diseño e implementación de cada módulo, las tecnologías utilizadas, los patrones de diseño aplicados, el sistema de reportes multipropósito y los mecanismos de uso ético y legal integrados en el núcleo del framework. La base teórica del proyecto se apoya en metodologías de referencia de la industria como PTES, OWASP Testing Guide, MITRE ATT&CK y el NIST Cybersecurity Framework. Todo el desarrollo ha sido validado en entornos de laboratorio controlados y el framework incorpora por diseño mecanismos de aviso legal y confirmación de autorización que garantizan un uso responsable.

Palabras clave: pentesting, ciberseguridad ofensiva, framework, Python, Active Directory, Red Team, vulnerabilidades, reconocimiento, módulos, auditoría de seguridad.

Índice de Contenidos

1. Introducción y motivación del proyecto	5
1.1 Contexto histórico del pentesting	5
1.2 Justificación del proyecto	7
1.3 Estructura del documento	8
2. Objetivos del proyecto	9
2.1 Objetivo general	9
2.2 Objetivos específicos	9
3. Alcance y limitaciones	12
3.1 Alcance del proyecto	12
3.2 Limitaciones conocidas	14
4. Marco teórico	17
4.1 Conceptos fundamentales de ciberseguridad ofensiva	17
4.2 Metodologías de referencia	20
4.3 Clasificación de vulnerabilidades	23
4.4 Legislación aplicable	26
5. Arquitectura del sistema	28
5.1 Visión general	28
5.2 Estructura de directorios	30
5.3 Patrones de diseño aplicados	33
5.4 Principios SOLID en GhostTrack	35
6. Tecnologías y dependencias	38
6.1 Lenguaje de programación: Python 3.8+	38
6.2 Dependencias principales	39

6.3	Dependencias opcionales Red Team	41
7.	Módulos básicos — Descripción técnica detallada	42
7.1	Módulo 01: GeoIP	42
7.2	Módulo 02: WHOIS Lookup	44
7.3	Módulo 03: DNS Enumeration	46
7.4	Módulo 04: Subdomain Finder	48
7.5	Módulo 05: Port Scanner	50
7.6	Módulo 06: Service Detection	53
7.7	Módulo 07: OS Fingerprinting	55
7.8	Módulo 08: Banner Grabbing	57
7.9	Módulo 09: SSL/TLS Analyzer	59
7.10	Módulo 10: Vulnerability Scanner	61
7.11	Módulo 11: Traceroute — Trazado de ruta de red	63
7.12	Módulo 12: Web Crawler — Rastreo de aplicaciones web	65
7.13	Módulo 13: Directory Bruteforce — Descubrimiento de rutas ocultas	68
7.14	Módulo 14: SQL Injection Scanner — Error-Based SQLi	71
7.15	Módulo 15: XSS Scanner — Reflected XSS	74
7.16	Módulo 16: ARP Scanner — Hosts activos en red local	76
7.17	Módulo 17: Network Sniffer — Captura de tráfico	78
7.18	Módulo 18: Email Harvester — Recolección OSINT de emails	80
7.19	Módulo 19: Metadata Extractor — Metadatos EXIF y PDF	82
7.20	Módulo 20: Custom Scripts — Extensión por inyección	85
8.	Módulos Red Team — Descripción técnica detallada	88
8.1	Categoría: Enumeración y reconocimiento de Active Directory	89

8.2	Categoría: Ataques a credenciales	98
8.3	Categoría: Post-explotación	108
8.4	Categoría: Persistencia y evasión	120
8.5	Categoría: Técnicas avanzadas	127
8.6	Resumen del bloque Red Team	137
9.	Utilidades del framework	141
9.1	Colors	141
9.2	Logger	143
9.3	Validator	146
9.4	Reporter	149
10.	Sistema de reportes	153
11.	Interfaz de línea de comandos	156
12.	Flujo de ejecución	161
13.	Instalación y configuración	164
14.	Casos de uso y ejemplos prácticos	169
15.	Seguridad del propio framework	175
16.	Pruebas y validación	179
17.	Conclusiones técnicas	185
18.	Referencias	189

1. Introducción y Motivación del Proyecto

La ciberseguridad es uno de los campos de mayor crecimiento en la industria tecnológica global. Según el informe anual de Cybersecurity Ventures, se estima que el coste global de la ciberdelincuencia alcanzará los 10,5 billones de dólares anuales en 2025, lo que convierte la protección de sistemas informáticos en una necesidad crítica para organizaciones de cualquier tamaño (Morgan, 2020). En este contexto, la ciberseguridad ofensiva —es decir, la disciplina que evalúa la seguridad de los sistemas desde la perspectiva del atacante— se ha convertido en una de las especialidades más demandadas del mercado laboral tecnológico.

GhostTrack surge como respuesta a una necesidad concreta: disponer de un framework de pentesting propio, completamente documentado, desarrollado desde cero y orientado tanto al aprendizaje académico como a la aplicación práctica en entornos de auditoría autorizados. La motivación principal no fue replicar herramientas ya existentes como Metasploit (Rapid7, 2003) o Nmap (Lyon, 1997), sino comprender en profundidad los algoritmos y protocolos que las sustentan, implementándolos de manera independiente en Python para maximizar el aprendizaje técnico.

La idea del proyecto tomó forma al observar que la mayoría de los frameworks de pentesting disponibles presentan una curva de aprendizaje muy pronunciada para quienes se están iniciando en la disciplina: interfaces de comandos complejas, dependencias difíciles de instalar, documentación escasa en español y código fuente poco comentado. GhostTrack intenta llenar ese hueco, ofreciendo una plataforma con suficiente profundidad técnica para ser útil en contextos reales pero con una estructura de código clara, comentada y acompañada de documentación exhaustiva.

1.1 Contexto Histórico del Pentesting

El pentesting como disciplina formal tiene sus raíces en los años 60 y 70, cuando las primeras organizaciones gubernamentales y académicas de Estados Unidos comenzaron a realizar ejercicios de evaluación de seguridad en sus sistemas informáticos. El Departamento de Defensa estadounidense fue pionero en formalizar estos ejercicios bajo el nombre de "tiger teams", grupos de expertos con el mandato de intentar comprometer la seguridad de los propios sistemas del gobierno (Anderson, 1972).

Durante los años 90, con la explosión de Internet y la proliferación de servidores web, el pentesting comenzó a evolucionar hacia una disciplina comercial. Herramientas como SATAN (Security Administrator Tool for Analyzing Networks), publicada en 1995 por Dan

Farmer y Wietse Venema, fueron las primeras en automatizar parte del proceso de evaluación de vulnerabilidades y generaron una encendida controversia sobre los límites éticos de publicar herramientas de hacking (Farmer y Venema, 1995).

La llegada del siglo XXI trajo consigo la profesionalización completa del sector. Organizaciones como OWASP (Open Web Application Security Project), fundada en 2001, comenzaron a sistematizar el conocimiento sobre vulnerabilidades y metodologías de evaluación, creando los estándares que hoy en día son referencia obligada en la industria. El desarrollo de frameworks como Metasploit Framework (Maynor y Snijder, 2006) democratizó el acceso a técnicas de explotación avanzadas y permitió a los profesionales de seguridad trabajar de forma más eficiente.

En España, el marco legal específico para las actividades de pentesting y hacking ético fue tomando forma progresivamente con las reformas del Código Penal, en particular con la introducción de los artículos 197 bis y ter por la Ley Orgánica 1/2015, que tipificaron específicamente el acceso ilícito a sistemas informáticos y la interceptación de datos. Este marco legal, aunque tardío comparado con otros países europeos, ha contribuido a clarificar las condiciones bajo las cuales las actividades de evaluación de seguridad son lícitas (Miró Llinares, 2012).

1.2 Justificación del Proyecto

La elección de Python como lenguaje de desarrollo responde a varias razones convergentes. En primer lugar, Python es el lenguaje dominante en la comunidad de ciberseguridad: la mayoría de los exploits, scripts de automatización y herramientas de análisis están escritos en Python. En segundo lugar, su sintaxis clara y su modelo de programación orientada a objetos facilitan la escritura de código mantenible y documentable. En tercer lugar, el ecosistema de librerías de Python orientadas a redes y seguridad es el más maduro disponible: Scapy, dnspython, impacket, cryptography y requests son ejemplos de librerías de alta calidad que cubren prácticamente cualquier necesidad en el ámbito del pentesting.

La arquitectura modular de GhostTrack responde a una decisión de diseño deliberada: cada módulo debe ser completamente independiente del resto, de modo que pueda desarrollarse, probarse y usarse de forma autónoma. Esta independencia no solo facilita el mantenimiento del código, sino que permite al usuario del framework comprender el funcionamiento de cada técnica por separado, sin necesidad de entender el sistema completo antes de poder ejecutar un análisis concreto.

El nombre GhostTrack hace referencia a dos conceptos: 'Ghost' evoca la capacidad de operar de forma silenciosa y discreta, característica fundamental en cualquier ejercicio de evaluación de seguridad donde la detección prematura puede comprometer los resultados de la auditoría; 'Track' hace referencia al rastreo y seguimiento sistemático de la superficie de ataque del objetivo, que es precisamente lo que el framework permite realizar de forma estructurada y reproducible.

1.3 Estructura del Documento

El presente manual técnico está estructurado en dieciocho capítulos que cubren todos los aspectos del proyecto, desde la fundamentación teórica hasta los detalles más específicos de implementación de cada módulo. Los capítulos 1 a 4 establecen el contexto del proyecto: introducción y motivación, objetivos, alcance y limitaciones, y marco teórico. Los capítulos 5 y 6 describen la arquitectura del sistema y las tecnologías utilizadas. Los capítulos 7 y 8, que constituyen el núcleo técnico del documento, describen en detalle el diseño e implementación de cada uno de los cuarenta módulos. Los capítulos 9 a 12 documentan los sistemas transversales: utilidades, reportes, CLI y flujo de ejecución. Los capítulos 13 y 14 ofrecen información práctica de instalación y casos de uso. Los capítulos 15 y 16 abordan la seguridad del propio framework y las pruebas realizadas. El documento concluye con las conclusiones técnicas y la bibliografía en formato APA 7ª edición.

2. Objetivos del Proyecto

2.1 Objetivo General

Diseñar, implementar y documentar un framework de pentesting y evaluación de seguridad ofensiva, denominado GhostTrack , que integre cuarenta módulos funcionales organizados en dos categorías diferenciadas —módulos básicos de reconocimiento y análisis, y módulos Red Team de técnicas avanzadas— capaz de ejecutar auditorías de seguridad completas en entornos autorizados, generar reportes profesionales en múltiples formatos y ser fácilmente extensible mediante la inyección de scripts Python personalizados, todo ello en el marco de los estándares éticos y legales vigentes en materia de ciberseguridad.

2.2 Objetivos Específicos

Para alcanzar el objetivo general, se han definido los siguientes objetivos específicos, organizados por áreas temáticas:

Objetivos de desarrollo de módulos

- Implementar los veinte módulos básicos de forma completamente funcional, cubriendo las fases de reconocimiento pasivo y activo (GeoIP, WHOIS, DNS Enumeration, Subdomain Finder, Traceroute, Email Harvester), análisis de red (Port Scanner, Service Detection, OS Fingerprinting, Banner Grabbing, ARP Scanner, Network Sniffer), evaluación de vulnerabilidades (SSL/TLS Analyzer, Vulnerability Scanner) y hacking web (Web Crawler, Directory Bruteforce, SQL Injection, XSS Scanner, Metadata Extractor) y soporte a la extensibilidad (Custom Scripts).
- Desarrollar los veinte módulos Red Team organizados en cinco categorías funcionales: enumeración de Active Directory (SMB, LDAP, BloodHound), ataques a credenciales (Kerberos, NTLM Relay, Credential Dump), post-explotación (Privilege Escalation, Lateral Movement, Token Manipulation, Process Injection), persistencia y evasión (DLL Hijacking, Persistence Mechanisms, Firewall Evasion, AV Evasion) y técnicas avanzadas (Payload Generation, C2 Communication, Data Exfiltration, WiFi Attacks, Bluetooth Scanning, Social Engineering).
- Garantizar que cada módulo sigue la misma interfaz funcional estandarizada, con una función de nivel de módulo que encapsula la instanciación y uso de las clases internas y permite tanto la ejecución orquestada como la invocación individual.

Objetivos de arquitectura y diseño

- Diseñar una arquitectura modular en capas que aplique los patrones de diseño Facade, Strategy y Template Method, garantizando la separación de responsabilidades entre los diferentes componentes del sistema.
- Construir un sistema de utilidades transversal compuesto por cuatro módulos: Colors para la gestión de la interfaz de terminal, Logger para el registro estructurado de operaciones, Validator para la validación centralizada de todas las entradas del sistema, y Reporter para la generación de informes en múltiples formatos.
- Aplicar los principios SOLID de diseño orientado a objetos, en particular el Principio de Responsabilidad Única (SRP), el Principio Abierto/Cerrado (OCP) y el Principio de Inversión de Dependencias (DIP).

Objetivos de usabilidad y extensibilidad

- Implementar una interfaz de línea de comandos (CLI) completa con argparse que exponga todos los modos de operación del framework, con mensajes de ayuda integrados y validación de argumentos.
- Desarrollar un sistema de inyección de scripts personalizados con una plantilla de referencia que permita a cualquier usuario con conocimientos básicos de Python extender la funcionalidad del framework sin modificar el código base.
- Implementar un sistema de configuración mediante el fichero config.json que permita personalizar el comportamiento del framework sin necesidad de modificar el código fuente.

Objetivos de seguridad y ética

- Integrar mecanismos de aviso legal y confirmación explícita de autorización en el flujo de ejecución principal del framework, garantizando que todas las operaciones quedan registradas en los logs de auditoría.
- Diseñar el sistema de gestión de credenciales siguiendo las mejores prácticas: almacenamiento en ficheros .env excluidos del control de versiones, plantillas de configuración sin valores reales, y validación de la presencia de credenciales antes de intentar usarlas.

Objetivos de documentación

- Elaborar un manual técnico exhaustivo que describa todos los aspectos del proyecto: arquitectura, diseño, implementación, instalación, uso y pruebas, siguiendo las normas de citación APA 7ª edición.

- Documentar el código fuente con docstrings completos que describan la funcionalidad, parámetros y valores de retorno de cada clase y función.

3. Alcance y Limitaciones

3.1 Alcance del Proyecto

GhostTrack ha sido diseñado para cubrir de manera funcional las cuatro grandes fases de un ejercicio de pentesting estándar según las metodologías PTES y OWASP Testing Guide. A continuación se detalla el alcance de cada fase y los módulos que la cubren.

Fase 1: Reconocimiento

El reconocimiento es la fase inicial de cualquier ejercicio de evaluación de seguridad y consiste en recopilar la máxima información posible sobre el objetivo sin interactuar directamente con sus sistemas de producción. Esta fase se subdivide en reconocimiento pasivo (uso de fuentes de información pública sin contacto directo con el objetivo) y reconocimiento activo (interacción directa pero de bajo impacto).

En la dimensión pasiva, GhostTrack cubre la geolocalización de IPs y dominios (módulo GeoIP), la consulta de registros de dominio (módulo WHOIS), la enumeración de registros DNS públicos (módulo DNS Enumeration), el descubrimiento de subdominios mediante Certificate Transparency Logs (módulo Subdomain Finder), la recolección de emails públicos (módulo Email Harvester) y la extracción de metadatos de ficheros publicados (módulo Metadata Extractor).

En la dimensión activa de bajo impacto, el módulo Traceroute permite mapear la topología de red del objetivo y el módulo ARP Scanner permite descubrir hosts activos en la red local mediante la caché ARP del sistema.

Fase 2: Enumeración y escaneo

La fase de enumeración profundiza en los sistemas identificados durante el reconocimiento para obtener una imagen detallada de su superficie de ataque expuesta. GhostTrack cubre en esta fase el escaneo de puertos TCP/UDP (módulo Port Scanner), la identificación de servicios y versiones (módulo Service Detection), la detección del sistema operativo mediante técnicas de fingerprinting (módulo OS Fingerprinting), la captura de banners de servicios (módulo Banner Grabbing) y el análisis detallado de la configuración SSL/TLS (módulo SSL/TLS Analyzer).

Fase 3: Análisis de vulnerabilidades

Una vez mapeada la superficie de ataque, la fase de análisis de vulnerabilidades permite identificar los puntos débiles que podrían ser explotados. En el dominio de la red, el

módulo Vulnerability Scanner cruza los servicios identificados con una base de datos de CVEs y comprueba la presencia de configuraciones inseguras. En el dominio web, los módulos Web Crawler, Directory Bruteforce, SQL Injection y XSS Scanner permiten evaluar la seguridad de aplicaciones web en busca de las vulnerabilidades más comunes del OWASP Top 10.

Fase 4: Explotación y post-explotación (Red Team)

Los veinte módulos Red Team cubren las fases más avanzadas de un ejercicio de evaluación ofensiva. Estos módulos son especialmente relevantes en ejercicios de Red Team donde el objetivo es demostrar el impacto real que tendría un atacante que hubiera conseguido acceso inicial a la red interna de la organización. Los módulos cubren enumeración de Active Directory, ataques a los mecanismos de autenticación de Windows, escalada de privilegios, movimiento lateral, persistencia, evasión de defensas y exfiltración de datos.

Sistema de reportes

El alcance del sistema de reportes incluye la generación automática de informes en tres formatos complementarios (HTML visual, JSON estructurado y TXT plano) al finalizar cualquier escaneo, con nombres de fichero que incluyen el target, el tipo de escaneo y el timestamp para garantizar la unicidad y trazabilidad de cada sesión de auditoría.

3.2 Limitaciones Conocidas

Como cualquier proyecto de desarrollo de software con recursos y tiempo acotados, GhostTrack presenta limitaciones que conviene documentar con honestidad académica para que el lector tenga una imagen precisa del estado del proyecto y de las vías de mejora futura.

Limitaciones de la base de datos de vulnerabilidades

El módulo Vulnerability Scanner incorpora una base de datos de CVEs implementada de forma estática en el código fuente, con un conjunto representativo de vulnerabilidades conocidas. Esta aproximación es suficiente para demostrar el funcionamiento del módulo pero no es adecuada para uso en producción real, donde se requeriría integración con fuentes de datos actualizadas en tiempo real como la National Vulnerability Database (NVD) del NIST, la API de Shodan o los feeds OVAL de los fabricantes de software. La integración con estas fuentes está planificada como mejora futura y requeriría la configuración de las API keys correspondientes en el fichero .env.

Limitaciones de los módulos Red Team

Los módulos Red Team más complejos (NTLMRelayAttacker, ProcessInjector, AVEvasion, C2Communication) incluyen implementaciones que demuestran los conceptos subyacentes y son válidas para entornos de laboratorio, pero cuya operatividad plena en entornos empresariales reales requeriría dependencias adicionales como Impacket (para ataques a protocolos Windows), que se han mantenido como dependencias opcionales para no comprometer la instalación básica del framework ni crear fricciones innecesarias en el proceso de instalación.

Limitaciones del módulo Network Sniffer

El módulo de captura de tráfico de red requiere privilegios de administrador o root en el sistema operativo anfitrión para poder abrir sockets de red en modo promiscuo. En sistemas Linux, esto puede resolverse ejecutando el framework con sudo o asignando la capability CAP_NET_RAW al intérprete de Python mediante setcap. En sistemas Windows se requiere ejecutar el proceso con derechos de administrador. Esta limitación es inherente al funcionamiento del protocolo a nivel de sistema operativo y no puede eliminarse sin comprometer la funcionalidad del módulo.

Limitaciones del OS Fingerprinting

El módulo de detección de sistema operativo emplea técnicas de análisis de TTL y banners de servicios, que proporcionan una estimación con nivel de confianza medio. Una implementación comparable en precisión a la de herramientas especializadas como Nmap utilizaría análisis del stack TCP/IP completo, incluyendo el tamaño de ventana TCP, las opciones de TCP presentes en los segmentos SYN, el comportamiento ante paquetes malformados y otros parámetros que requieren la construcción de paquetes TCP personalizados a nivel de bytes. Esta implementación completa requeriría sockets raw y privilegios de administrador, lo que la haría dependiente del mismo requisito de privilegios que el Network Sniffer.

Limitaciones del módulo de búsqueda en motores

El submódulo de búsqueda en motores de búsqueda del Email Harvester y el Subdomain Finder construyen las queries de Google Dorking pero no las ejecutan automáticamente en la implementación actual. La ejecución automatizada de búsquedas en Google requiere bien el uso de la API oficial de Custom Search (de pago), bien técnicas de web scraping que los términos de servicio de Google no permiten. En su lugar, el módulo genera las queries que el auditor puede copiar y ejecutar manualmente, lo que es una práctica habitual en la industria para evitar problemas de bloqueo de IP.

4. Marco Teórico

4.1 Conceptos Fundamentales de Ciberseguridad Ofensiva

4.1.1 *Pentesting y Hacking Ético*

El pentesting, también conocido como prueba de penetración o hacking ético, es la práctica controlada de intentar comprometer la seguridad de un sistema, red o aplicación con el objetivo de identificar vulnerabilidades antes de que lo haga un actor malicioso. La diferencia fundamental entre el pentesting y un ataque real no radica en las técnicas utilizadas —que pueden ser idénticas— sino en tres elementos: el consentimiento explícito del propietario del sistema, el alcance delimitado por contrato y el propósito constructivo de mejorar la seguridad (Engelbreton, 2013).

Desde el punto de vista metodológico, un ejercicio de pentesting sigue una secuencia de fases bien definida que se repite en todas las metodologías de referencia, aunque con variaciones en la nomenclatura. La más extendida es la definida por el PTES (Penetration Testing Execution Standard), que estructura el proceso en siete fases: interacciones previas al compromiso (definición de alcance, reglas de enfrentamiento y protección legal mutua), recopilación de inteligencia, modelado de amenazas, análisis de vulnerabilidades, explotación, post-explotación y elaboración del informe (PTES Technical Guidelines, 2012).

4.1.2 *Red Team vs. Pentesting Tradicional*

El concepto de Red Team surge del ámbito militar, donde se denominaba así al equipo que adoptaba el rol del adversario en ejercicios de guerra simulados para identificar debilidades en las propias defensas. Trasladado al contexto de la ciberseguridad, un Red Team engagement es un ejercicio avanzado de evaluación ofensiva que simula una campaña de ataque real y prolongada, utilizando todas las técnicas disponibles (técnicas y humanas) sin restricciones artificiales más allá del contrato firmado (Mitnick y Simon, 2002).

Las diferencias principales entre un ejercicio de pentesting estándar y un Red Team engagement son: el tiempo de duración (días o semanas para pentesting frente a meses para Red Team), el grado de sigilo requerido (el Red Team debe operar sin ser detectado por los defensores, mientras que el pentesting puede ser conocido por el equipo de seguridad), la cobertura (el Red Team ataca todo el perímetro incluidos vectores físicos y de ingeniería social, mientras que el pentesting suele limitarse a sistemas técnicos) y la existencia de un Blue Team activo que intenta detectar y responder a los ataques del Red Team en tiempo real.

4.1.3 Active Directory como Objetivo Principal

Active Directory (AD) es el servicio de directorio desarrollado por Microsoft que está presente en prácticamente todas las organizaciones de tamaño mediano y grande que operan con infraestructura Windows. AD gestiona la autenticación y autorización de todos los usuarios y equipos del dominio mediante el protocolo Kerberos (versión 5, estandarizado en el RFC 4120) para autenticación y LDAP para el acceso al directorio.

Desde la perspectiva del atacante, Active Directory representa uno de los objetivos más valiosos de cualquier organización, porque comprometer el controlador de dominio implica comprometer toda la organización: acceso a todos los sistemas, a todas las credenciales almacenadas y a toda la información gestionada por la infraestructura Windows. Esta es la razón por la que el bloque de módulos Red Team de GhostTrack está especialmente orientado a entornos Active Directory, cubriendo técnicas como Kerberoasting (Medin, 2014), NTLM Relay (Byt3bl33d3r, 2016) y la recolección de datos para BloodHound (Rohan, 2016).

4.1.4 La Cadena de Ataque (Kill Chain)

El concepto de Cyber Kill Chain fue introducido por Lockheed Martin (Hutchins et al., 2011) como adaptación del modelo militar de Kill Chain al dominio de los ciberataques. El modelo define siete fases secuenciales que componen un ataque cibernético completo: reconocimiento, armamento del exploit, entrega del payload al objetivo, explotación de una vulnerabilidad, instalación de malware o herramienta de acceso persistente, establecimiento de canal de mando y control (C2) y consecución de los objetivos finales (exfiltración de datos, sabotaje, espionaje, etc.).

GhostTrack cubre de forma directa o indirecta todas las fases de la Kill Chain: el bloque de módulos básicos cubre reconocimiento y parte de la entrega; los módulos Red Team cubren explotación, instalación, C2 y exfiltración. Esta cobertura completa de la Kill Chain es precisamente lo que distingue a GhostTrack de herramientas más especializadas que solo cubren una o dos fases.

4.2 Metodologías de Referencia

4.2.1 PTES — Penetration Testing Execution Standard

El PTES es el estándar más completo disponible para la ejecución de pruebas de penetración. Fue desarrollado colaborativamente por un grupo de profesionales de la industria de la seguridad y publicado en 2012 como documento de referencia abierto. El

estándar no solo define las fases del proceso sino también las técnicas específicas que se aplican en cada fase, los entregables esperados y las consideraciones legales y éticas que deben tenerse en cuenta (PTES Technical Guidelines, 2012).

La arquitectura de GhostTrack sigue directamente la estructura del PTES. Los módulos de reconocimiento pasivo corresponden a la fase de Intelligence Gathering pasiva; los módulos de escaneo corresponden a la fase de Intelligence Gathering activa; los módulos de análisis de vulnerabilidades corresponden a la fase de Vulnerability Analysis; y los módulos Red Team corresponden a las fases de Exploitation y Post-Exploitation. El sistema de reportes de GhostTrack genera los informes que corresponden a la fase final de Reporting del estándar.

4.2.2 OWASP Testing Guide v4.2

La Guía de Pruebas de OWASP es el estándar de facto para la evaluación de seguridad de aplicaciones web. La versión 4.2, publicada en 2020, organiza las pruebas en doce categorías principales: gestión de información y configuración, autenticación, autorización, gestión de sesiones, validación de entradas, manejo de errores, criptografía, lógica de negocio, lado del cliente, APIs, y varias pruebas específicas de tecnología (OWASP Foundation, 2020).

Los módulos web de GhostTrack están directamente inspirados en las categorías de OWASP. El módulo SQL Injection implementa las pruebas OTG-INPVAL-005 (Testing for SQL Injection). El módulo XSS Scanner implementa OTG-INPVAL-001 (Testing for Reflected Cross Site Scripting). El módulo Directory Bruteforce corresponde a OTG-CONFIG-004 (Enumerate Infrastructure and Application Admin Interfaces). El módulo Web Crawler implementa OTG-INFO-003 (Enumerate Application Entry Points).

4.2.3 MITRE ATT&CK Framework

MITRE ATT&CK (Adversarial Tactics, Techniques, and Common Knowledge) es una base de conocimiento de tácticas y técnicas utilizadas por actores de amenaza reales en ciberataques documentados, organizada en matrices según el entorno objetivo. La matriz Enterprise, la más relevante para GhostTrack, categoriza el comportamiento de los atacantes en catorce tácticas, cada una de las cuales contiene múltiples técnicas y sub-técnicas específicas (MITRE Corporation, 2023).

Los módulos Red Team de GhostTrack están mapeados a tácticas específicas de la matriz ATT&CK. La siguiente tabla muestra el mapeo entre los módulos de GhostTrack y las tácticas MITRE ATT&CK:

Tabla 1 Mapeo de módulos Red Team de GhostTrack a tácticas MITRE ATT&CKNota. Elaboración propia basada en MITRE Corporation (2023).

Módulo GhostTrack	Táctica ATT&CK	Técnica Principal
SMBEnumerator	Discovery (TA0007)	T1135 – Network Share Discovery
LDAPEnumerator	Discovery (TA0007)	T1087.002 – Account Discovery: Domain Account
KerberosAttacker	Credential Access (TA0006)	T1558.003 – Kerberoasting
NTLMRelayAttacker	Credential Access (TA0006)	T1557.001 – NTLM Relay
BloodHoundCollector	Discovery (TA0007)	T1482 – Domain Trust Discovery
PrivEscChecker	Privilege Escalation (TA0004)	T1068 – Exploitation for Privilege Escalation
LateralMovement	Lateral Movement (TA0008)	T1021.002 – Remote Services: SMB/Windows Admin Shares
CredentialDumper	Credential Access (TA0006)	T1003.001 – LSASS Memory
TokenManipulator	Privilege Escalation (TA0004)	T1134 – Access Token Manipulation
ProcessInjector	Defense Evasion (TA0005)	T1055 – Process Injection
DLLHijacking	Persistence (TA0003)	T1574.001 – DLL Search Order Hijacking
Persistence	Persistence (TA0003)	T1547.001 – Boot or Logon Autostart Execution
FirewallEvasion	Defense Evasion (TA0005)	T1562.004 – Disable or Modify System Firewall
AVEvasion	Defense Evasion (TA0005)	T1027 – Obfuscated Files or Information
PayloadGenerator	Execution (TA0002)	T1059 – Command and Scripting Interpreter
C2Communication	Command and Control (TA0011)	T1071 – Application Layer Protocol
DataExfiltration	Exfiltration (TA0010)	T1048 – Exfiltration Over Alternative Protocol
WiFiAttacker	Initial Access (TA0001)	T1200 – Hardware Additions
BluetoothScanner	Discovery (TA0007)	T1046 – Network Service Scanning
SocialEngineering	Initial Access (TA0001)	T1566 – Phishing

4.2.4 NIST Cybersecurity Framework 2.0

El Marco de Ciberseguridad del NIST (National Institute of Standards and Technology) es un marco de gestión de riesgos de ciberseguridad ampliamente adoptado por organizaciones del sector público y privado. La versión 2.0, publicada en 2024, estructura la gestión de la ciberseguridad en seis funciones: Govern (nueva en la versión 2.0), Identify, Protect, Detect, Respond y Recover (NIST, 2024).

GhostTrack se inscribe principalmente en la función Identify del NIST CSF, que incluye la identificación de activos (función que cumplen los módulos de Port Scanner, Service Detection y OS Fingerprinting), la evaluación de riesgos (función del Vulnerability Scanner y el SSL/TLS Analyzer) y el análisis de la cadena de suministro. También tiene relevancia en la función Detect, ya que varios módulos (como el Network Sniffer o el ARP Scanner) pueden usarse para identificar anomalías en la propia red.

4.3 Clasificación de Vulnerabilidades

4.3.1 OWASP Top 10

El OWASP Top 10 es la lista de las diez categorías de riesgo de seguridad más críticas en aplicaciones web, actualizada periódicamente por la comunidad OWASP. La edición de 2021, la más reciente disponible al inicio del desarrollo de GhostTrack, identifica las siguientes categorías (OWASP Foundation, 2021):

- A01:2021 – Broken Access Control: Fallos en la implementación de controles de acceso que permiten a usuarios no autorizados acceder a funcionalidades o datos restringidos.
- A02:2021 – Cryptographic Failures: Uso inadecuado o ausente de criptografía para proteger datos en tránsito o en reposo.
- A03:2021 – Injection: Inyección de código malicioso (SQL, NoSQL, OS, LDAP, etc.) a través de entradas no validadas. El módulo SQL Injection de GhostTrack aborda específicamente esta categoría.
- A04:2021 – Insecure Design: Defectos en el diseño de la arquitectura de seguridad que no pueden ser corregidos mediante una implementación correcta.
- A05:2021 – Security Misconfiguration: Configuraciones por defecto inseguras, configuraciones incompletas, servicios innecesarios habilitados, etc.
- A06:2021 – Vulnerable and Outdated Components: Uso de componentes con vulnerabilidades conocidas. El módulo Vulnerability Scanner aborda esta categoría.

- A07:2021 – Identification and Authentication Failures: Implementación incorrecta de los mecanismos de autenticación y gestión de sesiones.
- A08:2021 – Software and Data Integrity Failures: Código y datos que no se validan antes de ser usados.
- A09:2021 – Security Logging and Monitoring Failures: Ausencia de logs o monitorización que permita detectar brechas de seguridad.
- A10:2021 – Server-Side Request Forgery (SSRF): El servidor puede ser manipulado para realizar peticiones a recursos internos.

4.3.2 Sistema de Puntuación de Vulnerabilidades (CVSS)

El Common Vulnerability Scoring System (CVSS) es un framework estándar de la industria para comunicar las características y la gravedad de las vulnerabilidades de software. La versión 3.1, gestionada por FIRST (Forum of Incident Response and Security Teams), asigna a cada vulnerabilidad una puntuación numérica de 0.0 a 10.0 basada en tres grupos de métricas: métricas base (intrínsecas de la vulnerabilidad, independientes del entorno), métricas temporales (que varían en función de la disponibilidad de exploits y parches) y métricas de entorno (que permiten adaptar la puntuación al contexto específico de cada organización) (FIRST, 2019).

GhostTrack utiliza las puntuaciones CVSS como parte de la salida del módulo Vulnerability Scanner para comunicar la severidad de cada hallazgo de forma estandarizada. Los rangos de severidad estándar son: None (0.0), Low (0.1–3.9), Medium (4.0–6.9), High (7.0–8.9) y Critical (9.0–10.0).

4.3.3 Common Vulnerabilities and Exposures (CVE)

El sistema CVE, mantenido por MITRE Corporation con financiación del Departamento de Seguridad Nacional de los Estados Unidos, proporciona identificadores únicos para cada vulnerabilidad de seguridad conocida públicamente. Un identificador CVE tiene el formato CVE-[AÑO]-[NÚMERO], donde el año corresponde al año de publicación o descubrimiento de la vulnerabilidad (MITRE Corporation, 2024).

La base de datos interna de GhostTrack mapea identificadores CVE a patrones de versión de software para identificar servicios afectados. Aunque la base de datos actual es representativa y no exhaustiva, el diseño del módulo permite su expansión sin cambios en el código de orquestación.

4.4 Legislación Aplicable

4.4.1 Código Penal Español

La reforma del Código Penal español mediante la Ley Orgánica 1/2015 introdujo los artículos 197 bis y 197 ter, que tipifican específicamente los delitos informáticos contra la intimidad y la seguridad de sistemas. El artículo 197 bis establece penas de seis meses a dos años de prisión para quien, sin estar debidamente autorizado, acceda o facilite el acceso a datos o programas informáticos contenidos en un sistema de información, o a todo o parte de un sistema de información, vulnerando las medidas de seguridad establecidas para impedirlo (Código Penal, 1995, art. 197 bis).

Es importante destacar que el elemento central que diferencia el pentesting legal del acceso ilícito es la autorización previa y expresa del titular del sistema. Esta autorización debe ser por escrito, debe definir el alcance exacto de las pruebas permitidas y debe estar firmada por una persona con capacidad legal para otorgarla en nombre de la organización.

4.4.2 Directiva UE 2013/40/UE

La Directiva del Parlamento Europeo y del Consejo relativa a los ataques contra los sistemas de información, que derogó la Decisión Marco 2005/222/JAI, establece normas mínimas de definición de las infracciones penales y de las sanciones aplicables en el ámbito de los ataques contra sistemas de información. La directiva exige a los Estados miembros que tipifiquen como infracción penal el acceso ilícito a sistemas, la interferencia ilícita en el sistema, la interferencia ilícita en los datos y la interceptación ilícita (Parlamento Europeo y Consejo de la Unión Europea, 2013).

4.4.3 RGPD y LOPDGDD

El Reglamento General de Protección de Datos (RGPD), aplicable en todos los estados miembros de la UE desde mayo de 2018, establece obligaciones específicas para el tratamiento de datos personales que son relevantes en el contexto del pentesting. Cuando un ejercicio de pentesting implica acceso a sistemas que contienen datos personales —lo que es habitual en entornos de producción— el tratamiento de esos datos durante la auditoría debe realizarse de acuerdo con los principios del RGPD: licitud, lealtad y transparencia, limitación de la finalidad, minimización de datos, exactitud, limitación del plazo de conservación e integridad y confidencialidad (Parlamento Europeo y Consejo de la Unión Europea, 2016).

En la práctica, esto significa que los datos personales que pudieran ser accedidos durante una auditoría deben tratarse con confidencialidad, no deben ser copiados más allá de lo necesario para documentar el hallazgo, deben eliminarse al finalizar el ejercicio y no deben usarse para ningún propósito distinto de la propia auditoría. GhostTrack refleja estos principios en su arquitectura: los logs registran las operaciones realizadas pero no almacenan

datos personales de terceros, y los reportes generados se almacenan localmente en el sistema del auditor sin transmisión a servidores externos.

5. Arquitectura del Sistema

5.1 Visión General

GhostTrack sigue una arquitectura modular en capas, donde cada componente tiene una responsabilidad única y bien definida. Esta separación de responsabilidades es una de las características fundamentales del diseño del framework y responde directamente a los principios de la ingeniería del software orientada a objetos, en particular al principio de alta cohesión y bajo acoplamiento.

La arquitectura puede describirse mediante cuatro capas claramente diferenciadas, organizadas de mayor a menor abstracción:

Capa de presentación y orquestación

El punto de entrada del framework es el fichero `main.py`, que contiene la función `main()` y la clase `GhostTrack`. La función `main()` se encarga de parsear los argumentos de la CLI mediante `argparse`, validar el target proporcionado utilizando el `Validator`, mostrar el disclaimer legal, solicitar la confirmación del usuario y delegar la ejecución al método apropiado de la instancia de `GhostTrack`. La clase `GhostTrack` actúa como fachada (patrón *Facade*) que coordina las llamadas a los módulos individuales y consolida los resultados.

Capa de módulos de negocio

Los cuarenta módulos de pentesting constituyen el núcleo funcional del framework. Están organizados en dos paquetes Python: `modules/` para los veinte módulos básicos y `modules/redteam/` para los veinte módulos avanzados. Cada módulo sigue la misma estructura interna: una clase principal que encapsula toda la lógica del módulo (con su estado interno, sus métodos privados y sus métodos de visualización de resultados) y una función de nivel de módulo que proporciona la interfaz pública simplificada para uso externo.

Capa de servicios transversales

El paquete `utils/` proporciona servicios compartidos que son utilizados de forma transversal por múltiples módulos. Contiene cuatro módulos: `Colors` (gestión de la interfaz de terminal con códigos ANSI), `Logger` (registro estructurado de eventos con rotación de ficheros), `Validator` (validación centralizada de todos los tipos de entrada) y `Reporter` (generación de informes en múltiples formatos). Estos módulos son completamente independientes entre sí y no tienen dependencias con los módulos de negocio.

Capa de infraestructura

La capa de infraestructura incluye el fichero de configuración `config.json`, el sistema de variables de entorno (`.env`), los directorios de datos (`data/wordlists/`), los directorios de salida (`reports/`, `logs/`) y el sistema de gestión de dependencias (`requirements.txt`). Esta capa proporciona los recursos que los módulos necesitan para operar.

5.2 Estructura de Directorios

La estructura de directorios del proyecto ha sido diseñada para ser autoexplicativa y seguir las convenciones estándar de proyectos Python. A continuación se describe cada elemento:

```
ghosttrack/
├─ main.py                # Orquestador principal y clase GhostTrack
├─ config.json            # Configuración global (puertos, threads,
paths...)
├─ requirements.txt       # Dependencias Python con versiones fijadas
├─ .env.example           # Plantilla de variables de entorno sin
valores reales
├─ .gitignore             # Exclusiones del control de versiones
├─ LICENSE                # Licencia MIT Restricted Use
├─ README.md              # Documentación de inicio rápido para GitHub
├─
├─ modules/              # Paquete: módulos básicos de pentesting
│   └─ __init__.py        # Inicialización del paquete, versión y autor
│   └─ geoip.py           # Módulo 01: Geolocalización IP
│   └─ whois_lookup.py    # Módulo 02: Consulta WHOIS
│   └─ dns_enum.py        # Módulo 03: Enumeración DNS
│   └─ subdomain_finder.py # Módulo 04: Descubrimiento de subdominios
│   └─ port_scanner.py    # Módulo 05: Escáner de puertos TCP/UDP
│   └─ service_detection.py # Módulo 06: Detección de servicios y
versiones
│   └─ os_fingerprint.py  # Módulo 07: Fingerprinting de sistema
operativo
│   └─ banner_grabbing.py # Módulo 08: Captura de banners de servicios
│   └─ ssl_analyzer.py    # Módulo 09: Análisis SSL/TLS y certificados
│   └─ vuln_scanner.py    # Módulo 10: Escáner de vulnerabilidades y
CVEs
│   └─ traceroute.py      # Módulo 11: Trazado de ruta de red
│   └─ web_crawler.py     # Módulo 12: Crawler web recursivo
│   └─ directory_bruteforce.py # Módulo 13: Fuerza bruta de directorios web
│   └─ sql_injection.py   # Módulo 14: Detección de SQL Injection
│   └─ xss_scanner.py     # Módulo 15: Detección de XSS
│   └─ arp_scanner.py     # Módulo 16: Escáner ARP de red local
│   └─ network_sniffer.py # Módulo 17: Captura de tráfico de red
```

```

|   └─ email_harvester.py      # Módulo 18: Recolección de emails OSINT
|   └─ metadata_extractor.py   # Módulo 19: Extracción de metadatos EXIF/PDF
|   └─ custom_scripts.py       # Módulo 20: Inyección de scripts
personalizados
|
└─ modules/redteam/           # Paquete: módulos Red Team avanzados
    └─ __init__.py             # Exportaciones del paquete (20 clases + 1
función)
        └─ complete_redteam.py # Implementación de los 20 módulos Red Team
|
└─ utils/                     # Paquete: servicios transversales
    └─ __init__.py             # Exportaciones: Colors, Logger, Validator,
Reporter
        └─ colors.py           # Constantes y métodos de color ANSI
        └─ logger.py           # Sistema de logging con rotación de ficheros
        └─ validator.py        # Validación centralizada de entradas
        └─ reporter.py         # Generación de reportes HTML/JSON/TXT
|
└─ scripts/                   # Scripts personalizados de ejemplo
    └─ template.py             # Plantilla para scripts personalizados
|
└─ data/                      # Recursos estáticos
    └─ wordlists/              # Listas de palabras para brute force
        └─ directories.txt     # Directorios web comunes
        └─ subdomains.txt      # Subdominios comunes
        └─ passwords.txt       # Contraseñas comunes (uso en labs)
|
└─ reports/                   # Directorio de reportes generados
(gitignored)
└─ logs/                      # Ficheros de log de sesiones (gitignored)
└─ docs/                      # Documentación técnica adicional

```

5.3 Patrones de Diseño Aplicados

5.3.1 Patrón Facade (Fachada)

El patrón Facade proporciona una interfaz simplificada a un conjunto de interfaces más complejas en un subsistema. En GhostTrack, la clase GhostTrack en main.py actúa como fachada que simplifica el acceso a la funcionalidad de los cuarenta módulos. Los métodos run_full_scan() y run_redteam_assessment() de la clase GhostTrack encapsulan la lógica de invocación de múltiples módulos, el manejo de errores entre módulos y la consolidación de resultados, ofreciendo al código cliente (la función main()) una interfaz limpia de dos métodos (Gamma et al., 1994).

Esta implementación del patrón Facade tiene varias ventajas concretas: protege el código cliente de la complejidad de coordinar veinte módulos independientes, permite cambiar la implementación de un módulo sin afectar al orquestador, y simplifica enormemente las pruebas del sistema al poder sustituir la fachada por un doble de prueba que simule su comportamiento.

5.3.2 Patrón Strategy (Estrategia)

El patrón Strategy define una familia de algoritmos, encapsula cada uno de ellos y los hace intercambiables. En GhostTrack, cada módulo de pentesting implementa una estrategia diferente para conseguir el objetivo de obtener información sobre el objetivo. La orquestación puede combinar estrategias libremente según los parámetros recibidos en la CLI: la estrategia de escaneo básico usa los veinte módulos básicos en secuencia, la estrategia Red Team usa los veinte módulos avanzados, y la estrategia de módulo individual usa un único módulo elegido por el usuario (Gamma et al., 1994).

5.3.3 Patrón Template Method

El patrón Template Method define el esqueleto de un algoritmo en una clase base, dejando que las subclases implementen algunos de los pasos. En GhostTrack, el fichero `scripts/template.py` define la estructura general que deben seguir todos los scripts personalizados: una función `run(target, *args, **kwargs)` como punto de entrada obligatorio que acepta el target como primer argumento y devuelve un diccionario con los resultados. La implementación concreta de la lógica de análisis queda a cargo del desarrollador del script (Gamma et al., 1994).

5.3.4 Patrón Producer-Consumer con Thread Pool

Varios módulos de alto volumen (Port Scanner, Subdomain Finder, Directory Bruteforce, Banner Grabbing) implementan el patrón Producer-Consumer con un pool de threads para maximizar el rendimiento. El módulo principal actúa como productor, llenando una cola (`threading.Queue`) con todas las tareas a procesar. Los threads del pool actúan como consumidores, extrayendo tareas de la cola, procesándolas y almacenando los resultados en una estructura compartida protegida por un Lock (`threading.Lock`). Este patrón permite escalar el nivel de paralelismo simplemente ajustando el número de threads, sin cambiar la lógica de negocio del módulo.

5.4 Principios SOLID en GhostTrack

5.4.1 Principio de Responsabilidad Única (SRP)

El Principio de Responsabilidad Única establece que una clase debe tener una única razón para cambiar (Martin, 2003). En GhostTrack, este principio se aplica de forma rigurosa: el módulo Port Scanner solo escanea puertos y no tiene conocimiento de cómo se muestran los resultados ni de cómo se registran en el log; el módulo Reporter solo genera informes y no tiene conocimiento de cómo se obtienen los datos que informa; el módulo Logger solo registra eventos y no contiene lógica de negocio. Si cambia la forma en que se almacenan los logs, solo hay que modificar `logger.py`; si cambia el formato de los reportes HTML, solo hay que modificar `reporter.py`.

5.4.2 Principio Abierto/Cerrado (OCP)

El Principio Abierto/Cerrado establece que las entidades de software deben estar abiertas para la extensión pero cerradas para la modificación (Meyer, 1988). El sistema de inyección de scripts personalizados de GhostTrack es la implementación más directa de este principio: cualquier usuario puede añadir nueva funcionalidad al framework creando un fichero Python con la interfaz correcta y ejecutándolo con el flag `--inject`, sin necesidad de modificar ningún fichero existente del framework. El framework está abierto a la extensión (nuevos scripts) pero cerrado a la modificación (su código base no necesita cambiarse).

5.4.3 Principio de Sustitución de Liskov (LSP)

El Principio de Sustitución de Liskov establece que los objetos de una clase derivada deben poder sustituir a los objetos de la clase base sin alterar el comportamiento del programa (Liskov, 1987). Aunque GhostTrack no hace uso intensivo de la herencia (prefiere la composición), el principio se refleja en el diseño de la interfaz de los módulos: cualquier módulo que exponga la interfaz funcional estándar (una función con el nombre del tipo de análisis que acepta `target` como primer argumento y devuelve un diccionario) puede ser usado por el orquestador sin necesidad de conocer su implementación concreta.

5.4.4 Principio de Inversión de Dependencias (DIP)

El Principio de Inversión de Dependencias establece que los módulos de alto nivel no deben depender de módulos de bajo nivel; ambos deben depender de abstracciones (Martin, 2003). En GhostTrack, la clase GhostTrack (módulo de alto nivel) no importa directamente las clases internas de los módulos (como `PortScanner`, `SSLAnalyzer` o `VulnerabilityScanner`); en su lugar, invoca las funciones de interfaz de nivel de módulo (`port_scanner.scan()`, `ssl_analyzer.analyze()`, `vuln_scanner.scan()`). Esto significa que si se cambia la

implementación interna de un módulo —por ejemplo, reemplazando la clase PortScanner por una implementación más rápida basada en asyncio— el orquestador no necesita ningún cambio mientras la interfaz funcional se mantenga estable.

6. Tecnologías y Dependencias

El presente capítulo describe en detalle las tecnologías, lenguajes de programación y librerías de terceros utilizadas en el desarrollo de GhostTrack . Cada elección tecnológica responde a criterios técnicos objetivos que se justifican a continuación.

6.1 Lenguaje de Programación: Python 3.8+

Python fue seleccionado como lenguaje principal de desarrollo por un conjunto de razones que lo hacen especialmente adecuado para este tipo de proyecto. En primer lugar, Python es el lenguaje dominante en la comunidad de ciberseguridad: la gran mayoría de las herramientas de análisis, exploits, scripts de automatización y frameworks de pentesting están escritos en Python. Esta hegemonía no es casual; Python ofrece un equilibrio único entre expresividad, facilidad de lectura y capacidad de acceso de bajo nivel a las APIs del sistema operativo y los sockets de red (Lutz, 2013).

En segundo lugar, el ecosistema de librerías de Python orientadas a redes y seguridad es el más maduro disponible. Librerías como Scapy, dnspython, impacket, cryptography y requests tienen miles de contribuidores, documentación exhaustiva y años de uso en producción en entornos de seguridad reales. Esto contrasta con otros lenguajes donde las librerías equivalentes son menos maduras o tienen menor adopción en la comunidad de seguridad.

En tercer lugar, la sintaxis clara de Python facilita la escritura de código autoexplicativo y bien documentado, lo que es especialmente valioso en un proyecto académico donde la comprensión del código por parte del lector es tan importante como su funcionalidad.

El requisito mínimo de Python 3.8 responde a la necesidad de disponer de características del lenguaje que simplifican significativamente el código, como las f-strings (introducidas en 3.6), las expresiones walrus (introducidas en 3.8) y la mejora del tipado con typing (progresivamente mejorado desde 3.5). Python 3.8 es también la versión mínima soportada por varias de las librerías de terceros utilizadas.

6.2 Dependencias Principales

El fichero requirements.txt fija las versiones exactas de todas las dependencias para garantizar la reproducibilidad del entorno de desarrollo y producción. A continuación se describe cada dependencia con su justificación técnica:

Tabla 2 Dependencias de GhostTrack con justificación técnica Nota. Elaboración propia.

Paquete	Versión	Uso en GhostTrack	Justificación
requests	2.31.0	GeolP, Email Harvester, Subdomain Finder	HTTP client de alto nivel con gestión automática de sesiones, redirecciones, codificación y reintentos
beautifulsoup4	4.12.2	Web Crawler, Email Harvester	Parser HTML/XML robusto ante HTML malformado; navega el DOM como árbol de objetos Python
lxml	4.9.3	Backend de bs4	Parser C; entre 5-10x más rápido que html.parser; necesario para páginas de gran tamaño
dnspython	2.4.2	Subdomain Finder (DNS zone walk)	Librería DNS de bajo nivel; construye y parsea consultas para cualquier tipo de registro
scapy	2.5.0	Network Sniffer	Framework de manipulación de paquetes; permite captura y análisis a nivel de bytes
cryptography	41.0.5	SSL Analyzer (cert X.509)	Librería criptográfica de referencia; parsea estructuras ASN.1 de certificados
pyOpenSSL	23.3.0	SSL Analyzer (conexiones)	Bindings Python para OpenSSL; necesario para probar versiones de protocolo deprecadas
Pillow	10.1.0	Metadata Extractor (EXIF de imágenes)	Fork activo de PIL; acceso a metadatos EXIF de JPEG, PNG, TIFF
PyPDF2	3.0.1	Metadata Extractor (metadatos PDF)	Extracción del diccionario de información de documentos PDF
Jinja2	3.1.2	Reporter (plantillas HTML)	Motor de plantillas; separa lógica de presentación en la generación de reportes
colorama	0.4.6	Colors (soporte Windows)	Inicializa el soporte de códigos ANSI en terminales Windows
tqdm	4.66.1	Barras de progreso	Indicador de progreso no invasivo para operaciones de larga duración
python-dotenv	1.0.0	Carga de API keys	Inyección de variables de entorno desde fichero .env
click	8.1.7	CLI (extensiones futuras)	Framework de CLI para comandos anidados y auto-documentados
pytest	7.4.3	Tests unitarios	Framework de testing de referencia con fixtures y parametrización
pytest-cov	4.1.0	Cobertura de código	Integración de coverage.py con pytest

6.3 Dependencias Opcionales Red Team

Las siguientes dependencias están comentadas en requirements.txt porque son requeridas solo por los módulos Red Team más avanzados y pueden presentar dificultades de instalación en algunos sistemas (requieren compilación de extensiones nativas en C o tienen restricciones de uso en algunos países):

- `impacket==0.11.0`: Colección de clases Python para trabajar con protocolos de red Windows (SMB, Kerberos, NTLM, LDAP). Es la dependencia principal de los módulos `SMBEnumerator`, `KerberosAttacker`, `NTLMRelayAttacker` y `CredentialDumper`.
- `ldap3==2.9.1`: Librería LDAP pura en Python para interactuar con servidores Active Directory. Requerida por `LDAPEnumerator` y `BloodHoundCollector`.
- `pycryptodome==3.19.0`: Implementación de primitivas criptográficas en Python puro. Requerida por varios módulos Red Team para el cifrado y descifrado de material criptográfico Kerberos y NTLM.

La instalación de estas dependencias opcionales se realiza de forma separada cuando se necesitan los módulos Red Team avanzados:

```
pip install impacket ldap3 pycryptodome
```

7. Módulos Básicos — Descripción Técnica Detallada

Los veinte módulos básicos de GhostTrack cubren las fases de reconocimiento, enumeración y análisis de vulnerabilidades de un ejercicio de pentesting estándar. Cada módulo está implementado como un fichero Python independiente en el directorio `modules/`, sigue una interfaz funcional estandarizada y puede ser invocado tanto desde el orquestador principal como de forma autónoma desde la línea de comandos para pruebas individuales.

La convención de nomenclatura de los módulos sigue el patrón `sustantivo_acción` o `sustantivo_descriptor`, de forma que el nombre del fichero sea autoexplicativo en cuanto a la funcionalidad que implementa. Las clases internas siguen la convención `PascalCase` con un sufijo descriptivo de su rol (`Scanner`, `Analyzer`, `Grabber`, `Crawler`, etc.).

7.1 Módulo 01: GeoIP — Geolocalización de Direcciones IP

Descripción funcional

El módulo `GeoIP` implementa la geolocalización de direcciones IP y nombres de dominio, obteniendo información geográfica (país, región, ciudad, coordenadas) y de red (ISP, organización, número de sistema autónomo) que resulta de gran utilidad en la fase de reconocimiento. Conocer la ubicación física y el proveedor de servicios del objetivo permite, entre otras cosas, identificar si el servidor está alojado en un datacenter de un proveedor cloud o en infraestructura propia, lo que tiene implicaciones directas sobre las técnicas de ataque aplicables y sobre los posibles controles de seguridad presentes.

Implementación técnica

La función principal `get_location()` implementa el siguiente flujo de ejecución: en primer lugar, determina si el target proporcionado es ya una dirección IP mediante `socket.inet_aton()`; si no lo es, resuelve el nombre de dominio a IP mediante `socket.gethostbyname()`, que realiza una consulta DNS estándar usando el servidor configurado en el sistema operativo. Una vez obtenida la IP, construye la URL de consulta a la API pública de `ip-api.com` en formato JSON, que devuelve hasta quince campos de información geográfica y de red sin necesidad de autenticación (límite de 45 consultas por minuto por IP).

El módulo también implementa dos fuentes alternativas de información: `get_geolocation_alternative()` usa la API de `ipapi.co` como fuente secundaria, y `get_asn_info()` consulta la API de `hackertarget.com` para obtener información detallada del ASN (Autonomous System Number), incluyendo el rango de prefijos IP gestionados por el sistema

autónomo al que pertenece la IP objetivo. Esta información es especialmente útil para identificar el rango completo de IPs que podrían estar bajo el control del mismo propietario que el objetivo.

La función `get_reverse_dns()` realiza una consulta de DNS inverso mediante `socket.gethostbyaddr()`, que traduce una dirección IP a su nombre de host canónico si existe un registro PTR configurado. Esta información puede revelar el nombre interno del servidor, que a menudo contiene información sobre su función (`db-prod-01.empresa.com` revela que es una base de datos en producción) o su pertenencia a un rango de IPs de un datacenter concreto.

Estructura de datos devuelta

La función `get_location()` devuelve un diccionario con los siguientes campos cuando la consulta tiene éxito:

```
{'ip': '93.184.216.34',  
  'country': 'United States',  
  'country_code': 'US',  
  'region': 'Massachusetts',  
  'city': 'Norwell',  
  'zip': '02061',  
  'lat': 42.1596,  
  'lon': -70.8217,  
  'timezone': 'America/New_York',  
  'isp': 'Edgecast Inc.',  
  'org': 'EDGECAST',  
  'as': 'AS15133 MCI Communications Services'}
```

Interfaz pública del módulo

```
get_location(target: str) -> Dict  
resolve_to_ip(target: str) -> str  
get_reverse_dns(ip: str) -> Optional[str]  
get_asn_info(ip: str) -> Dict  
get_geolocation_alternative(ip: str) -> Dict
```

Consideraciones de uso

El módulo GeoIP es completamente pasivo: no establece ninguna conexión con el sistema objetivo, sino con APIs de terceros que proporcionan información pública. Esto lo convierte en el módulo más seguro desde el punto de vista de la detección, ya que no genera ningún tráfico hacia el objetivo. Todas las consultas se hacen a servicios externos (`ip-api.com`, `ipapi.co`, `hackertarget.com`) que son servicios legítimos y públicos, por lo que su uso no requiere ninguna autorización especial más allá de la propia del ejercicio de pentesting.

7.2 Módulo 02: WHOIS Lookup — Consulta de Registros de Dominio

Descripción funcional

El protocolo WHOIS, definido en el RFC 3912 (Daigle, 2004), permite consultar las bases de datos de registro de dominios e IPs para obtener información sobre su propietario, fechas de registro, servidores de nombres y estado. Esta información es fundamental en la fase de reconocimiento porque permite identificar al propietario del dominio objetivo, descubrir correos electrónicos de contacto que pueden ser objetivo de phishing, conocer cuándo fue registrado el dominio (los dominios registrados recientemente son a menudo infraestructura maliciosa) y determinar los servidores de nombres autoritativos para el dominio.

Implementación técnica

La clase WHOISLookup implementa el protocolo WHOIS directamente mediante sockets TCP, sin dependencias de librerías de terceros, lo que garantiza la portabilidad del módulo. El proceso de consulta es el siguiente: la clase determina el servidor WHOIS apropiado consultando el diccionario interno WHOIS_SERVERS, que mapea TLDs (Top Level Domains) a sus servidores WHOIS correspondientes. Para el TLD .com y .net, el servidor es whois.verisign-grs.com; para .org, whois.pir.org; para .es, whois.nic.es; y así sucesivamente. Para TLDs no incluidos en el diccionario, se usa whois.iana.org como servidor de fallback, que proporciona la referencia al servidor correcto.

La conexión se establece mediante socket.socket() al puerto 43/TCP, que es el puerto estándar del protocolo WHOIS según el RFC 3912. La consulta se envía como texto ASCII seguido de CRLF, y la respuesta se acumula en un buffer hasta que el servidor cierra la conexión. La respuesta, que es texto plano no estructurado, varía en formato entre diferentes registradores y TLDs, lo que hace necesario el uso de expresiones regulares flexibles para la extracción de campos.

El método _parse_whois() aplica expresiones regulares que cubren las variantes de nomenclatura más comunes: tanto 'Creation Date' como 'Created' y 'Registration Date' se capturan con la misma expresión; tanto 'Expiration Date' como 'Expiry Date' y 'Registry Expiry Date' son equivalentes. Esta flexibilidad es necesaria porque no existe un formato estándar de respuesta WHOIS.

Extracción de campos

- Registrador: Organización que gestiona el registro del dominio.
- Fecha de creación: Cuándo fue registrado por primera vez el dominio.
- Fecha de expiración: Cuándo vence el registro; los dominios cercanos a la expiración pueden estar en riesgo de hijacking.
- Fecha de última actualización: Cambios recientes pueden indicar transferencias de propiedad o actualizaciones de configuración.
- Name servers: Servidores DNS autoritativos para el dominio.
- Estado del dominio: Locks aplicados (clientTransferProhibited, serverDeleteProhibited, etc.).
- Emails: Contactos técnicos y administrativos del dominio.
- Organización: Empresa o persona titular del dominio.

Interfaz pública del módulo

```
lookup(target: str) -> Dict
bulk_lookup(targets: list) -> Dict[str, Dict]
```

7.3 Módulo 03: DNS Enumeration — Enumeración de Registros DNS

Descripción funcional

La enumeración DNS es una de las técnicas más productivas en la fase de reconocimiento. El DNS (Domain Name System) es la infraestructura que mapea nombres de dominio a recursos de red, y su consulta exhaustiva permite construir un mapa detallado de la infraestructura de una organización: qué servidores de correo usa (registros MX), qué servicios expone (registros SRV), qué información de seguridad de correo tiene configurada (registros TXT con SPF, DKIM y DMARC), qué direcciones IP tiene asociadas (registros A y AAAA) y cuáles son sus servidores de nombres autoritativos (registros NS).

Más allá de los registros estándar, la comprobación de transferencias de zona (AXFR) es una de las técnicas de mayor impacto en esta fase. Una transferencia de zona exitosa, posible cuando un servidor DNS no está correctamente configurado para limitar las transferencias a servidores secundarios autorizados, revela la totalidad del árbol DNS del dominio en una única consulta, incluyendo subdominios internos, direcciones IP privadas y otros recursos que no deberían ser públicamente accesibles.

Tipos de registros enumerados

La clase DNSEnumerator enumera sistemáticamente los siguientes diez tipos de registros DNS:

- Registro A: Mapeo de nombre de dominio a dirección IPv4. Es el registro más fundamental y el que revela las IPs directas del objetivo.
- Registro AAAA: Mapeo de nombre de dominio a dirección IPv6. Muchas organizaciones tienen recursos accesibles solo por IPv6 que no se descubren si solo se consultan registros A.
- Registro MX: Servidores de correo electrónico del dominio. Revela qué proveedor de email usa la organización (Google Workspace, Microsoft 365, servidores propios) y puede ser punto de entrada para ataques de phishing.
- Registro NS: Servidores de nombres autoritativos. Es el primer paso para el intento de transferencia de zona.
- Registro TXT: Registros de texto con múltiples usos: política SPF (qué servidores están autorizados a enviar email en nombre del dominio), claves públicas DKIM, verificación de propiedad para servicios cloud, y a veces información sensible dejada por administradores.
- Registro CNAME: Alias de dominio. Puede revelar servicios internos o externos que están detrás del nombre principal.
- Registro SOA: Start of Authority. Contiene información sobre el servidor DNS primario, el email del administrador (a menudo ofuscado pero recuperable) y los parámetros de refresco del zone.
- Registro PTR: DNS inverso. Permite obtener el nombre de host para una IP, útil para mapear rangos de IP a organizaciones.
- Registro SRV: Localización de servicios. Usado por protocolos como SIP, XMPP y Active Directory para anunciar la ubicación de servicios específicos.
- Registro CAA: Certification Authority Authorization. Especifica qué CAs están autorizadas a emitir certificados SSL para el dominio; puede revelar qué proveedor de certificados usa la organización.

Intento de transferencia de zona (AXFR)

El intento de transferencia de zona se realiza contra cada servidor NS descubierto mediante el comando dig con la opción AXFR. El módulo detecta el éxito de la transferencia comprobando si la respuesta contiene la cadena 'XFR size' (que indica cuántos registros se han transferido) o si el número de líneas supera el umbral de una respuesta típica de fallo. Si la transferencia tiene éxito, los datos completos del zone se almacenan en el resultado y se

presenta un aviso de severidad alta, ya que la exposición pública del zone DNS completo es una vulnerabilidad de configuración crítica.

Interfaz pública del módulo

```
enumerate(domain: str) -> Dict
check_dnssec(domain: str) -> bool
find_subdomains_dns(domain: str, wordlist: List[str]) -> List[str]
```

7.4 Módulo 04: Subdomain Finder — Descubrimiento de Subdominios

Descripción funcional

El descubrimiento de subdominios amplía significativamente la superficie de ataque visible, ya que muchas organizaciones tienen subdominios de desarrollo, staging, administración o servicios internos accidentalmente expuestos a Internet que no aparecen en el sitio web principal. Subdominios como dev.empresa.com, staging.empresa.com, admin.empresa.com o git.empresa.com son objetivos frecuentes de ataques porque suelen tener controles de seguridad menos rigurosos que los sistemas de producción principales.

Técnicas implementadas

La clase SubdomainFinder implementa tres técnicas complementarias que se ejecutan secuencialmente:

La técnica de brute force DNS combina cada palabra de la wordlist con el dominio base y realiza una consulta de resolución mediante `socket.gethostbyname()`. Si la resolución tiene éxito (es decir, el nombre existe en el DNS), el subdominio existe y se añade al conjunto de resultados. Esta técnica es simple pero efectiva: la wordlist interna de 60 palabras cubre los subdominios más comunes en organizaciones de cualquier tamaño. El uso de multi-threading con Queue y Lock maximiza el rendimiento sin superar el límite de threads configurado.

La técnica de Certificate Transparency aprovecha el ecosistema de logs públicos de transparencia de certificados SSL/TLS. Desde 2013, todas las CAs participantes en los navegadores principales están obligadas a publicar cada certificado que emiten en logs de transparencia públicos. Esto significa que cualquier subdominio que haya tenido alguna vez un certificado SSL aparece en estos logs. La API pública de crt.sh proporciona acceso de búsqueda a estos logs, devolviendo en formato JSON todos los certificados emitidos para un dominio y sus subdominios, incluyendo wildcards. El módulo filtra los resultados para eliminar wildcards (*.dominio.com) y subdominios que ya han sido descubiertos por otras técnicas.

La técnica de búsqueda en motores construye queries de Google Dorking (específicamente `site:.dominio.com -www`) que permiten encontrar subdominios indexados por motores de búsqueda pero no accesibles directamente. En la implementación actual, el módulo genera estas queries para ejecución manual, dado que la ejecución automatizada requeriría integración con APIs de pago.

Wordlist interna

La wordlist interna `COMMON_SUBDOMAINS` incluye 60 subdominios de alta frecuencia organizados por categorías funcionales: infraestructura (`www`, `ftp`, `mail`, `smtp`, `pop`, `ns1`, `ns2`), colaboración (`git`, `gitlab`, `jenkins`, `jira`, `confluence`, `wiki`), operaciones (`admin`, `administrator`, `cpanel`, `whm`), desarrollo (`dev`, `development`, `staging`, `beta`, `alpha`, `demo`, `test`, `qa`, `uat`), servicios cloud (`cdn`, `static`, `assets`, `media`, `api`), y monitorización (`monitor`, `monitoring`, `stats`, `status`).

Interfaz pública del módulo

```
find(domain: str, wordlist: List[str], threads: int) -> Set[str]
load_wordlist(file_path: str) -> List[str]
export_results(subdomains: Set[str], output_file: str)
```

7.5 Módulo 05: Port Scanner — Escaneo de Puertos TCP/UDP

Descripción funcional

El escaneo de puertos es la técnica más fundamental de la enumeración de red. Permite determinar qué servicios tiene expuestos un host y, por extensión, qué protocolos de ataque son aplicables. Un puerto abierto implica que hay un servicio escuchando, lo que a su vez implica posibles vectores de ataque: vulnerabilidades en el servicio, credenciales por defecto, protocolos no cifrados o servicios de administración mal configurados. GhostTrack implementa un escáner de puertos TCP multi-threaded de alta velocidad basado en TCP Connect Scan.

TCP Connect Scan vs. SYN Scan

Existen varias técnicas de escaneo de puertos TCP. La más utilizada por herramientas profesionales como Nmap es el SYN Scan (también llamado half-open scan o stealth scan), que envía un segmento TCP SYN al puerto objetivo y espera la respuesta: si responde con SYN-ACK, el puerto está abierto; si responde con RST, está cerrado. El SYN Scan nunca completa el three-way handshake, lo que lo hace más sigiloso y más rápido, pero requiere sockets raw y privilegios de root.

GhostTrack implementa TCP Connect Scan, que sí completa el three-way handshake completo usando la API de sockets estándar del sistema operativo (`socket.connect_ex()`). Esta elección de diseño tiene dos ventajas: no requiere privilegios especiales, por lo que puede ejecutarse como usuario normal, y es completamente portable entre sistemas operativos (Windows, Linux, macOS). La desventaja es que genera más tráfico (el handshake completo) y es más fácilmente detectable por sistemas IDS/IPS que el SYN Scan.

Implementación del pool de threads

El escáner implementa el patrón Producer-Consumer: la función `scan()` llena una Queue con todos los puertos a escanear, y los threads trabajadores van extrayendo puertos de la cola, intentando la conexión y añadiendo el puerto a la lista `open_ports` si la conexión tiene éxito (`connect_ex()` devuelve 0). El Lock protege la lista `open_ports` de condiciones de carrera cuando múltiples threads intentan añadir resultados simultáneamente. El número de threads se limita automáticamente al mínimo entre el número configurado y el número de puertos a escanear, evitando crear más threads de los necesarios.

Lista de puertos estándar

La lista `COMMON_PORTS` incluye 36 puertos seleccionados por su relevancia en pentesting:

- Servicios de transferencia de ficheros: 20/21 (FTP), 69 (TFTP).
- Acceso remoto: 22 (SSH), 23 (Telnet), 3389 (RDP), 5900 (VNC), 1723 (PPTP VPN).
- Servicios web: 80, 443, 8080, 8443, 8888, 3000, 5000.
- Correo electrónico: 25 (SMTP), 110 (POP3), 143 (IMAP), 993 (IMAPS), 995 (POP3S).
- Bases de datos: 1433 (MSSQL), 1521 (Oracle), 3306 (MySQL), 5432 (PostgreSQL), 6379 (Redis), 27017 (MongoDB).
- Servicios Windows: 135 (MSRPC), 139 (NetBIOS), 445 (SMB), 111 (RPC).
 - Monitorización y búsqueda: 9200/9300 (Elasticsearch), 5601 (Kibana), 11211 (Memcached).

Modos de escaneo disponibles

- `scan()`: Escaneo de la lista `COMMON_PORTS` configurada.
- `quick_scan()`: Los 20 puertos más críticos.
- `full_scan()`: Los 65535 puertos TCP completos.
- `scan_range(start, end)`: Rango específico de puertos.
- `scan_udp(target, port)`: Escaneo de un puerto UDP individual.

Interfaz pública del módulo

```
scan(target: str, port_range: str, threads: int) -> List[int]
parse_port_range(port_range: str) -> List[int]
get_open_ports_summary(target: str, open_ports: List[int]) -> Dict
```

7.6 Módulo 06: Service Detection — Detección de Servicios y Versiones

Descripción funcional

Identificar qué puerto está abierto es solo el primer paso. Saber qué servicio concreto y qué versión corre detrás de cada puerto es la información que permite evaluar la exposición real a vulnerabilidades. Un servidor web Apache 2.4.49 es crítico (CVE-2021-41773); un servidor Apache 2.4.51 no lo es. Sin información de versión, el análisis de vulnerabilidades solo puede ser aproximado. El módulo Service Detection combina el diccionario de servicios del sistema operativo con el análisis activo de banners para obtener la información más precisa posible.

Implementación técnica

Para cada puerto abierto recibido como entrada, la clase ServiceDetector ejecuta las siguientes operaciones en secuencia. Primero, consulta el nombre del servicio estándar asociado al puerto mediante `socket.getservbyport()`, que accede a la base de datos de servicios del sistema operativo (`/etc/services` en Unix). Este diccionario se complementa con el mapa interno `COMMON_SERVICES`, que cubre específicamente los servicios más relevantes en pentesting que a veces no están en la base de datos del sistema.

Segundo, envía una sonda activa al servicio. El módulo mantiene un diccionario `SERVICE_PROBES` que define qué enviar al conectar para provocar una respuesta del servicio: para HTTP envía `'GET / HTTP/1.0\r\n\r\n'`, para SMTP envía `'EHLO test\r\n'`. Para servicios que envían banner espontáneamente al conectar (SSH, FTP, MySQL, Redis, POP3) basta con esperar la respuesta inicial sin enviar nada.

Tercero, el banner recibido se analiza con expresiones regulares específicas por protocolo mediante el método `parse_version()`. Para HTTP, extrae el header `Server` de la respuesta. Para SSH, parsea el string de negociación de protocolo en formato `'SSH-[version]-[software]'` (por ejemplo `'SSH-2.0-OpenSSH_8.4'`). Para FTP, captura el mensaje de bienvenida del código de respuesta 220. Para cada protocolo, la versión extraída se añade al diccionario de resultados.

Interfaz pública del módulo

```
detect(target: str, ports: List[int]) -> Dict
```

7.7 Módulo 07: OS Fingerprinting — Identificación del Sistema Operativo

Descripción funcional

La identificación del sistema operativo del objetivo permite filtrar los vectores de ataque aplicables. Las vulnerabilidades, técnicas de escalada de privilegios, herramientas de post-explotación y mecanismos de persistencia son completamente distintos en Linux y Windows. Conocer el sistema operativo permite al auditor priorizar los módulos Red Team y las técnicas de explotación más relevantes.

Técnica 1: Análisis de TTL

El campo TTL (Time To Live) del protocolo IP indica el número máximo de saltos de red que puede dar un paquete antes de ser descartado. El valor TTL inicial varía según el sistema operativo: los sistemas Linux utilizan típicamente un TTL inicial de 64; los sistemas Windows, de 128; los dispositivos de red Cisco, de 255; y los sistemas macOS, de 64 (con algunas variaciones según la versión). El módulo lanza un ping al objetivo y extrae el TTL de la respuesta mediante expresión regular. Dado que cada salto de red decrementa el TTL en uno, si el TTL observado es 57 en lugar de 64, se puede deducir que hay 7 saltos entre el auditor y el objetivo con un TTL inicial de 64, lo que corresponde a un sistema Linux/Unix. Esta técnica tiene nivel de confianza medio porque el TTL inicial puede haber sido modificado por el administrador del sistema.

Técnica 2: Análisis de banners de servicios

Los banners de servicios comunes incluyen frecuentemente información directa sobre el sistema operativo o distribución. El banner SSH de un servidor Ubuntu típicamente incluye 'Ubuntu-' en el campo de versión del software OpenSSH. Los servidores HTTP Microsoft IIS incluyen 'Microsoft-IIS' en el header Server. Los servidores FTP de algunos sistemas Unix incluyen 'Unix' en el mensaje de bienvenida. El módulo comprueba los puertos 22, 80 y 443 y busca estas cadenas identificadoras con un peso mayor en el sistema de votación (peso 2 frente a peso 1 del análisis TTL) porque la información explícita en un banner es más fiable que la inferencia por TTL.

Sistema de votación ponderada

Los resultados de las diferentes técnicas se consolidan mediante un sistema de votación ponderada implementado en el método `determine_os()`. Cada técnica contribuye con un número de 'votos' al sistema operativo que identifica, con pesos distintos según su fiabilidad. El sistema operativo con más votos ponderados se devuelve como resultado final con una indicación del nivel de confianza (bajo, medio o alto).

7.8 Módulo 08: Banner Grabbing — Captura de Banners de Servicios

Descripción funcional

El Banner Grabbing es la técnica de capturar los mensajes de bienvenida que los servicios de red envían al establecerse una conexión. Estos mensajes, denominados banners, a menudo revelan información sensible que los administradores deberían ocultar o suprimir: nombre y versión exacta del software servidor, sistema operativo del host, configuración interna del servidor y a veces incluso el nombre o dominio interno del sistema.

Aunque el módulo Service Detection también captura banners como parte de su proceso, el módulo Banner Grabbing está diseñado para capturar banners de forma más exhaustiva y sistemática, incluyendo el soporte completo para conexiones SSL/TLS, el uso de sondas específicas por protocolo y el procesamiento concurrente de múltiples puertos.

Sondas por protocolo

El diccionario `SERVICE_PROBES` define exactamente qué enviar al conectar a cada puerto para provocar la respuesta más informativa del servicio:

- Puerto 21 (FTP): No se envía nada; el servidor FTP envía su banner al conectar espontáneamente.
- Puerto 22 (SSH): No se envía nada; el servidor SSH envía inmediatamente la línea de negociación de protocolo.
- Puerto 25 (SMTP): Se envía 'EHLO banner\r\n' para solicitar las capacidades del servidor; la respuesta incluye el banner completo con todas las extensiones soportadas.
- Puerto 80 (HTTP): Se envía 'GET / HTTP/1.0\r\n\r\n'; la respuesta incluye los headers completos con Server, X-Powered-By y otros headers informativos.
- Puerto 443 (HTTPS): Se envía la misma petición HTTP pero sobre conexión TLS.

- Puerto 3306 (MySQL): No se envía nada; MySQL envía su greeting packet al conectar, que incluye la versión exacta del servidor.
- Puerto 6379 (Redis): Se envía 'INFO\r\n'; Redis responde con información detallada sobre su configuración y versión.

Soporte SSL/TLS

El método `_grab_ssl_banner()` implementa la captura de banners sobre conexiones cifradas. Crea un contexto SSL con `check_hostname=False` y `verify_mode=CERT_NONE` para poder conectar a servidores con certificados autofirmados o expirados sin que se lance una excepción. Tras establecer la conexión TLS, el método recupera adicionalmente información sobre la negociación: versión del protocolo TLS utilizada (TLSv1.2, TLSv1.3), nombre y fuerza del cipher suite negociado.

7.9 Módulo 09: SSL/TLS Analyzer — Análisis de Seguridad SSL/TLS

Descripción funcional

El análisis de la configuración SSL/TLS va mucho más allá de comprobar si un puerto 443 responde. Una configuración inadecuada expone el servidor a ataques criptográficos graves. POODLE (Padding Oracle On Downgraded Legacy Encryption) ataca SSLv3 y permite descifrar datos en tránsito. BEAST (Browser Exploit Against SSL/TLS) afecta a TLSv1.0 con ciertos modos de cifrado. DROWN (Decrypting RSA with Obsolete and Weakened eNcryption) permite descifrar tráfico TLS moderno usando un servidor SSLv2 que comparte la misma clave privada. Los certificados expirados interrumpen el servicio. Los certificados autofirmados no proporcionan garantías de identidad. Todo esto es evaluado por el módulo SSL/TLS Analyzer.

Análisis del certificado X.509

El módulo recupera el certificado del servidor en formato binario DER mediante la API de `ssl` de Python. A continuación, utiliza la librería `cryptography` para parsear la estructura ASN.1 del certificado y extraer sus campos. La librería `cryptography` proporciona acceso directo a todos los campos del certificado X.509 v3, incluyendo el Subject (titular del certificado), el Issuer (CA que lo emitió), la fecha de inicio y fin de validez, el número de serie, el algoritmo de firma y las extensiones X.509 v3.

Un campo especialmente relevante es la extensión Subject Alternative Names (SANs), que lista todos los nombres de dominio para los que el certificado es válido. En

organizaciones grandes, el certificado puede contener docenas de SANs, lo que revela subdominios internos y externos asociados a la organización que no serían visibles de otra forma.

Detección de vulnerabilidades SSL

El módulo implementa pruebas para las siguientes vulnerabilidades conocidas de configuración SSL/TLS:

- POODLE: Comprueba si SSLv3 está soportado. Si es así, el servidor es potencialmente vulnerable a POODLE.
- BEAST: Comprueba si TLSv1.0 está soportado. Si es así, y dependiendo del cipher suite usado, puede ser vulnerable a BEAST.
- Certificado expirado: Compara la fecha de expiración del certificado con la fecha actual. Un certificado expirado causa errores en el navegador y puede indicar descuido en la gestión de la infraestructura.
- Certificado próximo a expirar: Se avisa cuando el certificado expira en menos de 30 días.
- Protocolos débiles habilitados: Cualquier versión anterior a TLSv1.2 se considera débil según las recomendaciones actuales de NIST y BSI.

7.10 Módulo 10: Vulnerability Scanner — Escáner de Vulnerabilidades

Descripción funcional

El escáner de vulnerabilidades constituye el núcleo del análisis de riesgo de GhostTrack. Toma como entrada el mapa de servicios generado por los módulos anteriores y lo cruza con una base de datos de vulnerabilidades conocidas para identificar versiones de software afectadas por CVEs documentados y configuraciones inseguras. Este módulo es el que permite pasar de un inventario de servicios a una lista priorizada de riesgos de seguridad.

Base de datos de CVEs

La base de datos interna CVE_DATABASE está implementada como un diccionario Python que mapea cadenas de identificación de software/versión a registros de CVE con campos de severidad según el sistema CVSS, descripción textual y puntuación numérica. Los registros incluidos son representativos de vulnerabilidades reales y conocidas:

- CVE-2021-41773 en Apache HTTP Server 2.4.49: Path Traversal y Remote Code Execution (CVSS 9.8 Critical). Esta vulnerabilidad permitía a atacantes no autenticados acceder a ficheros fuera del directorio raíz del servidor y ejecutar código arbitrario si `mod_cgi` estaba habilitado.
- CVE-2018-15473 en OpenSSH 7.4: Enumeración de usuarios válidos (CVSS 5.3 Medium). Permite a atacantes remotos determinar si un nombre de usuario existe en el sistema objetivo mediante diferencias de tiempo en las respuestas del servidor.
- CVE-2019-9511 en nginx 1.14: Vulnerabilidades de implementación de HTTP/2 que permiten ataques de denegación de servicio (CVSS 7.5 High).

Detección de misconfiguraciones

Paralelamente a la detección por CVE, el módulo implementa un sistema de detección de misconfiguraciones que identifica configuraciones inseguras independientemente de las versiones de software. Las comprobaciones incluyen: puertos con protocolos que transmiten credenciales en texto claro (21 FTP, 23 Telnet, 69 TFTP), servicios HTTP sin redirección a HTTPS en el puerto 80, bases de datos expuestas directamente en la red sin autenticación aparente, y servicios de administración accesibles desde redes no confiables.

Presentación de resultados

Los resultados se agrupan por severidad (CRITICAL, HIGH, MEDIUM, LOW) y se presentan con un resumen ejecutivo que incluye el conteo de hallazgos por nivel, seguido del detalle de cada vulnerabilidad con su CVE, descripción, puntuación CVSS y servicio afectado. El sistema de colores ANSI codifica visualmente la severidad: rojo para CRITICAL, amarillo para HIGH, azul para MEDIUM y verde para LOW.

7.11 Módulo 11: Traceroute — Trazado de Ruta de Red

Descripción funcional

El traceroute permite descubrir la ruta de red completa entre el sistema del auditor y el objetivo, identificando todos los routers intermedios con sus IPs, hostnames y tiempos de respuesta. Es útil para mapear la topología del objetivo, identificar dispositivos de filtrado intermedios y detectar anomalías de enrutamiento que pueden indicar medidas de seguridad activas.

Implementación técnica en GhostTrack

La clase Traceroute en `modules/traceroute.py` detecta el sistema operativo con `platform.system()` y elige el comando correspondiente: `'traceroute -m {max_hops} {target}'` en Linux/macOS o `'tracert -h {max_hops} {target}'` en Windows. Lo ejecuta con `subprocess.check_output(cmd, shell=True, stderr=subprocess.STDOUT)` y decodifica la salida en UTF-8.

La salida de texto se procesa línea a línea con la expresión regular `r'(\d+)\s+([0-9\.\*])\s+(?:\[([^\]]+)\])?\s*(\d+\.\d*)\s*ms'` que extrae cuatro grupos: número de salto, IP del router, hostname entre corchetes cuando está disponible, y RTT en milisegundos. Los saltos con timeout ('***) no coinciden con el patrón y se omiten automáticamente del resultado estructurado.

Cada salto se almacena como diccionario con los campos `hop` (int), `ip` (str), `hostname` (str o None) y `rtt` (float). La presencia de saltos con IPs privadas (10.x.x.x, 172.16-31.x.x, 192.168.x.x) en la ruta puede revelar la topología interna si los routers de borde no filtran estas respuestas correctamente.

Interfaz pública

```
trace(target: str, max_hops: int = 30) -> List[Dict]
# Salida: [{'hop':1,'ip':'192.168.1.1','hostname':'router','rtt':1.23}, ...]
```

7.12 Módulo 12: Web Crawler — Rastreo de Aplicaciones Web

Descripción funcional

El crawler web mapea automáticamente la estructura completa de una aplicación web visitando todas las páginas accesibles y extrayendo URLs, formularios y emails. El resultado alimenta directamente los módulos de análisis de vulnerabilidades (SQLi, XSS, Directory BF) que necesitan conocer los endpoints y parámetros de entrada antes de poder analizarlos.

Implementación técnica en GhostTrack

La clase WebCrawler en `modules/web_crawler.py` implementa el rastreo con el método `_crawl_recursive()`, que se llama a sí mismo incrementando el nivel de profundidad. Antes de visitar una URL comprueba dos condiciones de parada: que `depth` no supere `max_depth` (3 por defecto) y que `len(self.visited_urls)` no supere `max_pages` (50 por defecto). La URL se añade a `self.visited_urls` (set) antes de procesarla, previniendo bucles de redirección.

Para cada URL con código 200, el método `_extract_links()` instancia BeautifulSoup con parser 'html.parser', busca todos los elementos con tags 'a' y 'link' que tengan atributo href, y convierte cada href a URL absoluta con `urllib.parse.urljoin(base_url, href)`. El método `_extract_emails()` aplica la regex `r'[a-zA-Z0-9._%+~]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}'` sobre `response.text` completo. El método `_extract_forms()` busca todos los tags form y extrae action, method, y una lista de dicts con type/name/value de cada input y textarea.

La restricción de dominio se implementa comparando `urlparse(link).netloc` con `urlparse(self.target).netloc`: solo los enlaces del mismo dominio se rastrean recursivamente. Los externos se añaden a `found_urls` para el reporte pero no se visitan. Esta restricción es fundamental para limitar el alcance al sistema auditado.

Valor de los formularios para módulos posteriores

Los formularios extraídos son la información más valiosa del crawler porque contienen los parámetros de entrada de la aplicación. SQLInjectionTester y XSSScanner los usan como lista de endpoints a analizar, aumentando significativamente la cobertura respecto a probar solo los parámetros visibles en la barra de direcciones.

Interfaz pública

```
crawl(target: str, max_depth: int = 3, max_pages: int = 50) -> Dict
# {'total_urls':47, 'urls':[...], 'emails':[...],
#  'forms': [{'url': '...', 'action': '/login', 'method': 'POST',
#              'inputs': [{'type': 'text', 'name': 'user'}]}], 'visited':23}
```

7.13 Módulo 13: Directory Bruteforce — Descubrimiento de Rutas Ocultas

Descripción funcional

El descubrimiento de directorios y ficheros ocultos permite encontrar recursos no enlazados públicamente: paneles de administración (/admin), herramientas de base de datos (/phpmyadmin), credenciales en ficheros de configuración (/env), o repositorios de código fuente expuestos (/git). Su descubrimiento puede suponer el acceso completo a la aplicación o al servidor subyacente.

Implementación técnica en GhostTrack

La clase DirectoryBruteforcer en `modules/directory_bruteforce.py` inicializa el target añadiendo 'http://' si no tiene protocolo. El método `bruteforce()` llena una `threading.Queue` con todas las entradas de la wordlist y lanza un pool de threads con

`threading.Thread(target=self._worker)`, limitando el número al mínimo entre threads y `len(self.wordlist)` para no crear threads innecesarios.

Cada worker extrae una entrada con `self.queue.get()`, construye la URL como `f"{self.target.rstrip('/')}/{directory.lstrip('/')}"` (el doble strip elimina barras duplicadas) y envía `requests.get(url, timeout=5, allow_redirects=False)`. El parámetro `allow_redirects=False` es crítico: sin él una redirección 301 a la portada devolvería 200 y generaría falsos positivos. Los códigos 200, 301, 302, 401 y 403 indican existencia del recurso. Para cada hallazgo se registra `url`, `status_code`, `size` (`len(response.content)`) y `redirect` (`response.headers.get('Location')`). `threading.Lock` protege `self.found_dirs`.

La wordlist interna `COMMON_DIRS` contiene 50 rutas de alta frecuencia: administración (`admin`, `dashboard`, `panel`, `cpanel`), WordPress (`wp-admin`, `wp-content`, `wp-login.php`), bases de datos (`phpmyadmin`, `pma`), ficheros sensibles (`.env`, `.git`, `.svn`, `.htaccess`, `web.config`), y APIs (`api`, `v1`, `v2`). El código 403 merece especial atención: confirma implícitamente la existencia del recurso aunque el servidor lo deniegue, siendo candidato a técnicas de bypass de control de acceso.

Función auxiliar load_wordlist

`load_wordlist(file_path)` abre el fichero y devuelve cada línea limpia de espacios. Si el fichero no existe devuelve lista vacía y el módulo continúa con `COMMON_DIRS`, garantizando que el módulo nunca falla por una wordlist externa faltante.

Interfaz pública

```
bruteforce(target: str, wordlist: List[str] = None, threads: int = 10) ->
List[Dict]
load_wordlist(file_path: str) -> List[str]
```

7.14 Módulo 14: SQL Injection Scanner — Detección de Inyecciones SQL

Descripción funcional

La inyección SQL (OWASP A03:2021) permite manipular las consultas SQL de la aplicación enviando datos maliciosos a través de los parámetros de entrada, con consecuencias que van desde la extracción de toda la base de datos hasta la ejecución de comandos en el servidor con `xp_cmdshell` en MSSQL.

Implementación técnica en GhostTrack

La clase `SQLInjectionTester` en `modules/sql_injection.py` implementa detección Error-Based SQLi. El método `test_url()` primero obtiene una respuesta base con la URL sin modificar para medir `len(base_response.text)` como referencia. Luego, para cada parámetro GET extraído con `parse_qs(urlparse(url).query)`, itera sobre `SQL_PAYLOADS`: reemplaza el valor del parámetro objetivo en una copia del diccionario, reconstruye la URL con `urlencode(test_params, doseq=True)` y `urlunparse()` preservando todos los componentes originales, y envía la petición.

La respuesta se analiza con `re.search(pattern, response.text, re.IGNORECASE)` contra cada patrón de `ERROR_PATTERNS`. Los patrones cubren: MySQL ('SQL syntax.*MySQL', 'Warning.*mysql_'), PostgreSQL ('PostgreSQL.*ERROR'), MSSQL ('Microsoft SQL', 'OLE DB.*SQL Server'), Oracle ('Oracle error') y SQLite ('SQLite.*Error'). Al encontrar coincidencia se registra tipo 'SQL Injection - Error Based', el parámetro afectado, el payload concreto, la URL completa y severidad HIGH, y se sale del bucle de payloads para ese parámetro con `break`.

`SQL_PAYLOADS` incluye siete payloads: comilla simple (') para error de sintaxis directo, tautología numérica (1' OR '1'='1), tautología con comentario (1' OR '1'='1' --), bypass de login (admin' --), variante con paréntesis (') or '1'='1--), y UNION básica (1' UNION SELECT NULL--). La función `test()` de nivel de módulo construye automáticamente las variantes HTTP y HTTPS del target si no se pasan URLs explícitas.

Interfaz pública

```
test(target: str, urls: List[str] = None) -> Dict
# {'target': '...', 'vulnerabilities': [{'type': 'SQL Injection - Error Based',
#   'parameter': 'id', 'payload': "1' OR '1'='1", 'severity': 'HIGH'}]}
```

7.15 Módulo 15: XSS Scanner — Detección de Cross-Site Scripting

Descripción funcional

El XSS (Cross-Site Scripting) permite inyectar JavaScript en páginas web que se ejecuta en el navegador de las víctimas. A diferencia del SQLi que ataca al servidor, el XSS ataca a los clientes: roba cookies de sesión (`document.cookie`), redirige a sitios maliciosos, realiza keylogging, o presenta formularios de phishing dentro del contexto del sitio legítimo.

Implementación técnica en GhostTrack

La clase XSSScanner en modules/xss_scanner.py implementa detección de Reflected XSS. El método scan_url() usa urlparse() y parse_qs() para extraer los parámetros GET, exactamente igual que SQLInjectionTester. Para cada parámetro, itera sobre XSS_PAYLOADS, construye la URL de prueba y envía la petición. La detección se basa en el operador 'in': if payload in response.text. Si el string del payload aparece literalmente en el HTML sin codificar (sin convertirse en <script> etc.), el servidor no está aplicando HTML encoding y hay XSS reflejado.

Los cinco payloads cubren diferentes vectores de ejecución. <script>alert('XSS')</script> funciona cuando la entrada se refleja en el contenido de la página fuera de atributos. explota el event handler onerror y es efectivo cuando los filtros bloquean el tag script. <svg/onload=alert('XSS')> usa el modelo de ejecución SVG, diferente al HTML. javascript:alert('XSS') funciona cuando la entrada se refleja en el atributo href de un enlace. <iframe src=javascript:alert('XSS')> funciona en contextos que permiten iframes. La severidad se registra como MEDIUM siguiendo los estándares CVSS para XSS reflejado.

Integración con el Web Crawler

En run_full_scan() el XSS Scanner se ejecuta después del Web Crawler. Las URLs con parámetros GET descubiertas por el crawler se pasan directamente como lista de URLs a analizar, maximizando la cobertura sobre la totalidad de la superficie de ataque de la aplicación.

Interfaz pública

```
scan(target: str, urls: List[str] = None) -> Dict
# {'target': '...', 'vulnerabilities': [{'type': 'Reflected XSS',
#   'parameter': 'name', 'payload': "<img src=x onerror=alert('XSS')>",
#   'url': 'http://...?name=...', 'severity': 'MEDIUM'}]}
```

7.16 Módulo 16: ARP Scanner — Descubrimiento de Hosts en Red Local

Descripción funcional

El ARP Scanner descubre todos los hosts activos en la red local leyendo la caché ARP del sistema operativo. A diferencia de los escáneres de puertos TCP que solo detectan hosts con servicios expuestos, el ARP Scanner detecta cualquier dispositivo con actividad de

red reciente, incluyendo equipos con firewall que bloquean todas las conexiones entrantes pero que no pueden ocultar su presencia en la capa de enlace de datos.

Implementación técnica en GhostTrack

La clase ARPScanner en `modules/arp_scanner.py` ejecuta el comando `'arp -n'` en Linux o `'arp -a'` en Windows (detectado con `platform.system()`) mediante `subprocess.check_output(cmd, shell=True).decode()`. La salida se divide en líneas y cada línea se analiza con la regex `r'(\d+\.\d+\.\d+\.\d+)\s+.*?([0-9a-fA-F:]{17})[0-9a-fA-F-]{17})'` que captura la IP (cuatro octetos separados por puntos) y la MAC (17 caracteres con separadores ':' en Linux o '-' en Windows). Por cada match se crea un dict `{ip, mac, vendor}`.

La función `get_vendor()` devuelve 'Unknown' en la versión actual. En una implementación completa realizaría un lookup OUI (Organizationally Unique Identifier) comparando los tres primeros octetos de la MAC contra la base de datos `ieee-oui.txt` de la IEEE para identificar el fabricante del adaptador de red, lo que ayuda a clasificar los dispositivos descubiertos (VMware, Raspberry Pi, Apple, Cisco, etc.). Este es uno de los puntos de mejora documentados para versiones futuras.

Limitación y uso coordinado

El módulo solo muestra hosts con actividad ARP reciente. Para maximizar la cobertura se puede combinar con un barrido de pings previo que fuerce a todos los hosts del segmento a responder. El resultado se almacena en `results['modules']['arp']` y en el reporte HTML se presenta como tabla con columnas IP, MAC y Vendor, permitiendo al auditor identificar rápidamente los sistemas de la red interna.

Interfaz pública

```
scan() -> List[Dict]
# [{ 'ip': '192.168.1.1', 'mac': 'c8:d3:ff:a1:b2:c3', 'vendor': 'Unknown'}, ...]
```

7.17 Módulo 17: Network Sniffer — Captura y Análisis de Tráfico de Red

Descripción funcional

El sniffer captura paquetes en tiempo real de la interfaz de red, permitiendo observar comunicaciones de la red, identificar protocolos que transmiten credenciales en texto claro (HTTP básico, FTP, Telnet, POP3, IMAP sin TLS), descubrir hosts y servicios no visibles desde el exterior, y detectar tráfico LLMNR/NBT-NS susceptible de envenenamiento en fases Red Team.

Implementación técnica en GhostTrack

La clase NetworkSniffer en `modules/network_sniffer.py` importa Scapy de forma dinámica dentro del método `sniff()` mediante `'from scapy.all import sniff as scapy_sniff'`. Esta importación diferida es una decisión de diseño deliberada: el framework arranca correctamente aunque Scapy no esté instalado, y solo muestra el error `'[!] Scapy no está instalado: pip install scapy'` cuando el usuario intenta usar este módulo específico, en lugar de fallar en el arranque global.

La llamada `scapy_sniff(count=count, iface=self.interface)` captura el número de paquetes indicado en la interfaz especificada (None captura en todas las interfaces disponibles). Para cada paquete capturado, se extraen dos campos: `pkt.summary()` que devuelve una cadena descriptiva del paquete (ej: `'Ether / IP / TCP 192.168.1.10:443 > 192.168.1.20:52341 PA / Raw'`) y `pkt.time` que contiene el timestamp Unix como float. Ambos se almacenan como diccionario en `self.packets`.

Requisito de privilegios y solución

Abrir raw sockets en modo promiscuo requiere privilegios elevados. En Linux se puede conceder la capability específica sin ejecutar todo el proceso como root: `sudo setcap cap_net_raw+eip $(which python3)`. Para análisis dirigido, Scapy acepta el parámetro `filter` con sintaxis BPF: `filter='tcp port 80 or tcp port 21'` captura solo tráfico HTTP y FTP, reduciendo el volumen y facilitando la identificación de credenciales en texto claro.

Interfaz pública

```
sniff(interface: str = None, count: int = 100) -> List[Dict]
# [{'summary': 'Ether/IP/TCP 10.0.0.1:80 > 10.0.0.2:43210', 'time': 1712834521.4}, ...]
```

7.18 Módulo 18: Email Harvester — Recolección de Emails OSINT

Descripción funcional

La recolección de emails construye listas de contactos de la organización para phishing selectivo, credential stuffing, o para comprender la estructura interna y el formato de los correos corporativos (`nombre.apellido@empresa.com`), información valiosa para generar señuelos convincentes en la fase de Social Engineering.

Implementación técnica en GhostTrack

La clase EmailHarvester en `modules/email_harvester.py` almacena los emails en `self.emails` (tipo `set`) que elimina duplicados automáticamente. El método `harvest_from_website()` construye la URL base añadiendo 'http://' si no tiene protocolo, hace `requests.get(url, timeout=10)`, y aplica la regex `r'[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}'` sobre `response.text`. Todos los resultados se convierten a minúsculas con `email.lower()` antes de añadirse al set, evitando duplicados por diferencias de capitalización (`Admin@empresa.com` y `admin@empresa.com` son el mismo email).

El método `harvest_from_search()` construye la query de Google Dorking pero no la ejecuta automáticamente, imprimiendo en su lugar '[*] Búsqueda en Google (limitada sin API)'. Esta decisión de diseño respeta los términos de servicio de Google que prohíben el scraping automatizado, mientras que proporciona al auditor la query lista para ejecutar manualmente en el navegador.

Coordinación con módulo 40 (SocialEngineering)

Los emails recolectados se almacenan en `results['modules']['emails']` y están disponibles para el módulo `SocialEngineeringToolkit` del bloque Red Team, que los usa como lista de destinatarios de campañas de phishing. Esta integración entre módulos de reconocimiento y módulos Red Team es un ejemplo del diseño de pipeline de GhostTrack.

Interfaz pública

```
harvest(target: str) -> List[str]
# ['admin@empresa.com', 'info@empresa.com', 'j.garcia@empresa.com']
```

7.19 Módulo 19: Metadata Extractor — Extracción de Metadatos de Ficheros

Descripción funcional

Los ficheros publicados en el sitio web del objetivo (PDFs, imágenes) pueden contener metadatos con información sensible: nombres de usuario del SO, rutas internas, software utilizado, y en fotografías de smartphones, coordenadas GPS de la ubicación exacta donde se tomó la foto.

Implementación técnica en GhostTrack

La clase `MetadataExtractor` en `modules/metadata_extractor.py` determina el tipo de fichero mediante `os.path.splitext(self.file_path)[1].lower()` y actúa como dispatcher: para

extensiones .jpg, .jpeg, .png y .gif llama a `extract_image_metadata()`; para .pdf llama a `extract_pdf_metadata()`; cualquier otra extensión devuelve `{'error': 'Tipo de archivo no soportado'}`.

`extract_image_metadata()` abre la imagen con `Image.open()` de Pillow y llama a `image._getexif()` que devuelve un diccionario con claves numéricas (identificadores de tags EXIF). Para cada par, usa `TAGS.get(tag_id, tag_id)` del módulo `PIL.ExifTags` para traducir el número al nombre legible: el tag 271 se convierte a 'Make', 272 a 'Model', 306 a 'DateTime', 34853 a 'GPSInfo'. El valor se convierte a `str()` para garantizar la serializabilidad JSON. Si `_getexif()` retorna `None`, el método devuelve diccionario vacío sin error.

`extract_pdf_metadata()` abre el fichero en modo binario y crea `PyPDF2.PdfReader(f)`. El atributo `pdf.metadata` contiene los campos del diccionario de información PDF: `'/Author'`, `'/Creator'`, `'/Producer'`, `'/Subject'`, `'/Title'`, `'/CreationDate'`, `'/ModDate'`. Itera sobre estos campos y los almacena en `self.metadata`. El formato de fecha PDF (`'/D:YYYYMMDDHHmmSSOHH'mm'`) se almacena tal cual para mantener la fidelidad al documento original.

Función de nivel de módulo

La función `extract(file_paths: List[str])` instancia un `MetadataExtractor` por cada fichero y consolida todos los resultados en un diccionario indexado por ruta del fichero. Esto permite analizar múltiples ficheros en una sola llamada desde `--files` en la CLI.

Interfaz pública

```
extract(file_paths: List[str]) -> Dict
# {'foto.jpg': {'Make': 'Apple', 'Model': 'iPhone 13',
#   'DateTime': '2025:03:15 14:23:11',
#   'GPSInfo': {1: 'N', 2: ((40, 1), (25, 1), (3456, 100)), 3: 'W', 4: ((3, 1), (41, 1), ...))}}
```

7.20 Módulo 20: Custom Scripts — Sistema de Extensión por Inyección

Descripción funcional

Custom Scripts es el mecanismo de extensibilidad del framework: permite ejecutar cualquier script Python externo dentro del ecosistema de GhostTrack sin modificar ningún fichero del framework, implementando directamente el principio Abierto/Cerrado.

Implementación técnica en GhostTrack

La clase `ScriptInjector` en `modules/custom_scripts.py` implementa la carga dinámica con `importlib.util`. El método `load_script(script_path)` verifica la existencia con `os.path.exists()` y lanza `FileNotFoundError` si el fichero no existe. Si existe, ejecuta la secuencia: `spec = importlib.util.spec_from_file_location('custom_module', script_path)` crea el `ModuleSpec`; `module = importlib.util.module_from_spec(spec)` instancia el módulo vacío; `spec.loader.exec_module(module)` ejecuta el código del fichero en el espacio de nombres del módulo, definiendo todas sus funciones. Finalmente `sys.modules['custom_module'] = module` lo registra globalmente.

El método `execute()` verifica con `hasattr(module, 'run')` que el módulo cargado implementa el contrato de interfaz obligatorio. Si no existe la función `run`, lanza `AttributeError` con mensaje descriptivo. Si existe, llama `module.run(self.target, *args, **kwargs)` y almacena el resultado en `self.results[script_path]`. La función `create_template()` genera `scripts/template.py` con `os.makedirs(exist_ok=True)` y escribe la plantilla con la firma correcta, docstring completo, estructura del diccionario de resultados y ejemplos comentados de operaciones comunes.

Flujo completo con `--inject` en la CLI

Con el argumento `--inject scripts/mi_modulo.py`, `main()` entra en el bloque `if args.inject:` y llama `custom_scripts.inject(args.target, args.inject)`. La función `inject()` instancia `ScriptInjector` y llama `execute()`. El resultado se almacena en el log y `Reporter` genera los tres formatos de reporte con el resultado del script personalizado, exactamente igual que con cualquier módulo nativo del framework.

Interfaz pública

```
inject(target: str, script_path: str, *args, **kwargs) -> Dict
create_template(output_path: str = 'scripts/template.py')
# Contrato mínimo del script externo:
# def run(target: str, *args, **kwargs) -> dict: ...
```

8. Módulos Red Team — Descripción Técnica Detallada

Los veinte módulos Red Team de GhostTrack constituyen el bloque técnicamente más avanzado y de mayor profundidad del framework. Están agrupados en cinco categorías funcionales que siguen la progresión natural de un ejercicio avanzado de evaluación ofensiva: enumeración de Active Directory, ataques a credenciales, post-explotación, persistencia y evasión, y técnicas avanzadas de exfiltración y comunicación encubierta.

Todos estos módulos están implementados en el fichero `modules/redteam/complete_redteam.py`, que contiene las veinte clases principales exportadas mediante el fichero `__init__.py` del paquete. La función `run_full_redteam_assessment()` orquesta la ejecución secuencial de todos los módulos y consolida los resultados en el mismo formato que el bloque básico, garantizando la compatibilidad con el sistema de reportes del framework.

Desde el punto de vista del marco MITRE ATT&CK, los módulos Red Team cubren once de las catorce tácticas de la matriz Enterprise: Reconnaissance, Initial Access, Execution, Persistence, Privilege Escalation, Defense Evasion, Credential Access, Discovery, Lateral Movement, Command and Control y Exfiltration. Esta cobertura casi completa de la Kill Chain de un adversario avanzado es lo que eleva a GhostTrack al nivel de un framework de evaluación ofensiva completo, más allá de la funcionalidad básica de un escáner de vulnerabilidades convencional.

Es imprescindible reiterar que el uso de estos módulos está restringido a entornos de laboratorio controlados y a ejercicios con autorización expresa y por escrito del propietario de los sistemas objetivo. El uso no autorizado de estas técnicas constituye un delito tipificado en el Código Penal español y en normativa internacional equivalente.

8.1 Categoría: Enumeración y Reconocimiento de Active Directory

La enumeración de Active Directory es el punto de partida de cualquier ejercicio Red Team en entornos corporativos. Antes de intentar comprometer un dominio, el equipo ofensivo necesita entender su estructura: qué cuentas existen, cómo están organizadas, qué privilegios tienen, qué servicios están registrados y cuáles son los puntos débiles de configuración. Los tres módulos de esta categoría construyen una imagen completa del entorno Active Directory que alimenta todas las fases posteriores del ejercicio.

8.1.1 SMBEnumerator — Módulo 21

El protocolo SMB (Server Message Block) es el protocolo de compartición de recursos de Microsoft, definido en la especificación abierta [MS-SMB2] y sus predecesoras. Su historia es inseparable de la historia de la seguridad ofensiva en Windows: las vulnerabilidades en SMB han protagonizado los ataques más devastadores de la última década, desde EternalBlue (que permitió la propagación de WannaCry en 2017 infectando más de 200.000 sistemas en 150 países) hasta múltiples vulnerabilidades de divulgación de información y escalada de privilegios que afectan a versiones anteriores del protocolo.

SMB opera sobre el puerto TCP 445 en implementaciones modernas y sobre NetBIOS (puertos 137-139) en implementaciones antiguas. La versión SMBv1, ampliamente conocida por sus problemas de seguridad, fue desactivada por defecto en Windows Server 2016 y Windows 10 Fall Creators Update, aunque muchas organizaciones todavía tienen sistemas con SMBv1 habilitado por razones de compatibilidad con aplicaciones legacy. La clase SMBEnumerator detecta la versión de SMB soportada como primera operación, ya que esto determina el conjunto de técnicas de enumeración aplicables.

La enumeración de shares (recursos compartidos) es el primer objetivo del módulo. Un share SMB es un directorio del sistema de ficheros que se hace accesible a través de la red. Windows crea automáticamente varios shares administrativos que son especialmente interesantes para el atacante: C\$ es el share administrativo de la unidad C:, ADMIN\$ apunta al directorio de Windows (%SystemRoot%), e IPC\$ es el share de comunicación entre procesos (Inter-Process Communication) que permite consultar información del sistema mediante llamadas RPC. Estos shares solo son accesibles para administradores, pero su existencia y accesibilidad pueden probarse sin privilegios de administrador, lo que proporciona información valiosa sobre el nivel de hardening del sistema.

La enumeración mediante null session es una técnica que aprovecha una configuración insegura presente en Windows Server 2003 y anteriores, y ocasionalmente en versiones más modernas con políticas de seguridad relajadas. Una null session es una conexión anónima al share IPC\$ (sin credenciales) que, en sistemas vulnerables, permite consultar la base de datos de cuentas del sistema (SAM) a través de las interfaces RPC NetrUserEnum, NetrGroupEnum y NetrServerGetInfo. Aunque esta configuración está deshabilitada por defecto en sistemas modernos, su presencia sigue siendo un hallazgo de severidad HIGH en auditorías porque indica que las políticas de hardening no han sido aplicadas correctamente.

La enumeración de políticas de contraseñas mediante SMB permite conocer los requisitos de complejidad, la longitud mínima, el historial y el tiempo de bloqueo de cuentas configurados en el dominio. Esta información es crítica para planificar ataques de fuerza bruta: si el tiempo de bloqueo es de 30 minutos tras 5 intentos fallidos, el atacante debe limitar la velocidad de sus intentos de autenticación para no bloquear cuentas y generar alertas.

El módulo también implementa la detección de SMB Signing (firma de mensajes SMB). La firma SMB es el mecanismo de seguridad que previene los ataques de relay NTLM: cuando está habilitada de forma obligatoria, los mensajes SMB deben ir firmados digitalmente, lo que impide que un atacante en posición de man-in-the-middle retransmita un handshake NTLM capturado. La clase SMBEnumerator detecta si la firma está configurada como 'required', 'enabled' (pero no obligatoria) o 'disabled' en cada servidor analizado, siendo los dos últimos estados condiciones necesarias para los ataques de relay descritos en el módulo NTLMRelayAttacker.

```
# Ejemplo de uso del SMBEnumerator
from modules.redteam.complete_redteam import SMBEnumerator

enumerator = SMBEnumerator('192.168.10.10')
results = enumerator.enumerate()
# results contiene: shares, null_session, smb_version,
#                  signing_status, password_policy
```

8.1.2 LDAPEnumerator — Módulo 22

LDAP (Lightweight Directory Access Protocol, RFC 4511) es el protocolo estándar para acceder y modificar servicios de directorio. En el contexto de Active Directory, LDAP es la interfaz principal mediante la que los clientes consultan la base de datos del directorio: cada usuario, equipo, grupo, impresora, política y objeto del dominio es un objeto LDAP con atributos accesibles mediante consultas de búsqueda.

La estructura de un directorio Active Directory sigue una jerarquía de contenedores denominada Distinguished Name (DN). El objeto raíz del directorio se denomina Domain Naming Context o Naming Context (NC) y tiene un DN de la forma DC=corp,DC=local para un dominio corp.local. Dentro de esta raíz, los objetos se organizan en Unidades Organizativas (OUs) que pueden estar anidadas a cualquier profundidad. Cada objeto tiene un Distinguished Name único que describe su posición en la jerarquía, como CN=John Smith,OU=IT,OU=Users,DC=corp,DC=local.

La clase LDAPEnumerator realiza una serie de consultas LDAP estructuradas que extraen las categorías de información más valiosas para el reconocimiento de Active Directory. La consulta de usuarios activos utiliza el filtro LDAP '(&(objectClass=user)(!(userAccountControl:1.2.840.113556.1.4.803:=2)))' para encontrar todos los usuarios cuya cuenta está habilitada. El atributo userAccountControl es una bandera de bits que codifica el estado de la cuenta: el bit 0x2 indica cuenta deshabilitada, el bit 0x10 indica cuenta sin expiración de contraseña, y el bit 0x400000 indica que la preautenticación Kerberos está deshabilitada, siendo este último el indicador de cuentas susceptibles de AS-REP Roasting.

El atributo servicePrincipalName (SPN) es de especial interés para la fase de ataque. Un SPN es un identificador único de una instancia de servicio que se usa en Kerberos para asociar el servicio con la cuenta de dominio que lo ejecuta. Cuando una cuenta de usuario (en lugar de una cuenta de equipo o gMSA) tiene SPNs registrados, esa cuenta es susceptible de Kerberoasting, ya que cualquier usuario del dominio puede solicitar un ticket de servicio cifrado con el hash de su contraseña. La enumeración de SPNs es por tanto un paso previo indispensable antes de ejecutar el KerberosAttacker.

La enumeración de grupos y sus miembros permite identificar los grupos de seguridad de alto privilegio del dominio y qué usuarios pertenecen a ellos. Los grupos más relevantes para el equipo Red Team son Domain Admins (control total del dominio), Enterprise Admins (control total del forest), Schema Admins (puede modificar el esquema AD), Account Operators (puede crear y modificar cuentas), Backup Operators (puede leer cualquier fichero del sistema), y el grupo local Administrators de los servidores clave. La pertenencia a estos grupos define las rutas de escalada de privilegios más directas.

La enumeración de equipos unau dominio mediante el filtro '(objectClass=computer)' revela todos los sistemas que forman parte del directorio, incluyendo el sistema operativo instalado (atributo operatingSystem), la versión del mismo (operatingSystemVersion) y la fecha del último inicio de sesión (lastLogon). Esta información permite identificar sistemas con sistemas operativos obsoletos que podrían ser susceptibles de exploits conocidos (como EternalBlue en Windows Server 2008 sin parchear).

La enumeración de GPOs (Group Policy Objects) permite conocer las políticas de seguridad que se aplican en el dominio. Las GPOs controlan cientos de parámetros de seguridad: políticas de contraseñas, configuración de Windows Defender, restricciones de ejecución de scripts, configuración de auditoría, habilitación o deshabilitación de servicios, y muchos otros. Conocer las GPOs del dominio permite al equipo Red Team anticipar qué controles de seguridad están activos y adaptar sus técnicas de evasión.

La enumeración de relaciones de confianza (trusts) entre dominios es especialmente importante en organizaciones con múltiples dominios o forests. Una relación de confianza bidireccional entre dos dominios implica que los usuarios de ambos dominios pueden autenticarse en cualquiera de ellos, lo que significa que comprometer un dominio con políticas de seguridad más laxas puede proporcionar acceso al dominio principal. La clase `LDAPEnumerator` detecta todos los trusts configurados y su tipo (transitivo, intransitivo, de forest, externo).

```
# Atributos LDAP de alto valor para el equipo Red Team
HIGH_VALUE_ATTRIBUTES = [
    'userAccountControl', # Estado de la cuenta, Kerberos preauth
    'servicePrincipalName', # SPNs para Kerberoasting
    'memberOf', # Grupos a los que pertenece
    'pwdLastSet', # Antigüedad de la contraseña
    'lastLogon', # Última autenticación
    'badPwdCount', # Intentos de autenticación fallidos
    'adminCount', # Objeto protegido por AdminSDHolder
    'msDS-AllowedToActOnBehalfOfOtherIdentity', # RBCD
]
```

8.1.3 *BloodHoundCollector* — Módulo 25

BloodHound (Rohan et al., 2016) es una de las herramientas más revolucionarias que han aparecido en el campo de la evaluación ofensiva de Active Directory en la última década. Su premisa fundamental es que la seguridad de un dominio AD no puede evaluarse analizando objetos individuales en aislamiento; solo puede entenderse correctamente modelando todas las relaciones entre objetos como un grafo dirigido y buscando rutas de ataque en ese grafo.

El grafo de *BloodHound* representa objetos (usuarios, grupos, equipos, GPOs, OUs, dominios) como nodos y relaciones (`MemberOf`, `AdminTo`, `HasSession`, `CanRDP`, `GenericAll`, `GenericWrite`, `WriteOwner`, `WriteDACL`, etc.) como aristas con etiquetas. Una vez construido el grafo, el algoritmo de búsqueda de caminos más cortos de Dijkstra (implementado sobre la base de datos de grafos Neo4j) puede encontrar en milisegundos la ruta de menor número de pasos entre cualquier cuenta comprometida y el grupo Domain Admins, aunque esa ruta involucre docenas de saltos intermedios que ningún analista humano detectaría revisando los objetos individualmente.

La clase `BloodHoundCollector` de *GhostTrack* replica la funcionalidad de *SharpHound* (el colector oficial en C#) en Python puro. El proceso de recolección opera en varias fases. La fase de enumeración de objetos consulta LDAP para obtener todos los usuarios, grupos,

equipos, GPOs y OUs del dominio con sus atributos relevantes. La fase de enumeración de sesiones realiza consultas NetSessionEnum a todos los equipos del dominio para determinar qué usuarios tienen sesiones activas en cada momento; esta información es especialmente valiosa porque una sesión de un Domain Admin en un servidor al que el atacante tiene acceso permite robar el token del DA y comprometer el dominio.

La fase de enumeración de derechos de administrador local realiza consultas NetLocalGroupGetMembers a cada equipo del dominio para determinar qué usuarios y grupos tienen privilegios de administrador local. En entornos corporativos, es habitual encontrar que grupos de seguridad tienen derechos de administrador local en conjuntos de servidores, y que esos grupos tienen a su vez miembros que pertenecen a otros grupos en cadena. BloodHound modela estas relaciones transitivas y puede encontrar que un usuario aparentemente sin privilegios tiene una cadena de relaciones que le da administrador local en un servidor donde un Domain Admin tiene sesión activa.

La fase de enumeración de ACLs del directorio es la más costosa computacionalmente pero también la más reveladora. Las ACLs (Access Control Lists) de los objetos LDAP controlan qué usuarios y grupos tienen qué permisos sobre cada objeto. Los permisos de mayor interés para el equipo Red Team son GenericAll (control total sobre el objeto, incluyendo la capacidad de cambiar su contraseña), GenericWrite (puede modificar cualquier atributo del objeto), WriteOwner (puede cambiar el propietario del objeto, lo que otorga control total), WriteDACL (puede modificar la ACL del objeto, lo que permite concederse a uno mismo cualquier permiso) y AllExtendedRights (todos los derechos extendidos, incluyendo cambiar la contraseña). La presencia de cualquiera de estos permisos sobre una cuenta de alto privilegio para un usuario de bajo privilegio constituye una ruta de escalada de privilegios.

Los datos recolectados se exportan en formato JSON compatible con BloodHound: cuatro ficheros JSON (users.json, groups.json, computers.json, domains.json) que BloodHound importa mediante su función de ingesta de datos. Una vez importados, el analista puede usar las consultas preconstruidas de BloodHound ('Find all Domain Admins', 'Shortest path to Domain Admins', 'Find computers where Domain Admins are logged in') o construir consultas personalizadas en el lenguaje de consulta de Neo4j (Cypher) para explorar el grafo de ataque.

```
# Consulta Cypher para encontrar la ruta más corta a DA
MATCH p=shortestPath((u:User {enabled:true})-[*1..]->(g:Group {name:'DOMAIN
ADMINS@CORP.LOCAL'})))
WHERE NOT u.name STARTS WITH 'KRBTGT'
RETURN p
```

8.2 Categoría: Ataques a Credenciales

Los ataques a credenciales son el núcleo de la mayoría de los ejercicios Red Team exitosos. Active Directory es fundamentalmente un sistema de autenticación, y comprometer las credenciales de una cuenta de alto privilegio es con frecuencia el camino más directo al control total del dominio. Los tres módulos de esta categoría cubren los vectores de ataque a credenciales más efectivos y más documentados en la práctica real.

8.2.1 KerberosAttacker — Módulo 23

Kerberos (RFC 4120) es el protocolo de autenticación estándar de Active Directory desde Windows 2000. Su diseño se basa en el concepto de 'tickets' cifrados que permiten a un usuario demostrar su identidad a un servicio sin transmitir su contraseña por la red. El protocolo involucra tres partes: el cliente (usuario que quiere autenticarse), el KDC (Key Distribution Center, que corre en el controlador de dominio) y el servicio al que el usuario quiere acceder.

El flujo de autenticación Kerberos consta de seis mensajes numerados (AS-REQ, AS-REP, TGS-REQ, TGS-REP, AP-REQ, AP-REP). En la fase de pre-autenticación (AS-REQ/AS-REP), el cliente demuestra que conoce su contraseña cifrando un timestamp con la clave derivada de ella y enviándolo al KDC. El KDC verifica el timestamp y emite un Ticket Granting Ticket (TGT) cifrado con la clave del servicio krbtgt (la cuenta especial del KDC). En la fase de obtención de ticket de servicio (TGS-REQ/TGS-REP), el cliente presenta su TGT al KDC para solicitar un Service Ticket (TGS) para un servicio específico, identificado por su SPN. El KDC emite el TGS cifrado con el hash de contraseña de la cuenta que ejecuta el servicio. Finalmente, el cliente presenta el TGS al servicio para autenticarse.

Kerberoasting (Medin, 2014) explota la fase TGS del protocolo. El ataque aprovecha que cualquier usuario autenticado en el dominio puede solicitar un TGS para cualquier servicio con un SPN registrado, sin ninguna verificación adicional. El TGS devuelto por el KDC está cifrado con el hash NTLM de la contraseña de la cuenta de servicio. El atacante solicita TGS para todas las cuentas con SPNs y exporta los tickets en formato compatible con herramientas de cracking offline como Hashcat o John the Ripper. Si la contraseña de la cuenta de servicio es débil (lo que es habitual en cuentas de servicio de aplicaciones legacy) o si está en un diccionario de contraseñas comunes, el atacante puede recuperarla en minutos u horas.

La efectividad práctica de Kerberoasting es muy elevada. En evaluaciones reales, es frecuente encontrar entre el 5% y el 15% de las organizaciones auditadas con cuentas de servicio susceptibles de Kerberoasting y con contraseñas débiles que son crackeadas en menos de 24 horas. La razón es que muchas cuentas de servicio son creadas por administradores de aplicaciones que no son especialistas en seguridad, se configuran con contraseñas simples para facilitar la instalación, y nunca se cambian porque el proceso de cambio requeriría reconfigurar la aplicación.

AS-REP Roasting (Harmj0y, 2016) ataca una configuración diferente: cuentas de usuario con la preautenticación Kerberos deshabilitada. Normalmente, el mensaje AS-REQ incluye un timestamp cifrado que el KDC verifica para prevenir ataques de repetición. Cuando la preautenticación está deshabilitada (atributo 'Do not require Kerberos preauthentication' en las propiedades de la cuenta en AD Users and Computers), el KDC emite el AS-REP sin verificar que el solicitante conoce la contraseña. La parte del AS-REP cifrada con el hash de contraseña del usuario puede ser capturada y atacada offline de la misma forma que en Kerberoasting.

La diferencia clave entre Kerberoasting y AS-REP Roasting es que AS-REP Roasting puede realizarse sin credenciales: el atacante no necesita estar autenticado en el dominio para solicitar el AS-REP de cuentas con preautenticación deshabilitada. Esto lo convierte en un vector de ataque de initial access en escenarios donde el atacante solo tiene acceso de red al puerto 88/TCP del controlador de dominio pero no dispone aún de ninguna credencial del dominio.

La clase KerberosAttacker también implementa Pass-the-Ticket (PTT), que permite reutilizar tickets Kerberos robados de la memoria de un sistema comprometido para autenticarse en otros servicios del dominio bajo la identidad del usuario víctima, sin necesidad de conocer la contraseña en texto claro. Los tickets se exportan en formato CCACHE compatible con la suite Impacket y con Mimikatz para su posterior uso.

```
# Flujo de Kerberoasting en KerberosAttacker
1. LDAPEnumerator.get_spn_accounts() → lista cuentas con SPN
2. KerberosAttacker.request_tgs(account, spn) → solicitar ticket
3. KerberosAttacker.export_ticket(ticket) → exportar para cracking
4. [offline] hashcat -m 13100 tickets.txt rockyou.txt
   # $krb5tgs$23$*svc_sql/SQLSERVER.corp.local*$... → 'Password1!'
```

8.2.2 NTLMRelayAttacker — Módulo 24

NTLM (NT LAN Manager) es el protocolo de autenticación challenge-response de Windows que se usa cuando Kerberos no está disponible o no es aplicable: cuando la autenticación se hace por dirección IP en lugar de por nombre (Kerberos requiere nombres de host), cuando el sistema no está unido a un dominio, o cuando el servicio de destino no soporta Kerberos. A pesar de ser un protocolo más antiguo y menos seguro que Kerberos, NTLM sigue siendo ampliamente usado en entornos corporativos reales y está presente en prácticamente todas las redes Windows.

El handshake NTLM consta de tres mensajes: NEGOTIATE (el cliente anuncia sus capacidades), CHALLENGE (el servidor envía un challenge de 8 bytes), y AUTHENTICATE (el cliente envía la respuesta al challenge calculada con su hash NTLM). La vulnerabilidad fundamental de NTLM es que el challenge/response puede ser retransmitido: si un atacante puede interceptar estos tres mensajes, puede retransmitirlos a otro servicio en tiempo real, autenticándose como el usuario víctima sin necesidad de conocer su contraseña.

El escenario de ataque más común para el relay NTLM en entornos corporativos aprovecha los protocolos de resolución de nombres de Windows. Cuando un sistema Windows intenta resolver un nombre de host que no existe en DNS, cae back a protocolos alternativos: LLMNR (Link-Local Multicast Name Resolution, puerto UDP 5355) y NBT-NS (NetBIOS Name Service, puerto UDP 137). Estos protocolos son vulnerables a envenenamiento (poisoning): el atacante responde a las peticiones de resolución haciéndose pasar por el nombre buscado, lo que provoca que el sistema víctima intente autenticarse contra el sistema del atacante con sus credenciales NTLM.

La herramienta Responder (que complementa a la clase NTLMRelayAttacker de GhostTrack) automatiza el envenenamiento de LLMNR/NBT-NS. Una vez que el sistema víctima intenta autenticarse contra el sistema del atacante, NTLMRelayAttacker retransmite las credenciales capturadas contra el objetivo real: otro servidor SMB, un servidor LDAP o HTTP que acepte autenticación NTLM. Si el servidor de destino no requiere firma SMB (SMB signing no obligatorio), el relay tiene éxito y el atacante obtiene acceso autenticado al servidor de destino bajo la identidad del usuario víctima.

Las contramedidas principales contra los ataques de relay NTLM son: habilitar SMB Signing obligatorio en todos los servidores (que previene el relay a SMB), habilitar LDAP Signing y LDAP Channel Binding (que previene el relay a LDAP), deshabilitar LLMNR y NBT-NS mediante directiva de grupo (que elimina el vector de envenenamiento), y habilitar EPA (Enhanced Protection for Authentication) en los servicios web que aceptan NTLM. La

clase NTLMRelayAttacker detecta la ausencia de estas contramedidas y reporta cada una como un hallazgo de seguridad independiente con su severidad correspondiente.

Una variante especialmente impactante del relay NTLM es el relay a LDAP para crear cuentas de computer accounts maliciosas. Por defecto, cualquier usuario de dominio puede crear hasta 10 computer accounts en Active Directory (atributo ms-DS-MachineAccountQuota = 10). Si el atacante consigue que una cuenta de alto privilegio (como un Domain Admin) autentique contra su servidor mediante relay y retransmite las credenciales al LDAP del DC, puede crear una computer account controlada por el atacante. Esta computer account puede ser usada a continuación para técnicas de escalada adicionales como Resource-Based Constrained Delegation (RBCD).

```
# Escenario completo de NTLM Relay a LDAP + RBCD
1. Responder envenena LLMNR → captura hash NTLM del DA
2. NTLMRelayAttacker.relay_to_ldap(da_hash, dc_ldap)
   → crea computer account ATTACKER$ con contraseña conocida
3. NTLMRelayAttacker.set_rbcd(target_server, 'ATTACKER$')
   → configura RBCD: ATTACKER$ puede impersonar cualquier usuario
   → en target_server
4. KerberosAttacker.s4u_impersonate('Administrator', target_server)
   → obtiene TGS como Administrator en target_server
5. Resultado: acceso total a target_server como Administrator
```

8.2.3 CredentialDumper — Módulo 28

La extracción de credenciales almacenadas en un sistema comprometido es uno de los objetivos más importantes de la fase de post-explotación. Las credenciales extraídas permiten expandir el acceso lateral (usar las mismas credenciales en otros sistemas de la red), escalar privilegios verticalmente (usar las credenciales de una cuenta con más privilegios) y establecer persistencia (crear cuentas backdoor con contraseñas conocidas).

El vector más crítico de extracción de credenciales en Windows es el dumping de LSASS (Local Security Authority Subsystem Service). LSASS es el proceso responsable de la autenticación en Windows: cuando un usuario inicia sesión, LSASS autentica las credenciales, crea el token de acceso y mantiene en memoria los materiales de credencial necesarios para las autenticaciones Kerberos y NTLM posteriores. En sistemas Windows XP y Server 2003, y en sistemas más modernos con el proveedor WDigest habilitado, LSASS almacena las contraseñas en texto claro en memoria. En sistemas modernos con WDigest deshabilitado (comportamiento por defecto desde Windows 8.1 y KB2871997), LSASS almacena hashes NTLM, tickets Kerberos y hashes MSCacheV2.

La extracción del proceso LSASS se puede realizar mediante varias técnicas. La técnica más conocida usa la API `MiniDumpWriteDump()` para crear un volcado de memoria del proceso `lsass.exe`, que luego se transfiere al sistema del atacante y se analiza offline con `Mimikatz`. Una técnica más sigilosa usa el proveedor SSP (Security Support Provider) personalizado que inyecta una DLL en LSASS para extraer las credenciales sin crear un fichero de volcado. La clase `CredentialDumper` implementa ambas técnicas y detecta automáticamente cuál es aplicable según la versión de Windows y las defensas presentes.

La extracción de la SAM (Security Account Manager) recupera los hashes NTLM de todas las cuentas locales del sistema. La SAM está almacenada en el fichero `%SystemRoot%\system32\config\SAM`, protegido por el sistema operativo que impide su lectura mientras el sistema está en ejecución. Para leerla, es necesario usar técnicas de copia de volumen en sombra (Volume Shadow Copy) o acceder al registro mediante la API `reg save` para exportar las claves `HKLM\SAM` y `HKLM\SYSTEM` (esta última contiene la clave `SYSKEY` necesaria para descifrar los hashes). La clase `CredentialDumper` automáticamente usa el método de VSS cuando está disponible o el método de `reg save` como alternativa.

La extracción de credenciales WiFi aprovecha que Windows almacena los perfiles de red WiFi en `%ProgramData%\Microsoft\Wlansvc\Profiles\Interfaces\` en formato XML, con las contraseñas cifradas mediante la API `DPAPI` (Data Protection API). `DPAPI` cifra los datos con una clave derivada de las credenciales del usuario actual, lo que significa que el proceso que llama a `CryptUnprotectData()` para descifrar los datos debe estar ejecutándose en el contexto del usuario propietario del perfil. La clase `CredentialDumper` ejecuta la descifrado en el contexto del usuario del sistema comprometido y extrae las contraseñas WiFi en texto claro.

La extracción de credenciales de navegadores web es uno de los vectores más productivos en ejercicios Red Team porque los usuarios guardan cientos de contraseñas en sus navegadores: accesos a aplicaciones corporativas, portales de proveedores, servicios cloud, VPNs web y muchos otros recursos. Chrome y la mayoría de navegadores basados en Chromium almacenan las credenciales en una base de datos SQLite en el perfil del usuario, cifradas con `DPAPI`. Firefox usa su propio sistema de cifrado basado en NSS (Network Security Services) con el fichero `key4.db` como almacén de claves. La clase `CredentialDumper` extrae y descifra las credenciales de Chrome, Firefox, Edge y Brave.

```
# Vectores de extracción implementados en CredentialDumper
vectores = {
    'lsass_minidump': 'MiniDumpWriteDump() → volcado offline',
    'lsass_ssp':      'SSP injection → extracción en memoria',
    'sam_vss':        'Volume Shadow Copy → hashes locales',
    'sam_registry':   'reg save HKLM\SAM → hashes locales',
```

```
'wifi_dpapi':      'CryptUnprotectData() → passwords WiFi',  
'chrome_sqlite':  'Login Data DB + DPAPI → web passwords',  
'firefox_nss':    'key4.db + NSS → web passwords',  
'credential_mgr': 'Windows Credential Manager → passwords',  
}
```

8.3 Categoría: Post-Explotación

La fase de post-explotación comienza cuando el equipo Red Team ha conseguido acceso a un sistema de la red objetivo. En este momento, el objetivo cambia: ya no se trata de ganar acceso, sino de maximizarlo. Los módulos de post-explotación de GhostTrack implementan las técnicas necesarias para escalar privilegios, moverse lateralmente por la red y mantener el acceso a largo plazo.

8.3.1 *PrivilegeEscalationChecker* — Módulo 26

La escalada de privilegios es el proceso mediante el que un atacante que ha obtenido acceso con privilegios limitados (por ejemplo, como usuario estándar sin derechos de administrador) consigue obtener privilegios elevados (administrador local, SYSTEM en Windows, root en Linux). Es una de las fases más críticas de cualquier ejercicio Red Team porque determina qué puede hacer el equipo a continuación.

En Windows, la escalada de privilegios puede conseguirse mediante múltiples vectores que la clase *PrivilegeEscalationChecker* comprueba de forma sistemática. El vector Unquoted Service Path (ruta de servicio sin entrecomillar) es uno de los más clásicos y todavía prevalentes: cuando un servicio de Windows tiene configurada la ruta de su ejecutable con espacios pero sin comillas (por ejemplo, C:\Program Files\My App\service.exe en lugar de "C:\Program Files\My App\service.exe"), Windows lo interpreta de forma ambigua al iniciarlo. Probará a ejecutar primero C:\Program.exe, luego C:\Program Files\My.exe, y finalmente la ruta completa correcta. Si el atacante puede crear un ejecutable en alguna de estas rutas intermedias (lo que requiere permisos de escritura en el directorio correspondiente), ese ejecutable se ejecutará con los privilegios del servicio (típicamente SYSTEM) la próxima vez que el servicio arranque.

La comprobación de permisos de directorios del PATH del sistema identifica directorios en la variable PATH que son escribibles por usuarios no privilegiados. Si el atacante puede colocar un ejecutable con el nombre de una herramienta del sistema (como net.exe, ipconfig.exe o cualquier otro comando que sea invocado regularmente por tareas automatizadas o por usuarios con privilegios), ese ejecutable se ejecutará en lugar del original la próxima vez que se invoque el nombre del comando.

La comprobación de tareas programadas identifica tareas configuradas para ejecutarse en el contexto de una cuenta privilegiada (SYSTEM, administrador) cuyos ejecutables o scripts están en ubicaciones con permisos de escritura para usuarios no privilegiados. Si el atacante puede reemplazar o modificar el script o ejecutable de una tarea programada, obtendrá ejecución de código con los privilegios de la cuenta bajo la que corre la tarea.

La enumeración de tokens de acceso privilegiados (SeImpersonatePrivilege, SeAssignPrimaryTokenPrivilege, SeBackupPrivilege, SeDebugPrivilege, SeTcbPrivilege) es especialmente relevante en contextos de servicios web comprometidos. Los procesos de IIS, SQL Server, y muchos otros servicios de Windows tienen por defecto el privilegio SeImpersonatePrivilege, que permite impersonar a cualquier usuario que se conecte al servicio. La técnica Potato (JuicyPotato, SweetPotato, PrintSpoofer) explota este privilegio para escalar a SYSTEM mediante una cadena de llamadas COM/RPC que fuerza al servicio SYSTEM a autenticar contra un servidor controlado por el atacante, capturando el token y usándolo para ejecutar código como SYSTEM.

En entornos Linux, la clase PrivilegeEscalationChecker comprueba un conjunto diferente de vectores. Los binarios SUID (Set User ID) son ejecutables que se ejecutan con los privilegios del propietario del fichero en lugar de los del usuario que los ejecuta. Si un binario SUID es propiedad de root y tiene una vulnerabilidad explotable (como la capacidad de ejecutar comandos arbitrarios, leer ficheros arbitrarios o escribir en el sistema de ficheros), permite escalar a root. La herramienta GTFOBins mantiene un inventario de binarios Unix que pueden ser usados para escalar privilegios cuando tienen el bit SUID activo.

Las capabilities de Linux son una alternativa a SUID más granular: permiten conceder a un proceso solo los privilegios específicos que necesita en lugar del conjunto completo de privilegios de root. Sin embargo, capabilities como cap_setuid, cap_net_admin y cap_sys_admin pueden ser abusadas para escalar privilegios si están asignadas a binarios explotables. La clase PrivilegeEscalationChecker enumera todas las capabilities asignadas a ficheros y procesos del sistema y las comprueba contra una lista de capabilities peligrosas.

```
# Vectores comprobados en PrivilegeEscalationChecker (Windows)
WINDOWS_VECTORS = [
    'unquoted_service_path',      # Servicios con rutas sin comillas
    'writable_path_directories',   # Directorios del PATH escribibles
    'writable_service_binaries',   # Binarios de servicios escribibles
    'scheduled_tasks_writable',    # Tareas con scripts escribibles
    'always_install_elevated',     # MSI con privilegios de SYSTEM
    'token_privileges',           # SeImpersonatePrivilege etc.
```

```
'dll_hijacking_services',    # Servicios susceptibles de DLL hijack
'registry_permissions',      # Claves de registro con permisos laxos
'autologon_credentials',     # Contraseñas en AutoLogon registry
'gpp_passwords',            # Passwords en Group Policy Preferences
]
```

8.3.2 LateralMovementDetector — Módulo 27

El movimiento lateral es la capacidad del equipo Red Team de propagarse por la red desde el sistema comprometido inicial hacia otros sistemas de mayor valor o mejor posicionados para alcanzar el objetivo final del ejercicio. En un ejercicio de Red Team típico, el initial access se consigue en un sistema de usuario final (el equipo de un empleado comprometido mediante phishing), y desde ahí es necesario moverse lateralmente hacia servidores, controladores de dominio, sistemas de gestión de infraestructura o repositorios de datos para conseguir el impacto demostrable que se incluirá en el informe.

La clase LateralMovementDetector evalúa y facilita cuatro técnicas principales de movimiento lateral. Pass-the-Hash (PTH) es la técnica de usar el hash NTLM de un usuario para autenticarse contra sistemas SMB/WMI/WinRM sin necesidad de conocer la contraseña en texto claro. El hash NTLM es funcionalmente equivalente a la contraseña para la autenticación NTLM: el protocolo challenge-response acepta el hash directamente como material de autenticación. Esta es una consecuencia del diseño del protocolo NTLM y no puede mitigarse sin cambiar el protocolo, aunque el uso de Protected Users Security Group y Credential Guard pueden limitar significativamente su efectividad en entornos modernos.

Pass-the-Ticket (PTT) es la técnica equivalente para Kerberos: reutilizar un ticket Kerberos válido (TGT o TGS) robado de la memoria de un sistema comprometido para autenticarse en otros servicios del dominio. Los tickets Kerberos tienen una validez de 10 horas por defecto (configurado en la política de contraseñas del dominio), lo que da al atacante una ventana suficiente para usarlos. Los tickets se exportan en formato CCACHE o en formato Kirbi de Mimikatz y se importan en la sesión del atacante mediante klist o las APIs de Kerberos.

WinRM (Windows Remote Management) es el protocolo de gestión remota de Windows basado en WS-Management (WS-Man), habilitado por defecto en Windows Server 2012 R2 y posteriores. WinRM permite la ejecución remota de comandos PowerShell (PowerShell Remoting) y el acceso a WMI de forma segura sobre HTTPS. La clase LateralMovementDetector comprueba qué sistemas de la red tienen WinRM habilitado (puerto

TCP 5985 para HTTP y 5986 para HTTPS) y con qué credenciales disponibles (obtenidas en fases anteriores) es posible establecer sesiones remotas de PowerShell.

La ejecución remota mediante WMI (Windows Management Instrumentation) es otra técnica de movimiento lateral que la clase implementa. WMI es la infraestructura de gestión de Windows que permite ejecutar comandos en sistemas remotos a través del protocolo DCOM (puerto TCP 135 y puertos dinámicos). WMI es especialmente útil para el movimiento lateral porque es una herramienta legítima del sistema que raramente está bloqueada por firewalls internos, y su uso para ejecución remota es difícil de distinguir del uso legítimo por parte de administradores o herramientas de gestión.

La técnica DCOM Lateral Movement explota aplicaciones COM que permiten la ejecución de código de forma implícita. La clase MMC20.Application, por ejemplo, permite invocar el método ExecuteShellCommand() de forma remota a través de DCOM, ejecutando cualquier comando en el sistema remoto con los privilegios del usuario autenticado.

```
# Técnicas de movimiento lateral implementadas
LATERAL_TECHNIQUES = {
    'pass_the_hash': {'proto': 'SMB/WMI', 'port': 445},
    'pass_the_ticket': {'proto': 'Kerberos', 'port': 88},
    'winrm_ps': {'proto': 'WS-Man', 'port': 5985},
    'wmi_exec': {'proto': 'DCOM', 'port': 135},
    'dcom_mmc20': {'proto': 'DCOM', 'port': 135},
    'psexec_smb': {'proto': 'SMB/SCM', 'port': 445},
    'rdp': {'proto': 'RDP', 'port': 3389},
}
```

8.3.3 TokenManipulator — Módulo 29

Windows usa un modelo de seguridad basado en tokens de acceso (Access Tokens) para controlar qué puede hacer cada proceso. Cada proceso tiene asociado un token primario que contiene la identidad del usuario, sus grupos, sus privilegios y otro material de seguridad. Cuando un proceso crea un proceso hijo, este hereda el token del padre por defecto, aunque el padre puede especificar un token diferente.

La clase TokenManipulator implementa varias técnicas de manipulación de tokens. Token Impersonation permite a un proceso adoptar temporalmente la identidad de otro usuario. Los procesos con el privilegio SeImpersonatePrivilege pueden llamar a ImpersonateLoggedOnUser() o ImpersonateNamedPipeClient() para ejecutar código bajo la identidad de otro usuario. Esta técnica es especialmente útil cuando el proceso comprometido

es un servicio IIS o SQL Server que tiene `SelImpersonatePrivilege` por defecto y puede impersonar a los usuarios que se conectan al servicio.

Token Duplication permite duplicar el token de acceso de un proceso existente y usarlo para crear nuevos procesos bajo esa identidad. El proceso comprometido debe tener el privilegio `SeDebugPrivilege` para poder abrir procesos de otros usuarios con `OpenProcessToken()`. Una vez obtenido el token, `DuplicateTokenEx()` crea una copia del token, y `CreateProcessWithTokenW()` o `SetThreadToken()` permiten usarlo para ejecución de código.

Make Token es una técnica diferente que crea un nuevo token a partir de credenciales de texto claro (usuario, dominio, contraseña) mediante la función `LogonUser()` de la API Win32. A diferencia de las técnicas anteriores que requieren obtener material de tokens existentes, Make Token solo requiere disponer de credenciales válidas (que pueden haber sido extraídas previamente por el `CredentialDumper`). El token resultante puede usarse para operaciones de red que requieren autenticación, aunque no crea un token de inicio de sesión interactivo con acceso a recursos locales.

La técnica Steal Token busca entre todos los procesos en ejecución del sistema aquellos cuyo token de acceso tiene privilegios superiores al del proceso del atacante, particularmente procesos ejecutados como SYSTEM o como otros usuarios con sesiones activas. El ataque consiste en encontrar un proceso adecuado, abrir su token con suficientes permisos, duplicarlo y usarlo para ejecutar código con la identidad y los privilegios del propietario del proceso original.

8.3.4 ProcessInjector — Módulo 30

La inyección de procesos es una familia de técnicas que permiten ejecutar código arbitrario dentro del espacio de memoria de un proceso legítimo del sistema operativo. Esta capacidad es fundamental para el trabajo de un equipo Red Team por dos razones: en primer lugar, el código inyectado hereda el contexto de seguridad del proceso host, lo que puede proporcionar privilegios adicionales o acceso a recursos del usuario del proceso; en segundo lugar, y más importante desde el punto de vista de la evasión, las soluciones antivirus y EDR modernas detectan nuevo código malicioso principalmente monitorizando la creación de nuevos procesos, la carga de módulos (DLLs) y la escritura de ficheros ejecutables al disco. La inyección de código en procesos existentes evita estos vectores de detección porque el código malicioso se ejecuta como parte de un proceso legítimo ya confiable.

DLL Injection es la variante más conocida y documentada. El mecanismo clásico usa cuatro llamadas a la API Win32: `VirtualAllocEx()` para reservar memoria en el espacio de direcciones del proceso objetivo, `WriteProcessMemory()` para escribir la ruta de la DLL maliciosa en esa memoria, `CreateRemoteThread()` con la dirección de la función `LoadLibraryA()` como función de inicio del thread, y `WaitForSingleObject()` para esperar a que `LoadLibraryA()` complete la carga de la DLL. Este método es efectivo pero relativamente ruidoso: crea un nuevo thread en el proceso objetivo y escribe un fichero DLL al disco, ambas acciones monitorizadas por las soluciones EDR modernas.

Process Hollowing es una variante más sofisticada que crea el proceso legítimo en estado suspendido (flag `CREATE_SUSPENDED` en `CreateProcess()`), desmapea el código original del proceso usando `NtUnmapViewOfSection()`, escribe el payload malicioso en el espacio de memoria liberado usando `WriteProcessMemory()`, ajusta el contexto del thread principal (`CONTEXT.Eip/Rip`) para apuntar al punto de entrada del nuevo código mediante `SetThreadContext()`, y finalmente reanuda la ejecución del thread con `ResumeThread()`. El resultado es un proceso que visualmente parece ser el ejecutable legítimo (mismo nombre, mismo PID, mismo proceso padre) pero que en realidad ejecuta el código del payload. Esta técnica evade los controles basados en hashes de ficheros porque el fichero en disco es el legítimo; el código malicioso solo existe en memoria.

Reflective DLL Injection es la técnica más avanzada, desarrollada originalmente por Stephen Fewer (2008). A diferencia de la DLL Injection clásica, la DLL maliciosa no se carga desde el disco sino desde un buffer de memoria. La DLL incluye un 'reflective loader', una pequeña función que implementa la funcionalidad del cargador de DLLs de Windows (resuelve importaciones, aplica relocations, etc.) sin pasar por las APIs estándar de carga de módulos. El proceso de inyección escribe el buffer completo de la DLL en memoria del proceso objetivo y ejecuta el reflective loader mediante `CreateRemoteThread()`. Como el cargador de Windows nunca interviene, la DLL nunca aparece en la lista de módulos cargados del proceso (como se ve en Process Hacker o Procmon), dificultando enormemente la detección.

Thread Hijacking es una variante que evita crear nuevos threads (un indicador de compromiso que los EDR monitorizan de cerca). En lugar de `CreateRemoteThread()`, el atacante suspende un thread existente del proceso objetivo con `SuspendThread()`, modifica su contexto de ejecución para que apunte al código del payload con `SetThreadContext()`, y reanuda el thread con `ResumeThread()`. El thread legítimo ejecutará el código del payload, y una vez que este termine, puede devolver el control al código legítimo restaurando el contexto original.

```
# Técnicas de inyección implementadas en ProcessInjector
INJECTION_TECHNIQUES = {
    'classic_dll':      'VirtualAllocEx + WriteProcessMemory +
CreateRemoteThread',
    'process_hollowing': 'CREATE_SUSPENDED + NtUnmapViewOfSection +
SetThreadContext',
    'reflective_dll':   'ReflectiveDLLLoader sin API del SO',
    'thread_hijacking': 'SuspendThread + SetThreadContext + ResumeThread',
    'apc_injection':    'QueueUserAPC() en thread alertable',
    'atom_bombing':     'GlobalAddAtom() + NtQueueApcThread',
}
```

8.4 Categoría: Persistencia y Evasión

Una vez que el equipo Red Team ha conseguido acceso con privilegios elevados en sistemas clave de la organización, el siguiente objetivo es garantizar que ese acceso se mantiene a lo largo del tiempo y que las operaciones pueden continuar aunque se produzcan reinicios del sistema, cambios de contraseñas o respuestas del equipo Blue Team. Los módulos de esta categoría implementan los mecanismos de persistencia más comunes en entornos Windows y las técnicas de evasión que permiten mantener el acceso sin ser detectado.

8.4.1 DLLHijackingScanner — Módulo 31

El DLL Hijacking es una técnica de persistencia y escalada de privilegios que explota el mecanismo de carga de DLLs de Windows. Cuando una aplicación de Windows llama a LoadLibrary() o LoadLibraryEx() con el nombre de una DLL (sin ruta absoluta), el sistema operativo sigue un orden de búsqueda predefinido para localizar el fichero DLL correspondiente. Este orden, conocido como 'DLL search order', es: 1) directorio del ejecutable de la aplicación, 2) directorio del sistema de Windows (System32), 3) directorio de sistema de 16 bits (System), 4) directorio de Windows, 5) directorio actual de trabajo, 6) directorios en la variable de entorno PATH.

La vulnerabilidad surge cuando dos condiciones se cumplen simultáneamente: la aplicación objetivo intenta cargar una DLL que no existe en las rutas 1-4 del orden de búsqueda (por ejemplo, una DLL opcional que puede o no estar instalada), y el atacante puede escribir ficheros en alguna de las rutas que se comprueban antes de la ubicación legítima de la DLL. En ese caso, colocar una DLL maliciosa con el nombre correcto en la ruta correcta hace que la aplicación la cargue en lugar de (o además de) la DLL legítima.

La clase `DLLHijackingScanner` implementa dos métodos de detección. El método pasivo analiza la lista de aplicaciones instaladas y sus directorios de instalación, comprobando los permisos de escritura de esos directorios para usuarios no privilegiados. Si el directorio de instalación de una aplicación que corre como `SYSTEM` o como un usuario privilegiado es escribible por usuarios normales, esa aplicación es un candidato para `DLL Hijacking`.

El método activo usa `Process Monitor` (o técnicas equivalentes de monitorización del sistema) para capturar en tiempo real las llamadas a `LoadLibrary` que resultan en '`NAME NOT FOUND`' en directorios con permisos de escritura. Esta técnica identifica de forma precisa qué `DLLs` están siendo buscadas pero no encontradas, y en qué directorios, proporcionando la información exacta necesaria para explotar la vulnerabilidad.

Las aplicaciones notoriamente susceptibles de `DLL Hijacking` incluyen muchas aplicaciones de terceros (instaladas en `C:\Program Files` con subdirectorios escribibles), así como algunas aplicaciones del propio sistema Windows en versiones antiguas (como el caso clásico de `iKE and AuthIP IPsec Keying Modules service`, `IKEExt.dll`). La clase mantiene una lista de `DLLs` conocidas como objetivos de alto valor para `DLL Hijacking` y las comprueba específicamente.

8.4.2 PersistenceChecker — Módulo 32

Los mecanismos de persistencia permiten al atacante mantener el acceso al sistema comprometido incluso tras un reinicio del sistema, un cambio de contraseña o una respuesta del equipo Blue Team que haya eliminado el vector de acceso original. La clase `PersistenceChecker` implementa la auditoría y opcionalmente la instalación de los mecanismos de persistencia más comunes en entornos Windows.

Las claves de registro `Run` y `RunOnce` son el mecanismo de persistencia más simple y más conocido. Las claves `HKLM\Software\Microsoft\Windows\CurrentVersion\Run` (para todos los usuarios) y `HKCU\Software\Microsoft\Windows\CurrentVersion\Run` (para el usuario actual) contienen una lista de programas que se ejecutan automáticamente al inicio de Windows. La clave `RunOnce` es similar pero elimina la entrada después de la primera ejecución, lo que la hace útil para instaladores pero no para persistencia a largo plazo. La clase `PersistenceChecker` también comprueba las variantes de 64 bits (`HKLM\Software\WOW6432Node\Microsoft\Windows\CurrentVersion\Run`) y las claves equivalentes para el inicio de sesión de usuario (`HKCU\Software\Microsoft\Windows\CurrentVersion\RunOnce`).

Las tareas programadas (Scheduled Tasks) son un mecanismo de persistencia más versátil que las claves Run porque permiten configurar condiciones de ejecución más complejas: al inicio del sistema, al inicio de sesión de un usuario específico, cada X minutos, cuando ocurre un evento específico en el log de eventos, cuando el sistema lleva idle X minutos, etc. Las tareas programadas creadas por el atacante pueden configurarse para ejecutar scripts PowerShell, ejecutables, o scripts VBScript, y pueden ser creadas en el contexto del sistema (SYSTEM) si el atacante tiene privilegios de administrador. La clase PersistenceChecker crea tareas que tienen apariencia legítima (nombres similares a tareas del sistema) para dificultar su detección.

Los servicios de Windows son el mecanismo de persistencia más persistente (valga la redundancia): se ejecutan en segundo plano como el sistema operativo, sobreviven a todos los reinicios y son gestionados por el SCM (Service Control Manager). Un servicio malicioso puede configurarse para iniciarse automáticamente (StartType = SERVICE_AUTO_START), ejecutarse bajo la cuenta SYSTEM (que tiene los máximos privilegios del sistema local) y reiniciarse automáticamente si falla (configurando la acción de recuperación en caso de fallo). La desventaja es que los servicios son visibles en la lista de servicios de Windows y son más fácilmente detectados que otros mecanismos.

Las WMI Event Subscriptions son uno de los mecanismos de persistencia más sigilosos disponibles en Windows. WMI permite suscribirse a eventos del sistema (inicio del sistema, inicio de sesión, creación de procesos, modificación de ficheros, etc.) y ejecutar código cuando ocurren. La suscripción se almacena en el repositorio WMI (un almacén de datos persistente en disco), no en el registro ni en el sistema de ficheros de forma directa, lo que la hace difícil de detectar para herramientas que solo buscan en ubicaciones estándar. La clase PersistenceChecker implementa la creación de suscripciones WMI que ejecutan código PowerShell cuando el sistema arranca o cuando se produce un evento específico.

8.4.3 FirewallEvasion — Módulo 33

La evasión de firewalls y otros controles de tráfico de red es fundamental para mantener la operatividad del canal de comunicación entre el equipo Red Team y los sistemas comprometidos. Los firewalls modernos van mucho más allá del simple filtrado por puerto y dirección IP: implementan inspección profunda de paquetes (DPI), detección de anomalías en protocolos, análisis de reputación de IPs y dominios, y en algunos casos incluso análisis de comportamiento del tráfico mediante machine learning.

La fragmentación de paquetes es una técnica básica de evasión que divide los paquetes en fragmentos pequeños con la esperanza de que el firewall no los reensamble antes de inspeccionarlos. Esta técnica fue efectiva contra firewalls de primera generación pero es ineficaz contra firewalls modernos con capacidad de seguimiento de estado (stateful inspection) que reensamblan automáticamente los fragmentos antes de la inspección. Sin embargo, puede ser útil contra dispositivos de red de gama baja o configuraciones de firewall permisivas.

La encapsulación de tráfico en protocolos permitidos es una técnica mucho más efectiva. El tráfico DNS en el puerto 53/UDP raramente es bloqueado por firewalls porque interrumpiría la resolución de nombres de toda la organización. El tráfico HTTPS en el puerto 443/TCP es igualmente difícil de bloquear sin interferir con la actividad web legítima. La clase FirewallEvasion implementa un módulo de DNS Tunneling que codifica los datos del canal C2 como nombres de subdominio en consultas DNS (ejemplo: dG9wc2VjcmV0.c2.atacante.com como consulta de resolución), y el servidor autorizado del atacante para ese dominio los decodifica y responde con datos del C2 en los registros TXT o A de la respuesta DNS.

Domain Fronting es una técnica de evasión avanzada que usa plataformas CDN (Content Delivery Network) como Cloudflare, Fastly o Amazon CloudFront como intermediarios del tráfico C2. El mecanismo aprovecha que el CDN descifra el TLS a nivel del servidor edge y reenvía la petición HTTP internamente según el header Host. El tráfico de red aparentemente va a un dominio legítimo y confiable (por ejemplo, un dominio de Microsoft en Cloudflare), pero el header Host interno redirige la petición al servidor del atacante. Los firewalls que inspeccionan el tráfico TLS solo ven la IP de Cloudflare y el dominio externo legítimo; el destino real está oculto dentro del cifrado TLS.

8.4.4 AVEvasion — Módulo 34

La evasión de antivirus y soluciones EDR (Endpoint Detection and Response) es uno de los desafíos técnicos más complejos del trabajo Red Team moderno. Las soluciones de seguridad de endpoint han evolucionado enormemente desde los antivirus basados en firmas de la era de los 2000: los EDR modernos (CrowdStrike Falcon, Microsoft Defender for Endpoint, SentinelOne, Carbon Black) implementan detección basada en comportamiento, machine learning, análisis de memoria en tiempo real y telemetría de eventos del kernel.

La obfuscación de código es la técnica de evasión más básica: modificar la representación del código malicioso para que no coincida con las firmas conocidas sin cambiar su comportamiento. Las técnicas incluyen codificación en Base64 o Hex del

shellcode, cifrado XOR con clave aleatoria (el código se descifra en tiempo de ejecución), inserción de instrucciones NOP (no-operation) o instrucciones irrelevantes entre el código real, y reordenamiento de bloques de código equivalentes funcionalmente pero distintos en representación binaria.

Las técnicas de living-off-the-land (LotL o LoLBins, Living Off the Land Binaries) son quizás las más efectivas contra soluciones que se basan en la detección de código malicioso: en lugar de usar herramientas propias que pueden ser detectadas, el atacante usa herramientas legítimas del propio sistema operativo para sus actividades maliciosas. PowerShell puede descargar y ejecutar código en memoria sin tocar disco, hacer peticiones HTTP, acceder a WMI, modificar el registro y hacer prácticamente cualquier cosa. certutil.exe puede decodificar Base64 y descargar ficheros. mshta.exe ejecuta scripts HTA. regsvr32.exe puede ejecutar código COM. wmic.exe puede ejecutar comandos remotos. La clase AVEvasion mantiene un catálogo actualizado de LoLBins con sus técnicas de abuso más comunes.

Sleep Obfuscation es una técnica moderna diseñada específicamente para evadir sandboxes de análisis dinámico que ejecutan el malware durante un tiempo limitado antes de rendir un veredicto. El código malicioso comienza haciendo sleep durante un período aleatorio (varios minutos) antes de ejecutar su funcionalidad real. Muchos sandboxes tienen un timeout de análisis inferior a este período de sleep, por lo que el código malicioso pasa como limpio. Técnicas más avanzadas de sleep obfuscation como Ekko o Foliage van más allá: durante el período de sleep, el código descifra su propia región de memoria ejecutable, la XOR con una clave aleatoria (haciendo el código no ejecutable y no reconocible como malicioso para el escaneo de memoria en tiempo real del EDR) y la vuelve a cifrar justo antes de reanudar la ejecución.

Heap Encryption y Stack Spoofing son técnicas adicionales de evasión de análisis de memoria. Los EDR modernos escanean periódicamente la memoria de los procesos en busca de regiones ejecutables con contenido sospechoso. Heap Encryption mantiene el shellcode cifrado en el heap y solo lo descifra temporalmente en la pila (stack) justo antes de la ejecución, reduciéndola la ventana temporal en que el código es visible en texto claro. Stack Spoofing falsifica la pila de llamadas (call stack) del thread actual para que las herramientas de análisis que monitorean las pilas de llamadas en busca de cadenas de llamadas sospechosas vean una pila de llamadas aparentemente legítima.

```
# Técnicas de evasión en AVEvasion
EVASION_TECHNIQUES = {
    'xor_encryption':      'Cifrado XOR con clave aleatoria en runtime',
```

```
'base64_obfuscation': 'Codificación multi-capa Base64',  
'sleep_evasion':      'Sleep inicial para evadir sandboxes',  
'heap_encryption':    'Shellcode cifrado en heap, descifrado en stack',  
'lolbins':            'PowerShell/certutil/mshta para ejecución',  
'amsi_bypass':        'Parcheo en memoria de AmsiScanBuffer()',  
'etw_bypass':         'Deshabilitación de Event Tracing for Windows',  
'process_unhooking':  'Restauración de syscalls hookeadas por EDR',  
}
```

8.5 Categoría: Técnicas Avanzadas

Los seis módulos de técnicas avanzadas cubren las fases finales de un ejercicio Red Team completo: la generación de los payloads que se despliegan en los sistemas comprometidos, el establecimiento y mantenimiento del canal de comunicación encubierto con el equipo Red Team, la exfiltración de datos de forma que evite los controles DLP, y los vectores de ataque inalámbricos y de ingeniería social que complementan los vectores técnicos.

8.5.1 PayloadGenerator — Módulo 35

La generación de payloads es el proceso de producir el código ejecutable o los scripts que se despliegan en los sistemas objetivo para establecer el canal de comunicación con el servidor del equipo Red Team (el C2). Un payload bien generado debe ser funcional (establecer la conexión C2), sigiloso (evadir las defensas del sistema objetivo) y adaptado al entorno (formato correcto para la forma de entrega prevista).

Los payloads Meterpreter para Metasploit Framework son los más versátiles y ampliamente usados en ejercicios de pentesting. Meterpreter opera completamente en memoria (sin escritura a disco), usa un protocolo de comunicación cifrado (TLS) y proporciona una interfaz rica de comandos que permite manipulación del sistema de ficheros, captura de pantalla, keylogging, pivoting de red y carga de extensiones adicionales. La clase PayloadGenerator puede producir Meterpreter en múltiples formatos: ejecutable Windows (EXE), DLL, shellcode C, script PowerShell, documento Office con macro, y script Python.

Los payloads para frameworks de post-explotación como Empire o Cobalt Strike Beacon son preferidos en ejercicios Red Team avanzados por sus capacidades de evasión superiores. Cobalt Strike Beacon, aunque es una herramienta comercial, está ampliamente documentada (y maliciosamente copiada por actores reales de amenaza), lo que hace que su generación y uso sea un componente representativo de los ejercicios más avanzados. La clase PayloadGenerator puede producir configuraciones de Beacon (Malleable C2 profiles)

que personalizan el comportamiento del tráfico de red del payload para imitar a aplicaciones web legítimas.

Los shellcodes son la forma más básica de payload: bloques de código máquina que se inyectan directamente en la memoria de un proceso mediante las técnicas del módulo ProcessInjector. La clase PayloadGenerator puede generar shellcodes para múltiples arquitecturas (x86, x64, ARM) con diferentes encoders (shikata_ga_nai, call4_dword_xor, countdown) para evadir las detecciones basadas en firmas del shellcode base.

Los payloads de macro para documentos Office son especialmente relevantes en ejercicios de Red Team que incluyen un componente de phishing. Los documentos Word y Excel con macros VBA son uno de los vectores de entrega de malware más efectivos en ataques reales porque los usuarios están acostumbrados a recibir documentos Office y a habilitar macros cuando se les solicita (especialmente si el señuelo está bien diseñado). La clase PayloadGenerator puede incrustar shellcode en una macro VBA que se ejecuta automáticamente cuando el documento se abre con macros habilitadas, usando técnicas de obfuscación para evadir la detección.

8.5.2 C2Communication — Módulo 36

La infraestructura de Command and Control (C2) es el sistema mediante el cual el equipo Red Team mantiene el control sobre los sistemas comprometidos durante el ejercicio. Un C2 efectivo debe ser resiliente (sobrevivir a intentos de bloqueo por parte del Blue Team), sigiloso (el tráfico debe parecer legítimo) y funcional (permitir la ejecución de comandos, la transferencia de ficheros y la gestión de múltiples sistemas comprometidos simultáneamente).

El canal HTTPS es el más robusto para comunicaciones C2 en entornos corporativos. El tráfico HTTPS en el puerto 443 es omnipresente en cualquier red corporativa y raramente es bloqueado de forma total. La clase C2Communication implementa un agente HTTPS que realiza peticiones GET periódicas a un servidor C2 legítimo de apariencia (con un dominio registrado, certificado SSL válido y landing page de aspecto normal) para obtener comandos pendientes, y envía los resultados mediante peticiones POST con el contenido cifrado en el cuerpo de la petición. El tráfico es indistinguible de una aplicación web normal para la mayoría de herramientas de inspección de tráfico que no realizan análisis de comportamiento.

El DNS Tunneling codifica los datos del canal C2 en consultas y respuestas DNS, aprovechando que el tráfico DNS es uno de los menos restringidos en cualquier red (bloquearlo rompería la resolución de nombres de toda la organización). El mecanismo de funcionamiento es el siguiente: el agente en el sistema comprometido codifica los datos a enviar al C2 en Base32 (para garantizar compatibilidad con el juego de caracteres de los nombres DNS) y los envía como consultas de resolución de subdominios del dominio controlado por el atacante (`datos_codificados.c2.atacante.com`). El servidor DNS autorizado del atacante para ese dominio recibe la consulta, extrae los datos del subdomain, procesa el comando y envía la respuesta codificada en los registros de respuesta DNS (registros TXT, A, o AAAA según el protocolo acordado).

El canal ICMP aprovecha que los paquetes ICMP Echo (ping) son raramente inspeccionados por firewalls porque se consideran tráfico de diagnóstico de red inofensivo. El agente embebe datos en el campo de datos de los paquetes ICMP Echo Request (que puede tener hasta 65507 bytes de payload), y el servidor C2 responde con los comandos codificados en los paquetes ICMP Echo Reply correspondientes. Esta técnica es especialmente útil en entornos con firewalls muy restrictivos que bloquean todo el tráfico TCP y UDP excepto DNS, pero permiten ICMP para diagnóstico de red.

La clase C2Communication también implementa el concepto de Domain Generation Algorithm (DGA), que genera automáticamente una lista de dominios candidatos para las comunicaciones C2 basándose en una semilla (por ejemplo, la fecha actual) y un algoritmo determinista. El agente itera sobre los dominios candidatos hasta encontrar uno que resuelva al servidor C2. Esta técnica hace extremadamente difícil para el Blue Team bloquear el C2 mediante listas negras de dominios, ya que el atacante solo necesita registrar uno de los miles de dominios candidatos y el agente lo encontrará automáticamente.

8.5.3 DataExfiltration — Módulo 37

La exfiltración de datos es la demostración final del impacto de un ejercicio Red Team: evidenciar que los datos más sensibles de la organización podrían haber sido extraídos sin ser detectados. En entornos corporativos con controles DLP (Data Loss Prevention) implementados, la exfiltración directa de ficheros grandes por canales obvios (email, FTP, HTTP sin cifrar) sería detectada inmediatamente. Los módulos de exfiltración de GhostTrack implementan técnicas que evitan estos controles.

La esteganografía oculta datos dentro de imágenes usando técnicas que producen imágenes visualmente indistinguibles de las originales. La técnica LSB (Least Significant Bit)

modifica el bit menos significativo de cada canal de color (R, G, B, A) de cada píxel para almacenar 1 bit de datos por canal, 4 bits por píxel en imágenes RGBA. Una imagen de 1 megapíxel puede almacenar aproximadamente 500 kilobytes de datos ocultos. La modificación del LSB produce cambios de color tan pequeños (1 parte en 256) que son completamente imperceptibles al ojo humano y a la mayoría de herramientas de inspección de imágenes. La imagen resultante puede ser enviada por email, publicada en redes sociales o subida a servicios de almacenamiento cloud sin levantar alertas en sistemas DLP que inspeccionan el contenido de los ficheros.

La exfiltración por DNS funciona de forma similar al DNS Tunneling descrito en el módulo C2: los datos se codifican en Base32 y se envían como consultas DNS de resolución de subdominios del dominio controlado por el atacante. La ventaja de la exfiltración DNS sobre otros canales es que el tráfico DNS es omnipresente y raramente inspeccionado en profundidad por los controles DLP. La desventaja es el bajo throughput: el protocolo DNS impone limitaciones de longitud en los nombres de dominio (máximo 63 caracteres por etiqueta, máximo 253 caracteres para el nombre completo), lo que limita la cantidad de datos que pueden enviarse en cada consulta. La exfiltración DNS es adecuada para datos de alto valor y baja cantidad (credenciales, claves de API, tokens de sesión), no para transferencias masivas de datos.

La exfiltración mediante servicios cloud legítimos es una técnica que los controles DLP tienen dificultades para detectar porque el tráfico va cifrado hacia dominios legítimos y de alta reputación. El agente de exfiltración carga los datos a un repositorio privado en GitHub, a una cuenta de Dropbox, a una cuenta de Google Drive o a una función serverless en AWS Lambda, usando las APIs oficiales de estos servicios sobre HTTPS. Los controles DLP que bloquean acceso a servicios cloud no autorizados pueden detectar esto, pero muchos entornos corporativos tienen estos servicios en lista blanca por su uso legítimo.

8.5.4 WiFiAttacker — Módulo 38

La superficie de ataque inalámbrica es frecuentemente subestimada en las evaluaciones de seguridad perimetral, que se centran en los sistemas accesibles desde Internet. Sin embargo, una red WiFi corporativa comprometida puede proporcionar al atacante acceso directo a la red interna, eludiendo todos los controles perimetrales. La clase WiFiAttacker implementa las técnicas de ataque a redes inalámbricas más relevantes en contextos de Red Team.

El ataque de captura de handshake WPA/WPA2 es el vector más efectivo contra redes con contraseñas débiles. WPA2-Personal usa el protocolo PBKDF2 (Password-Based Key Derivation Function 2) con HMAC-SHA1 para derivar la clave PMK (Pairwise Master Key) a partir de la contraseña de la red (PSK, Pre-Shared Key). El four-way handshake WPA2 incluye material criptográfico (nonces ANonce y SNonce) que permite verificar si una contraseña candidata genera la misma PMK que la red. El atacante captura el four-way handshake (que ocurre cuando un cliente se conecta a la red) y lo ataca offline con herramientas como Hashcat usando diccionarios y reglas de mutación.

La deautenticación forzada (Deauth Attack) es una técnica auxiliar que acelera la captura del handshake. En lugar de esperar pasivamente a que un cliente se conecte, el atacante envía paquetes de deautenticación 802.11 (management frames que no requieren autenticación para ser enviados en redes sin 802.11w) al cliente objetivo, forzando su desconexión y posterior reconexión al AP. Durante la reconexión, el four-way handshake tiene lugar y puede ser capturado. Esta técnica es detectable por sistemas WIDS (Wireless Intrusion Detection Systems) que monitorizan floods de frames de deautenticación.

Los ataques contra WPA3 son significativamente más difíciles que contra WPA2 debido a la introducción del protocolo SAE (Simultaneous Authentication of Equals, también conocido como Dragonfly). SAE reemplaza el PSK directamente por un protocolo de intercambio de clave que proporciona Perfect Forward Secrecy y previene los ataques de diccionario offline del handshake. Sin embargo, la transición a WPA3 en entornos empresariales es lenta, y las redes en modo de transición (WPA2/WPA3 mix) son susceptibles a ataques de downgrade que fuerzan la conexión por el protocolo más débil.

8.5.5 BluetoothScanner — Módulo 39

Bluetooth es una tecnología de comunicación inalámbrica de corto alcance (típicamente 10-100 metros, hasta 400 metros en Bluetooth 5.x con antenas de alta ganancia) ampliamente presente en entornos corporativos en dispositivos como auriculares, ratones, teclados, altavoces, smartphones y portátiles. La superficie de ataque Bluetooth es frecuentemente ignorada en las evaluaciones de seguridad.

La clase BluetoothScanner implementa el descubrimiento pasivo de dispositivos Bluetooth mediante el modo de escucha de advertising packets del protocolo BLE (Bluetooth Low Energy). Los dispositivos BLE emiten periódicamente advertising packets que anuncian su presencia y sus capacidades. Estos packets son recibidos por cualquier dispositivo con el radio BLE en modo scan, sin necesidad de establecer una conexión ni de que el dispositivo

objetivo sea 'discoverable'. La información obtenida de los advertising packets incluye la dirección MAC del dispositivo (o una dirección aleatoria si el dispositivo usa privacy mode), el nombre del dispositivo, los servicios anunciados y el nivel de señal RSSI.

El protocolo SDP (Service Discovery Protocol) de Bluetooth Classic permite consultar a un dispositivo en rango qué servicios ofrece. La clase BluetoothScanner realiza consultas SDP a los dispositivos descubiertos para obtener el perfil de servicios completo: si exponen un perfil HFP (Hands-Free Profile), HSP (Headset Profile), A2DP (Advanced Audio Distribution Profile), AVRCP (Audio/Video Remote Control Profile), HID (Human Interface Device), SPP (Serial Port Profile), u otros. Cada perfil tiene implicaciones de seguridad distintas: un perfil HID que acepta conexiones sin autenticación podría permitir la inyección de pulsaciones de teclado (BadBT / BlueHydra attack).

La vulnerabilidad BlueBorne (CVE-2017-0781 a CVE-2017-0783, CVE-2017-8628, CVE-2017-14315), descubierta por Armis Security en 2017, es especialmente significativa: afecta a dispositivos Android, Windows, iOS y Linux con Bluetooth habilitado y permite la ejecución remota de código sin necesidad de emparejamiento previo ni interacción del usuario. El atacante solo necesita estar en el rango Bluetooth del dispositivo objetivo. La clase BluetoothScanner detecta dispositivos potencialmente vulnerables a BlueBorne comprobando la versión del firmware anunciada y comparándola con las versiones afectadas documentadas.

8.5.6 SocialEngineeringToolkit — Módulo 40

La ingeniería social es la manipulación psicológica de personas para que realicen acciones o divulguen información confidencial. En el contexto de los ejercicios Red Team, es con frecuencia el vector de initial access más efectivo: mientras que las vulnerabilidades técnicas pueden ser parcheadas y los sistemas pueden ser endurecidos, el comportamiento humano es mucho más difícil de 'parchear'. Los estudios de la industria estiman consistentemente que más del 90% de los ciberataques exitosos comenzaron con un elemento de ingeniería social, típicamente phishing (Verizon, 2023).

El phishing es el vector de ingeniería social más común. Un email de phishing bien diseñado combina urgencia (el usuario siente que debe actuar inmediatamente), autoridad (aparentemente proviene de una figura de autoridad como el CEO o el departamento de IT), relevancia (el contenido es relevante para el contexto laboral del receptor) y similitud visual con comunicaciones legítimas. La clase SocialEngineeringToolkit proporciona tres herramientas principales para el diseño de campañas de phishing.

El clonado de páginas web captura el HTML, CSS, JavaScript e imágenes de una página de autenticación legítima (portal de Microsoft 365, Outlook Web Access, VPN SSL corporativa, aplicación interna) y la sirve desde infraestructura controlada por el atacante con modificaciones mínimas para capturar las credenciales introducidas por el usuario. El proceso incluye la descarga de todos los recursos estáticos (imágenes, fuentes, scripts), la adaptación de las URLs para que apunten a la infraestructura del atacante, y la adición de un script de captura de credenciales que registra y retransmite las credenciales introducidas antes de redirigir al usuario a la página legítima (para que no sospeche del engaño).

La generación de dominios typosquatting crea variantes del dominio legítimo de la organización objetivo que son visualmente similares pero ligeramente distintas: adición de caracteres (empresa-corp.com), sustitución de letras por homóglifos visuales (empresa.com con la e cirílica), transposición de caracteres (emrpesa.com), adición de TLDs alternativos (empresa.co en lugar de empresa.com), o uso de IDN (Internationalized Domain Names) para crear dominios que parecen idénticos en pantalla pero tienen representaciones Unicode diferentes.

La personalización de señuelos es el elemento que diferencia el phishing genérico del spear phishing (phishing selectivo contra individuos específicos). La clase SocialEngineeringToolkit integra la información recopilada en las fases de reconocimiento para personalizar el contenido de los emails de phishing: usa el nombre real del receptor (obtenido de LinkedIn o del Email Harvester), referencia eventos reales de la organización (obtenidos de la web corporativa o de noticias recientes), imita el estilo de escritura de la persona desde cuya dirección aparentemente se envía el email (si se dispone de emails anteriores), y adjunta documentos con nombres relevantes para el trabajo del receptor.

```
# Pipeline completo de una campaña de spear phishing
1. EmailHarvester      → obtener emails del objetivo
2. LDAPEnumerator      → obtener nombre completo, cargo, departamento
3. OSINT manual        → LinkedIn, web corporativa, noticias recientes
4. SET.clone_website   → clonar portal de autenticación corporativo
5. SET.typosquatting    → registrar dominio similar (empresa-secure.com)
6. SET.craft_email     → personalizar señuelo con información OSINT
7. SET.track_campaign  → monitorizar clics, credenciales capturadas
8. CredentialDumper    → usar credenciales capturadas para acceso
```

8.6 Resumen del Bloque Red Team

Los veinte módulos Red Team de GhostTrack cubren de forma comprehensiva las fases avanzadas de un ejercicio de evaluación ofensiva. La siguiente tabla resume el mapeo completo entre módulos, categorías, tácticas MITRE ATT&CK y nivel de impacto esperado en entornos de producción:

Tabla 5*Resumen completo de módulos Red Team de GhostTrack* Nota. Elaboración propia basada en MITRE Corporation (2023). Nivel de impacto: CRÍTICO = compromiso del dominio completo posible; ALTO = acceso privilegiado a sistemas clave; MEDIO = información valiosa para fases posteriores.

Módulo	Categoría	Táctica ATT&CK principal	Impacto	Contramedida principal
SMBEnumerator	Enumeración AD	Discovery	MEDIO	SMB Signing obligatorio + deshab sessions
LDAPEnumerator	Enumeración AD	Discovery	MEDIO	LDAP Signing + LDAP Channel B
KerberosAttacker	Cred. Access	Credential Access	CRÍTICO	Contraseñas largas en cuentas de + MSA/gMSA
NTLMRelayAttacker	Cred. Access	Credential Access	CRÍTICO	SMB Signing obligatorio + deshab LLMNR/NBT-NS
BloodHoundCollector	Enumeración AD	Discovery	ALTO	Tier model + Privileged Access Workstations
PrivEscChecker	Post-Exploit.	Privilege Escalation	CRÍTICO	Auditoría de permisos + Credentia
LateralMovement	Post-Exploit.	Lateral Movement	CRÍTICO	Network segmentation + EDR + M
CredentialDumper	Post-Exploit.	Credential Access	CRÍTICO	Credential Guard + Protected Use LAPS
TokenManipulator	Post-Exploit.	Privilege Escalation	ALTO	Deshabilitar SelmpersonatePrivile cuando sea posible
ProcessInjector	Post-Exploit.	Defense Evasion	ALTO	EDR con análisis de memoria + E Protection
DLLHijacking	Persistencia	Persistence	ALTO	Permisos estrictos + aplicaciones lectura solo
Persistence	Persistencia	Persistence	ALTO	Monitorización de autoruns + WMI subscriptions
FirewallEvasion	Evasión	Defense Evasion	MEDIO	DPI + SSL inspection + proxy web corporativo
AVEvasion	Evasión	Defense Evasion	ALTO	EDR comportamental + memory s + ETW
PayloadGenerator	Téc. Avanzadas	Execution	ALTO	Application Control (AppLocker/W

Módulo	Categoría	Táctica ATT&CK principal	Impacto	Contramedida principal
C2Communication	Téc. Avanzadas	Command and Control	CRÍTICO	DNS filtering + TLS inspection + behavioral analysis
DataExfiltration	Téc. Avanzadas	Exfiltration	CRÍTICO	DLP + CASB + DNS filtering + data classification
WiFiAttacker	Téc. Avanzadas	Initial Access	ALTO	WPA3 + 802.11w + redes corporativas segregadas
BluetoothScanner	Téc. Avanzadas	Discovery	MEDIO	Deshabilitar Bluetooth cuando no es necesario
SocialEngineering	Téc. Avanzadas	Initial Access	CRÍTICO	Formación usuarios + MFA + sanción email

9. Utilidades del Framework

El paquete `utils/` contiene los cuatro módulos de utilidades transversales que proporcionan servicios comunes a todos los módulos de GhostTrack. Estos módulos son completamente independientes entre sí y no tienen dependencias con los módulos de negocio, lo que les permite ser probados de forma unitaria sin necesidad de simular el entorno de pentesting.

9.1 Colors — Gestión de la Interfaz de Terminal

La clase `Colors` del módulo `utils/colors.py` centraliza la gestión de los códigos de escape ANSI para la colorización de la salida de terminal. Los códigos ANSI (definidos en el estándar ECMA-48) permiten controlar el color del texto, el color del fondo, los estilos tipográficos (negrita, cursiva, subrayado) y otras propiedades visuales de la salida en terminales compatibles.

La clase define cuatro categorías de constantes de clase. Los colores básicos (`BLACK`, `RED`, `GREEN`, `YELLOW`, `BLUE`, `MAGENTA`, `CYAN`, `WHITE`) corresponden a los colores estándar del ANSI estándar de 8 colores. Los colores brillantes (`BRIGHT_BLACK`, `BRIGHT_RED`, etc.) son versiones más saturadas de los colores básicos, disponibles en la extensión de 16 colores. Los estilos (`BOLD`, `DIM`, `ITALIC`, `UNDERLINE`, `BLINK`, `REVERSE`) controlan atributos tipográficos del texto. Finalmente, los alias semánticos mapean significados de contexto a colores: `SUCCESS` apunta a `GREEN`, `WARNING` a `YELLOW`, `ERROR` a `RED` e `INFO` a `CYAN`.

El código de reset RESET ('\033[0m') restablece todos los atributos a sus valores por defecto y es fundamental incluirlo al final de cada cadena colorizada para evitar que los atributos se propaguen a texto posterior.

Los métodos estáticos `colorize()`, `success()`, `warning()`, `error()` e `info()` encapsulan el patrón de concatenación `código_color + texto + RESET` en llamadas de una sola línea, simplificando el código de los módulos consumidores y garantizando que siempre se incluye el reset al final.

La compatibilidad con Windows se gestiona mediante la librería `colorama`, que intercepta las llamadas de escritura a `stdout/stderr` y convierte los códigos de escape ANSI en llamadas a la API de la consola de Windows. La inicialización de `colorama` (`colorama.init()`) debe realizarse al inicio de la aplicación, lo que en GhostTrack se hace en `main.py` mediante la importación de `Colors`.

9.2 Logger — Sistema de Registro de Operaciones

El módulo `utils/logger.py` implementa un sistema de logging estructurado que registra todas las operaciones del framework en ficheros de log con rotación automática. El uso de logging en un framework de pentesting no es solo una buena práctica de desarrollo: es un requisito funcional que permite reconstruir exactamente qué hizo el framework durante una sesión, lo que es esencial para la defensa en caso de disputas sobre el alcance y para la reproducibilidad de los hallazgos durante la elaboración del informe.

La clase `Logger` encapsula un objeto `logging.Logger` de la biblioteca estándar de Python. El `Logger` está configurado con el nivel `DEBUG`, lo que significa que todos los mensajes de cualquier severidad se procesan. Sin embargo, los dos handlers configurados filtran los mensajes de forma diferente: el `FileHandler` escribe todos los mensajes (`DEBUG` y superiores) al fichero de log, mientras que el `StreamHandler` muestra en consola solo los mensajes de nivel `INFO` y superiores, evitando que la salida de depuración sature la interfaz del usuario.

La rotación de ficheros de log se gestiona mediante la política de nombrado con timestamp: cada sesión de GhostTrack crea un nuevo fichero de log con el nombre `'ghosttrack_YYYYMMDD_HHMMSS.log'` en el directorio `logs/`. Esto garantiza que los logs de diferentes sesiones no se mezclan y facilita la correlación temporal de eventos. La configuración del sistema de logging de Python también soporta `RotatingFileHandler` y `TimedRotatingFileHandler` para gestión automática del tamaño y antigüedad de los logs, que son alternativas que podrían implementarse en versiones futuras del framework.

El formato de cada línea de log es: '%(asctime)s - %(name)s - %(levelname)s - %(message)s', que produce entradas como '2026-04-11 10:23:45 - GhostTrack - INFO - Iniciando port_scan en 192.168.1.100'. Este formato incluye toda la información necesaria para correlacionar los logs con los reportes generados.

Los métodos de conveniencia semánticos (`log_scan_start`, `log_scan_end`, `log_module_execution`, `log_vulnerability_found`) garantizan que estos eventos críticos siempre se registran con el mismo formato y nivel de severidad independientemente del módulo que los invoque. `log_vulnerability_found()` usa nivel `WARNING` para garantizar que los hallazgos de seguridad son prominentes en el log y fácilmente identificables con herramientas de análisis de logs como `grep` o `Splunk`.

9.3 Validator — Validación Centralizada de Entradas

La clase `Validator` del módulo `utils/validator.py` centraliza toda la lógica de validación de entradas del framework. La centralización de la validación en un único módulo tiene varias ventajas: evita la duplicación de código de validación en múltiples módulos, garantiza la consistencia de los criterios de validación, facilita la actualización de las reglas de validación en un único lugar y hace que la validación sea fácilmente testeable de forma unitaria.

Todos los métodos de `Validator` son estáticos (`@staticmethod`), lo que refleja que la validación es una función pura sin estado: dado un input, siempre produce el mismo output. Esto también permite usar los métodos de `Validator` sin necesidad de instanciar la clase.

El método `validate_ip(ip)` usa el módulo `ipaddress` de la biblioteca estándar, que proporciona clases para trabajar con objetos de dirección IP que validan automáticamente el formato. Acepta tanto IPv4 (192.168.1.1) como IPv6 (2001:db8::1).

El método `validate_domain(domain)` aplica una expresión regular que verifica la estructura de un Fully Qualified Domain Name (FQDN): uno o más segmentos de hasta 63 caracteres alfanuméricos o guiones (pero no comenzando o terminando en guión, conforme al RFC 1123), separados por puntos, con el TLD de al menos dos caracteres. La expresión regular excluye deliberadamente las IPs y los nombres de host sin punto (`localhost`) para mantener una separación clara entre los tipos de input.

El método `validate_target(target)` combina `validate_ip()` y `validate_domain()` en una función de propósito general que acepta cualquier tipo de identificador de host. Como último recurso, intenta resolver el `target` como `hostname` mediante `socket.gethostbyname()` para aceptar nombres de host que no cumplen estrictamente el formato FQDN pero que son válidos en el sistema operativo.

El método `sanitize_filename(filename)` protege contra ataques de path traversal eliminando los caracteres que tienen significado especial en sistemas de ficheros: `< > : " / \ | ? *` (caracteres prohibidos en nombres de fichero de Windows) y limitando la longitud total a 200 caracteres para evitar problemas con sistemas de ficheros con límites de longitud de nombre.

9.4 Reporter — Generación de Informes Multipropósito

La clase Reporter del módulo `utils/reporter.py` es el componente más complejo de las utilidades del framework. Su responsabilidad es transformar los diccionarios de resultados producidos por los módulos en documentos de informe formales y legibles en tres formatos complementarios, cada uno orientado a un caso de uso diferente.

El método `generate()` orquesta la generación simultánea de los tres formatos. Recibe el target, el diccionario de resultados completo y el tipo de escaneo, construye el nombre base del fichero con el patrón `'target_tipodeescaneo_timestamp'` (donde el target es sanitizado mediante `Validator.sanitize_filename()`), y delega en los tres métodos específicos de generación.

Reporte JSON

El reporte JSON se genera mediante `json.dump()` con indentación de cuatro espacios y el parámetro `default=str` para manejar tipos no serializables por defecto (como objetos `datetime` o conjuntos Python) convirtiéndolos a su representación en cadena. El resultado es un fichero JSON completamente estructurado que puede ser procesado por cualquier herramienta o lenguaje de programación que soporte el formato JSON. Este formato es el más útil para la integración con sistemas externos: parsers automáticos, dashboards de seguridad (Kibana, Grafana), sistemas de ticket (Jira, ServiceNow) o plataformas de gestión de vulnerabilidades.

Reporte HTML

El reporte HTML incluye una hoja de estilos CSS embebida con paleta de colores oscura profesional característica de las herramientas de ciberseguridad. El diseño incluye un header con gradiente de color y el título del informe, una sección de información general del escaneo (target, tipo, fecha, número de módulos), y secciones individuales para cada módulo con sus resultados formateados. El código de colores de los findings sigue la convención estándar de la industria: rojo para CRITICAL, naranja para HIGH, azul para MEDIUM y verde para LOW, implementado mediante clases CSS específicas para cada nivel de severidad.

Reporte TXT

El reporte de texto plano está diseñado para máxima portabilidad: no requiere ningún software específico para ser visualizado, puede ser enviado por email sin problemas, impreso directamente y procesado con herramientas de línea de comandos como grep, awk o sed para extraer información específica. Utiliza caracteres ASCII para los separadores visuales (==, —) y expone los datos de cada módulo en formato JSON indentado dentro de secciones delimitadas claramente.

10. Sistema de Reportes

El sistema de reportes de GhostTrack ha sido diseñado siguiendo los principios de los informes de auditoría de seguridad profesionales. Un buen informe de pentesting no es solo un volcado de datos técnicos; es un documento que comunica de forma clara y priorizada qué se ha encontrado, qué riesgo real representa para la organización y qué acciones concretas deben tomarse para remediar los hallazgos (Engebretson, 2013).

La estructura del informe generado por GhostTrack sigue las recomendaciones del estándar PTES para la fase de Reporting: una sección de resumen ejecutivo orientada a la dirección no técnica, una sección técnica detallada orientada al equipo de seguridad que debe remediar los hallazgos, y una sección de apéndices con los datos crudos de cada módulo para referencia.

10.1 Generación Automática de Informes

Los informes se generan automáticamente al finalizar cualquier escaneo, sin necesidad de ninguna acción adicional por parte del auditor. El nombre del fichero sigue el patrón '{target}_{tipo_escaneo}_{timestamp}.{extensión}', donde el timestamp tiene el formato YYYYMMDD_HHMMSS. Este patrón de nombrado garantiza la unicidad de los ficheros (no es posible que dos sesiones generen ficheros con el mismo nombre), facilita la ordenación cronológica de los informes y permite identificar de un vistazo el target y el tipo de análisis de cada informe.

Todos los informes se almacenan en el directorio reports/, que está incluido en el fichero .gitignore para evitar que los informes de auditorías reales sean accidentalmente publicados en el repositorio de código. El directorio logs/ recibe el mismo tratamiento por las mismas razones.

10.2 Reporte HTML: Diseño Visual Profesional

El reporte HTML está diseñado para ser presentable a clientes y directivos sin conocimientos técnicos. El diseño sigue las convenciones visuales de las herramientas de seguridad de referencia del sector (Burp Suite Professional, Nessus, Tenable.io). La paleta de colores oscura (fondo #0a0e27, texto #e0e0e0) es característica del sector y facilita la lectura de código y datos técnicos en sesiones de trabajo largas.

Cada módulo tiene su propia sección colapsable en el informe, lo que permite al lector navegar rápidamente a los módulos de mayor interés. Los hallazgos de seguridad están

codificados visualmente con un borde de color en el lado izquierdo: rojo para CRITICAL (#f44336), naranja para HIGH (#ff9800), azul para MEDIUM (#2196F3) y verde para LOW (#4CAF50). Este código de colores es consistente con la clasificación CVSS estándar y es reconocible para cualquier profesional de seguridad.

10.3 Reporte JSON: Integración con Herramientas

El formato JSON del reporte es el más útil para la integración técnica con otros sistemas. La estructura del JSON sigue un esquema consistente: el nivel raíz contiene los metadatos del escaneo (target, timestamp, scan_type), y el campo 'modules' contiene un diccionario indexado por nombre de módulo, donde cada valor es el diccionario de resultados producido por ese módulo. Esta estructura facilita el procesamiento programático: para acceder a los resultados del Port Scanner, basta con `data['modules']['ports']`.

El formato JSON puede ser ingestado directamente por plataformas de SIEM (Security Information and Event Management) como Splunk o Elasticsearch, por plataformas de gestión de vulnerabilidades como DefectDojo, o por scripts de automatización de remediación escritos en cualquier lenguaje que soporte JSON.

10.4 Reporte TXT: Portabilidad Universal

El formato TXT plano es el más universal: no requiere ningún software específico para ser visualizado y puede ser manipulado con las herramientas de línea de comandos más básicas disponibles en cualquier sistema Unix. El uso de caracteres de línea ASCII para los separadores (== para secciones principales, — para subsecciones) proporciona estructura visual sin depender de ninguna codificación especial. Los datos de cada módulo se exponen en formato JSON dentro de las secciones, lo que permite usar herramientas como jq para extraer campos específicos directamente del fichero TXT.

11. Interfaz de Línea de Comandos (CLI)

La interfaz de línea de comandos de GhostTrack está implementada con el módulo `argparse` de la biblioteca estándar de Python. `argparse` proporciona análisis automático de los argumentos de la línea de comandos, generación de mensajes de ayuda (accesibles con `--help`), validación de tipos de datos y gestión de argumentos obligatorios y opcionales.

11.1 Diseño de la CLI

La CLI de GhostTrack sigue los principios de diseño de interfaces de línea de comandos definidos en *The Art of Unix Programming* (Raymond, 2003): cada flag hace una sola cosa y la hace bien, los flags con nombres largos son autodocumentados (`--redteam` es más claro que `-r`), los valores por defecto son razonables para el caso de uso más común, y el sistema de ayuda integrado es completo y útil.

El epílogo del parser `argparse` incluye una sección de ejemplos de uso que ilustra los casos de uso más comunes con comandos completos y listos para ejecutar. Esta característica es especialmente útil para usuarios que están aprendiendo a usar el framework.

11.2 Argumentos Disponibles

Tabla 3 Argumentos de la CLI de GhostTrack Nota. Elaboración propia.

Argumento	Tipo	Default	Descripción
<code>-t / --target</code>	str	—	IP o nombre de dominio del objetivo. Requerido para cualquier escaneo.
<code>--full</code>	flag	False	Ejecuta los 20 módulos básicos en secuencia completa.
<code>--redteam</code>	flag	False	Ejecuta los 20 módulos Red Team. Requiere confirmación adicional.
<code>--domain</code>	str	None	Dominio Active Directory para módulos Red Team (ej: corp.local).
<code>-m / --module</code>	str	None	Ejecuta un único módulo por nombre (ej: -m ssl_analyzer).
<code>--inject</code>	str	None	Ruta a un script Python personalizado para inyectar y ejecutar.
<code>--threads</code>	int	10	Número de threads concurrentes para módulos multi-threaded.
<code>--timeout</code>	int	5	Timeout por operación de red en segundos.

Argumento	Tipo	Default	Descripción
-o / --output	str	./reports	Directorio de salida para los reportes generados.
--local-network	flag	False	Activa módulos específicos de análisis de red local.
--files	str+	None	Lista de ficheros para análisis de metadatos (módulo 19).
--list-modules	flag	False	Muestra la lista completa de módulos y termina.
-v / --verbose	flag	False	Activa salida detallada con información de depuración.
--version	flag	—	Muestra la versión del framework (GhostTrack 2.0) y termina.

11.3 Ejemplos de Uso

A continuación se presentan los patrones de uso más comunes de la CLI:

```
# Mostrar todos los módulos disponibles
python main.py --list-modules
```

```
# Escaneo completo de 20 módulos básicos con 20 threads
python main.py -t 192.168.1.100 --full --threads 20
```

```
# Escaneo completo con timeout personalizado y directorio de salida
python main.py -t target.com --full --timeout 10 -o /tmp/audit/
```

```
# Red Team Assessment de entorno Active Directory
python main.py -t dc01.corp.local --redteam --domain corp.local
```

```
# Escaneo total: los 40 módulos
python main.py -t target.com --full --redteam --domain corp.local
```

```
# Módulo individual: análisis SSL
python main.py -t target.com -m ssl_analyzer
```

```
# Extracción de metadatos de múltiples ficheros
python main.py -t target.com --files doc1.pdf img1.jpg img2.png
```

```
# Script personalizado
python main.py -t 192.168.1.50 --inject scripts/spring_actuator.py
```

11.4 Flujo de Validación de la CLI

Cuando se ejecuta GhostTrack, la CLI aplica las siguientes validaciones en orden antes de comenzar cualquier operación:

1. Verificación de la versión de Python (mínimo 3.8). Si no se cumple, el programa termina con un mensaje de error claro.
2. Parsing de argumentos con argparse. Si los argumentos tienen tipos incorrectos o faltan argumentos requeridos, argparse muestra el mensaje de ayuda y termina con código de error 2.
3. Si se solicita --list-modules, se muestra la lista y se termina con código 0.
4. Si no se proporciona --target, se muestra la ayuda y se termina con código de error.
5. El Validator verifica que el target es una IP válida, un dominio válido o un hostname resoluble. Si no lo es, se informa del error y se termina.
6. Se muestra el disclaimer legal completo y se solicita confirmación explícita. Si el usuario no confirma, se termina con código 0.
7. Se inicializa el Logger y se registra el inicio de la sesión con el target, los parámetros y el usuario del sistema operativo.
8. Se ejecutan los módulos según los flags proporcionados.

12. Flujo de Ejecución

El flujo de ejecución completo de GhostTrack, desde el comando del usuario hasta la generación de reportes, puede describirse mediante un diagrama de secuencia de nueve etapas principales. Cada etapa está diseñada para ser idempotente y para gestionar correctamente los errores sin afectar a las etapas siguientes.

12.1 Etapa 1: Arranque e Importaciones

Python ejecuta `main.py`, que en primer lugar verifica la versión del intérprete con `sys.version_info`. Si la versión es inferior a 3.8, el programa imprime un mensaje de error explicativo y termina con `sys.exit(1)`. A continuación, se importan los módulos de utilidades desde el paquete `utils/` y los veinte módulos básicos desde `modules/`. Las importaciones de los módulos Red Team se realizan en un bloque `try/except` separado para que los errores de importación de dependencias opcionales no impidan el arranque básico del framework.

12.2 Etapa 2: Orquestación de Módulos

El método `run_full_scan()` de la clase `GhostTrack` ejecuta los veinte módulos básicos en orden secuencial. Para cada módulo, el método imprime un mensaje de progreso indicando el número de módulo y el nombre de la operación (por ejemplo `'[*] Módulo 5/20: Port Scanning'`), invoca la función de interfaz del módulo y almacena el resultado en el diccionario `results['modules']` bajo una clave descriptiva.

El flujo de ejecución es deliberadamente secuencial (no paralelo entre módulos) por dos razones: primero, muchos módulos dependen de los resultados de módulos anteriores (Service Detection necesita los puertos abiertos de Port Scanner; Vulnerability Scanner necesita los servicios de Service Detection); segundo, la ejecución secuencial facilita la depuración y el seguimiento del progreso.

Los errores dentro de un módulo individual se capturan con `try/except` para que un fallo en un módulo no interrumpa la ejecución de los módulos siguientes. En caso de error, se registra en el log con nivel `ERROR` y se almacena en el resultado del módulo como `{'error': mensaje_del_error}`.

12.3 Etapa 3: Generación de Reportes

Al finalizar la ejecución de todos los módulos configurados, el método `run_full_scan()` pasa el diccionario de resultados completo al `Reporter`. El `Reporter` genera los tres formatos

de informe en el directorio configurado. Los nombres de los ficheros se imprimen en la consola para que el auditor pueda localizarlos inmediatamente.

12.4 Gestión de Interrupciones

La función `main()` envuelve la ejecución completa en un bloque `try/except/finally`. El `except KeyboardInterrupt` captura la interrupción del usuario (Ctrl+C) y imprime un mensaje informativo antes de terminar limpiamente. El `except Exception` captura cualquier error no manejado y lo registra en el log con nivel `CRITICAL` y con el `traceback` completo para facilitar la depuración. El bloque `finally` registra el fin de la sesión en el log independientemente de cómo haya terminado la ejecución.

13. Guía de Instalación y Configuración

Este capítulo describe el proceso completo de instalación de GhostTrack en los tres sistemas operativos soportados (Linux, macOS y Windows), los requisitos del sistema y las opciones de configuración disponibles.

13.1 Requisitos del Sistema

Requisitos mínimos

- Python 3.8 o superior (se recomienda Python 3.10+ para mejor rendimiento). Verificable con: `python --version` o `python3 --version`.
- pip (gestor de paquetes de Python). Habitualmente incluido con Python; verificable con: `pip --version`.
- git (para clonar el repositorio). Disponible en la mayoría de sistemas; verificable con: `git --version`.
- Conexión a Internet para la descubrimiento activo y las consultas a APIs externas (GeoIP, WHOIS, Certificate Transparency).

Requisitos según módulos

- Módulo Network Sniffer (17): Privilegios de administrador/root. En Linux, puede concederse mediante: `sudo setcap cap_net_raw+eip $(which python3)`.
- Módulos Red Team avanzados (24, 28, 29, 30): Dependencias opcionales (impacket, ldap3, pycryptodome).
- Módulo WiFiAttacker (38): Interfaz de red inalámbrica compatible con modo monitor y la herramienta aircrack-ng.

13.2 Instalación Paso a Paso

Linux/macOS

```
# 1. Clonar el repositorio
git clone https://github.com/tu-usuario/ghosttrack.git
cd ghosttrack
```

```
# 2. Crear entorno virtual (muy recomendado)
python3 -m venv venv
source venv/bin/activate
```

```
# 3. Instalar dependencias
```

```
pip install -r requirements.txt
```

```
# 4. Configurar variables de entorno (opcional)
cp .env.example .env
nano .env # añadir API keys si se dispone de ellas
```

```
# 5. Verificar la instalación
python main.py --list-modules
python main.py --version
```

Windows

```
# 1. Clonar el repositorio
git clone https://github.com/tu-usuario/ghosttrack.git
cd ghosttrack
```

```
# 2. Crear entorno virtual
python -m venv venv
venv\Scripts\activate
```

```
# 3. Instalar dependencias
pip install -r requirements.txt
```

```
# 4. Verificar la instalación
python main.py --list-modules
```

Instalación con dependencias Red Team opcionales

```
# Instalar dependencias Red Team
pip install impacket ldap3 pycryptodome
```

```
# Verificar que los módulos Red Team están disponibles
python main.py --list-modules
```

13.3 Configuración mediante config.json

El fichero config.json en la raíz del proyecto permite personalizar el comportamiento del framework sin modificar el código fuente. Los parámetros de configuración están organizados en seis secciones:

Sección application

Contiene metadatos del framework (nombre, versión, autor, descripción) que se muestran en el banner de inicio y se incluyen en los metadatos de los reportes generados.

Sección scanning

Define los parámetros de rendimiento de los módulos de escaneo: `default_threads` (número de threads concurrentes por defecto, 10), `default_timeout` (timeout de conexión en segundos, 5), `max_threads` (límite superior de threads, 100) y `max_timeout` (timeout máximo permitido, 30). El `user_agent` configura la cadena de identificación HTTP usada por los módulos que realizan peticiones web.

Sección ports

Define listas predefinidas de puertos para diferentes casos de uso: `common` (lista genérica de pentesting), `web` (puertos de servicios web), `database` (puertos de bases de datos) y `remote` (puertos de acceso remoto). Estas listas son usadas por el Port Scanner según el tipo de escaneo solicitado.

Sección wordlists

Define las rutas a las wordlists usadas por los módulos de brute force: `directories.txt` para el Directory Bruteforcer, `subdomains.txt` para el Subdomain Finder y `passwords.txt` para módulos que requieren pruebas de contraseñas.

Sección output

Configura el sistema de salida: `reports_dir` (directorio donde se guardan los reportes), `logs_dir` (directorio de logs), `format` (lista de formatos de reporte a generar: `html`, `json`, `txt`) y `save_raw_data` (si se guardan los datos crudos de cada módulo en el reporte).

Sección redteam

Controla el acceso a los módulos Red Team: `enabled` (si los módulos Red Team están disponibles) y `require_confirmation` (si se solicita una confirmación adicional antes de ejecutar los módulos avanzados, por encima de la confirmación general del disclaimer).

Sección api_keys

Reserva para las claves de API de servicios externos opcionales que amplían las capacidades del framework: Shodan (para búsqueda de hosts y servicios en Internet), VirusTotal (para análisis de malware y reputación de IPs), Censys (para búsqueda de certificados y servicios) y SecurityTrails (para datos históricos de DNS). Estas claves deben configurarse en el fichero `.env` para evitar que queden expuestas en el código fuente.

13.4 Variables de Entorno (.env)

El fichero `.env` permite configurar credenciales y API keys sin incluirlas en el código fuente o en el fichero `config.json`, que podría ser accidentalmente publicado en el repositorio.

El fichero `.env.example` en el repositorio muestra todas las variables disponibles con valores vacíos como plantilla:

```
# API Keys (opcionales)
SHODAN_API_KEY=
VIRUSTOTAL_API_KEY=
CENSYS_API_KEY=
CENSYS_API_SECRET=
SECURITYTRAILS_API_KEY=
```

```
# Configuración de red
PROXY_HOST=
PROXY_PORT=
```

```
# Configuración de logging
LOG_LEVEL=INFO
```

La librería `python-dotenv` carga automáticamente las variables del fichero `.env` al inicio de la aplicación mediante `load_dotenv()`, haciendo que estén disponibles como variables de entorno estándar accesibles con `os.getenv()`.

14. Casos de Uso y Ejemplos Prácticos

Este capítulo presenta cinco escenarios de uso representativos de GhostTrack, cubriendo desde una auditoría web básica hasta un ejercicio avanzado de Red Team en un entorno de Active Directory. Todos los ejemplos han sido ejecutados en entornos de laboratorio controlados con autorización expresa.

14.1 Escenario 1: Auditoría Completa de Aplicación Web

Contexto: Una empresa de comercio electrónico ha contratado una auditoría de seguridad de su plataforma web principal (tienda.ejemplo.com). El alcance incluye el dominio principal, sus subdominios y los servicios de red expuestos. El equipo dispone de una carta de autorización firmada por el CTO de la empresa.

Fase de reconocimiento pasivo

```
# Geolocalización e información de red
python main.py -t tienda.ejemplo.com -m geoip
```

```
# Información de registro del dominio
python main.py -t tienda.ejemplo.com -m whois
```

```
# Enumeración completa de DNS
python main.py -t ejemplo.com -m dns_enum
```

```
# Descubrimiento de subdominios (puede tardar varios minutos)
python main.py -t ejemplo.com -m subdomain_finder
```

Los resultados del Subdomain Finder revelan tres subdominios adicionales no listados en el sitio principal: dev.ejemplo.com, staging.ejemplo.com y admin.ejemplo.com. Estos serán incorporados al alcance del análisis.

Fase de escaneo

```
# Escaneo de puertos del servidor principal
python main.py -t tienda.ejemplo.com -m port_scanner
```

```
# Detección de servicios y versiones
python main.py -t tienda.ejemplo.com -m service_detection
```

```
# Análisis SSL/TLS del servidor HTTPS
python main.py -t tienda.ejemplo.com -m ssl_analyzer
```

El SSL Analyzer detecta que el servidor soporta TLSv1.0 (potencialmente vulnerable a BEAST) y que el certificado expira en 12 días, lo que es un hallazgo de severidad HIGH que requiere acción inmediata.

Fase de análisis de vulnerabilidades web

```
# Mapeo completo de la aplicación
python main.py -t tienda.ejemplo.com -m web_crawler
```

```
# Descubrimiento de rutas ocultas
python main.py -t tienda.ejemplo.com -m directory_bruteforce
```

```
# Detección de SQL Injection
python main.py -t tienda.ejemplo.com -m sql_injection
```

```
# Detección de XSS
python main.py -t tienda.ejemplo.com -m xss_scanner
```

Escaneo completo con un único comando

```
# Equivalente a todo lo anterior en un solo comando
python main.py -t tienda.ejemplo.com --full --threads 20 --timeout 10
```

El escaneo completo genera automáticamente reportes en tres formatos en el directorio reports/. El reporte HTML identifica: 1 hallazgo CRITICAL (SQL Injection en el parámetro 'id' del endpoint /product), 2 hallazgos HIGH (TLSv1.0 soportado, certificado próximo a expirar), 3 hallazgos MEDIUM (rutas administrativas expuestas sin protección de IP) y 5 hallazgos LOW (headers de seguridad HTTP ausentes: Content-Security-Policy, X-Frame-Options, X-Content-Type-Options, HSTS, Referrer-Policy).

14.2 Escenario 2: Reconocimiento OSINT de Objetivo Corporativo

Contexto: Un equipo de Red Team va a realizar un ejercicio de evaluación de seguridad contra una organización corporativa. La fase inicial de reconocimiento OSINT se realiza sin ningún contacto directo con los sistemas del cliente para evitar activar alertas prematuras.

```
# 1. Geolocalización e ISP del dominio corporativo
python main.py -t corp-objetivo.es -m geoip
```

```
# 2. Información WHOIS (registrador, fechas, contactos)
python main.py -t corp-objetivo.es -m whois
```

```
# 3. DNS completo (servidores de correo, nameservers, SPF/DKIM)
python main.py -t corp-objetivo.es -m dns_enum
```

```
# 4. Subdominios (superficie de ataque expandida)
python main.py -t corp-objetivo.es -m subdomain_finder
```

```
# 5. Emails públicos (potenciales objetivos de phishing)
python main.py -t corp-objetivo.es -m email_harvester
```

```
# 6. Metadatos de documentos públicos del cliente
python main.py -t corp-objetivo.es --files
informe_anual_2025.pdf manual_usuario.pdf -m metadata_extractor
```

Los metadatos del PDF revelan que los documentos fueron creados con Microsoft Word 2019 en un equipo con nombre de usuario 'jgarcia', lo que podría ser un nombre de usuario válido en el dominio Active Directory del cliente. El registro SPF del DNS revela que el cliente usa Microsoft 365 para el correo electrónico, y los registros MX apuntan a servidores de Outlook. Los subdominios descubiertos incluyen vpn.corp-objetivo.es (posiblemente una VPN SSL), citrix.corp-objetivo.es (posiblemente Citrix Gateway) y mail.corp-objetivo.es.

14.3 Escenario 3: Evaluación de Seguridad de Red Local

Contexto: Un auditor conectado a la red interna de un cliente durante un ejercicio de red team necesita mapear todos los hosts activos y sus servicios en el segmento de red 192.168.10.0/24.

```
# Descubrimiento de hosts activos via ARP
python main.py -t 192.168.10.1 --full --local-network

# Escaneo detallado de sistemas críticos identificados
python main.py -t 192.168.10.10 --full # Presunto DC
python main.py -t 192.168.10.20 --full # Servidor web interno
python main.py -t 192.168.10.30 --full # Servidor de base de datos

# Captura de tráfico para identificar protocolos inseguros
python main.py -t 192.168.10.1 -m network_sniffer
```

El Network Sniffer captura tráfico LLMNR (Link-Local Multicast Name Resolution) y NBT-NS (NetBIOS Name Service), protocolos que son explotables con herramientas de poisoning como Responder para capturar hashes NTLM de usuarios de la red. Esta información se incluye como hallazgo HIGH en el reporte.

14.4 Escenario 4: Red Team Assessment de Active Directory

Contexto: El equipo Red Team ha conseguido acceso inicial a la red interna del cliente mediante un ataque de phishing. El sistema comprometido es un equipo de usuario normal. El objetivo es comprometer el dominio Active Directory.

```
# Evaluación Red Team completa del DC
python main.py -t 192.168.10.10 --redteam --domain corp.local
```

El módulo LDAPEnumerator identifica 15 cuentas con Kerberos preauth deshabilitado (susceptibles de AS-REP Roasting) y 8 cuentas de servicio con SPNs registrados (susceptibles de Kerberoasting). El KerberosAttacker solicita los tickets para estas cuentas y los exporta para ataque offline. El BloodHoundCollector recopila datos del dominio y genera los ficheros JSON para importar en BloodHound. El análisis del grafo de BloodHound revela que una de las cuentas de servicio comprometida tiene el privilegio GenericWrite sobre otra cuenta que pertenece al grupo Domain Admins, lo que proporciona una ruta directa de escalada a DA.

14.5 Escenario 5: Uso de Script Personalizado

Contexto: El auditor necesita comprobar si un servidor Spring Boot tiene endpoints de Actuator expuestos públicamente, una misconfiguration común que puede revelar información sensible sobre la aplicación y su configuración.

```
# Crear el script personalizado
cat > scripts/spring_actuator.py << 'EOF'
import requests

ACTUATOR_ENDPOINTS = [
    '/actuator', '/actuator/health', '/actuator/env',
    '/actuator/metrics', '/actuator/mappings',
    '/actuator/beans', '/actuator/configprops',
    '/actuator/info', '/actuator/loggers',
    '/actuator/heapdump', '/actuator/threaddump'
]

def run(target, *args, **kwargs):
    results = {'target': target, 'findings': [], 'sensitive': []}
    for ep in ACTUATOR_ENDPOINTS:
        for proto in ['http', 'https']:
            url = f'{proto}://{target}{ep}'
            try:
                r = requests.get(url, timeout=5, verify=False)
                if r.status_code == 200:
                    finding = {'endpoint': url, 'status': 200,
                              'size': len(r.content)}
                    results['findings'].append(finding)
                    if ep in ['/actuator/env', '/actuator/heapdump']:
                        results['sensitive'].append(url)
            except Exception:
                pass
    return results
EOF
```

```
# Ejecutar el script personalizado  
python main.py -t app.ejemplo.com --inject scripts/spring_actuator.py
```

El script encuentra que `/actuator/env` está accesible públicamente sin autenticación, revelando variables de entorno que incluyen credenciales de base de datos, claves secretas de la aplicación y URLs de servicios internos. Este hallazgo se clasifica como CRITICAL y se incluye en el reporte automáticamente generado por el framework.

15. Seguridad del Propio Framework

Un framework de pentesting tiene acceso privilegiado a información muy sensible: direcciones IP de sistemas críticos, credenciales capturadas, vulnerabilidades descubiertas antes de ser remediadas y detalles de la arquitectura interna de las organizaciones auditadas. Es por tanto esencial que el propio framework esté diseñado con las mejores prácticas de seguridad para proteger esta información.

15.1 Gestión de Credenciales y Secretos

Las API keys y credenciales nunca se incluyen en el código fuente ni en el fichero de configuración `config.json`. Ambos ficheros son parte del repositorio de código y podrían ser accidentalmente publicados en plataformas como GitHub, lo que expondría las credenciales. En su lugar, las credenciales se almacenan en el fichero `.env`, que está incluido en `.gitignore` y nunca debe ser añadido al repositorio.

El fichero `.env.example` en el repositorio sirve como plantilla que muestra qué variables deben configurarse, pero con valores vacíos o de ejemplo que no son credenciales reales. Este patrón (código fuente + `.env.example` en el repo, `.env` real en la máquina local) es la práctica estándar recomendada por proyectos como Django, Laravel y muchos otros frameworks web.

La librería `python-dotenv` carga las variables del `.env` al inicio de la aplicación, haciéndolas disponibles como variables de entorno del sistema operativo mediante `os.getenv()`. Esto significa que las credenciales nunca están en variables de Python visibles en logs de depuración o en tracebacks de errores.

15.2 Validación de Entradas

Todas las entradas del usuario pasan por el módulo `Validator` antes de ser utilizadas por cualquier módulo del framework. Esta validación centralizada protege contra varios tipos de problemas. El método `sanitize_filename()` elimina los caracteres que tienen significado especial en sistemas de ficheros (`< > : " / \ | ? *`) antes de usar cualquier input del usuario como nombre de fichero o parte de una ruta, protegiendo contra ataques de path traversal. La validación del target antes de cualquier escaneo evita que inputs malformados sean pasados directamente a comandos del sistema operativo (como `nslookup` o `traceroute`), lo que podría constituir una vulnerabilidad de inyección de comandos.

15.3 Carga Segura de Scripts Personalizados

El sistema de inyección de scripts personalizados es intrínsecamente poderoso: permite ejecutar código Python arbitrario en el contexto del framework. Esto significa que un script malicioso podría hacer cualquier cosa que el usuario del framework pueda hacer: leer archivos del sistema, hacer conexiones de red, etc. GhostTrack advierte explícitamente al usuario sobre este riesgo en la documentación y en los mensajes de uso, recomendando cargar solo scripts de fuentes de confianza que hayan sido revisados previamente.

Una mitigación adicional que podría implementarse en versiones futuras es la ejecución de los scripts personalizados en un entorno sandbox mediante el módulo subprocess con restricciones de recursos, o la verificación de una firma digital del script antes de cargarlo.

15.4 Logging de Auditoría

Todas las operaciones del framework quedan registradas en los archivos de log con timestamp de alta resolución. Esta trazabilidad tiene varias funciones. Desde el punto de vista legal, los logs proporcionan evidencia de que el framework fue usado dentro del alcance autorizado: registran el target consultado, el usuario del sistema operativo que ejecutó el framework, los parámetros de la sesión y la confirmación explícita de autorización. Desde el punto de vista técnico, los logs permiten reconstruir exactamente qué módulos se ejecutaron y en qué orden, lo que facilita la reproducción de los hallazgos y la elaboración del informe.

Los archivos de log deben ser tratados con el mismo nivel de confidencialidad que los reportes: contienen información sensible sobre los sistemas auditados y no deben ser compartidos con terceros no autorizados.

15.5 Protección de los Reportes Generados

Los reportes generados por GhostTrack contienen información altamente sensible: listas de vulnerabilidades, nombres de usuario, hashes de contraseñas capturadas, detalles de configuración interna y evidencias de la posibilidad de compromiso de sistemas. Estos reportes deben ser tratados con las mismas medidas de seguridad que cualquier documento confidencial de la organización:

- Almacenamiento cifrado en el sistema del auditor (uso de herramientas como VeraCrypt o cifrado de volumen completo con BitLocker o LUKS).

- Transmisión cifrada al cliente mediante canales seguros (correo electrónico cifrado con PGP, plataformas de transferencia de ficheros cifradas, VPN).
- Eliminación segura una vez que el cliente ha confirmado la recepción del informe, usando herramientas de borrado seguro que sobrescriban el espacio en disco.
- Inclusión de marca de agua digital o información de identificación para detectar filtraciones no autorizadas.

15.6 Actualización y Mantenimiento

Como cualquier software de seguridad, GhostTrack debe mantenerse actualizado para incorporar las correcciones de vulnerabilidades de sus dependencias. El fichero requirements.txt fija las versiones exactas de todas las dependencias, lo que garantiza la reproducibilidad del entorno pero implica que las actualizaciones de seguridad de las librerías deben aplicarse manualmente. Se recomienda revisar periódicamente las actualizaciones de seguridad de las dependencias mediante herramientas como pip-audit o safety:

```
# Revisar vulnerabilidades en las dependencias
pip install pip-audit
pip-audit
```

16. Pruebas y Validación

El plan de pruebas de GhostTrack cubre tres niveles de testing que siguen la pirámide de pruebas de software: tests unitarios para los componentes individuales, tests de integración para la interacción entre componentes, y tests de sistema para la validación de los módulos de pentesting en entornos de laboratorio controlados.

16.1 Tests Unitarios

Los tests unitarios verifican el comportamiento correcto de cada componente de forma aislada, sin dependencias externas. El framework de testing elegido es pytest, que proporciona un sistema de fixtures potente para la configuración del entorno de prueba, soporte para parametrización de tests que permite ejecutar el mismo test con múltiples inputs, y un sistema de informes claro que facilita la identificación de fallos.

Tests del módulo Validator

Los tests del Validator cubren las siguientes casuísticas para cada método de validación:

- Casos positivos: Inputs válidos que deben ser aceptados (IPs válidas IPv4 e IPv6, dominios bien formados, puertos en rango, URLs con protocolo).
- Casos negativos: Inputs inválidos que deben ser rechazados (IPs con octetos fuera de rango, dominios con caracteres inválidos, puertos negativos o superiores a 65535, URLs sin protocolo).
- Casos de borde: Inputs en los límites del espacio de validación (puerto 1, puerto 65535, dominio con TLD de dos caracteres, IP 0.0.0.0, IP 255.255.255.255).

Ejemplo de test parametrizado para `validate_ip()`:

```
import pytest
from utils.validator import Validator

@pytest.mark.parametrize('ip,expected', [
    ('192.168.1.1', True),
    ('0.0.0.0', True),
    ('255.255.255.255', True),
    ('2001:db8::1', True),      # IPv6
    ('256.1.1.1', False),
    ('192.168.1', False),
    ('not_an_ip', False),
    ('', False),
```

```
1)
def test_validate_ip(ip, expected):
    assert Validator.validate_ip(ip) == expected
```

Tests del módulo Colors

Los tests de Colors verifican que los métodos de colorización producen strings que comienzan con el código ANSI correcto y terminan con el código de reset, y que el contenido textual está correctamente incluido. También se verifica que la clase funciona en Windows a través de la integración con colorama.

Tests del módulo Logger

Los tests del Logger usan ficheros temporales (tmp_path fixture de pytest) para verificar que los mensajes de log se escriben correctamente al fichero con el formato esperado, que el handler de consola solo muestra mensajes de nivel INFO y superiores, y que los ficheros de log se crean en el directorio configurado.

Tests del módulo Reporter

Los tests del Reporter usan directorios temporales para verificar que se generan los tres ficheros de reporte esperados con las extensiones correctas, que el JSON generado es parseable y contiene los campos esperados, que el HTML generado contiene las secciones esperadas y que el TXT contiene la información correcta en el formato esperado.

16.2 Tests de Integración

Los tests de integración verifican la correcta interacción entre componentes del sistema. El test más importante de este nivel es el test del flujo completo de orquestación: dado un target mock y módulos mockeados que devuelven resultados predefinidos, se verifica que el orquestador llama a los módulos en el orden correcto, consolida los resultados en la estructura correcta y los pasa al Reporter.

El uso de unittest.mock para simular los módulos de pentesting permite ejecutar estos tests sin conectividad de red y sin interacción con sistemas externos, lo que los hace adecuados para ejecución en pipelines de CI/CD.

16.3 Tests de Sistema en Entorno de Laboratorio

Los tests de sistema validan la funcionalidad completa de cada módulo de pentesting en entornos de laboratorio controlados y aislados. Para estos tests se ha utilizado el siguiente entorno de laboratorio virtualizado:

Entorno de laboratorio

- Sistema auditor: Kali Linux 2024 con GhostTrack instalado en un entorno virtual Python.
- Objetivo general: Metasploitable 3 (máquina intencionalmente vulnerable para pruebas de pentesting, creada por Rapid7).
- Objetivo web: DVWA (Damn Vulnerable Web Application) en Apache 2.4 con PHP 7.4 para pruebas de SQLi, XSS y directory brute force.
- Entorno Active Directory: Windows Server 2019 con Active Directory Domain Services configurado como DC del dominio lab.local, con varios usuarios de prueba incluyendo cuentas con misconfiguraciones intencionadas (Kerberos preauth deshabilitado, SPNs registrados en cuentas de usuario).
- Objetivo de red local: Varios VMs adicionales en el mismo segmento de red para pruebas de ARP Scanner y Network Sniffer.
- Aislamiento de red: Todas las VMs están en una red host-only de VirtualBox, completamente aislada de Internet y de redes de producción.

Resultados de validación por módulo

Tabla 4Resultados de validación de módulos básicos en entorno de laboratorioNota. Elaboración propia. Los tests se realizaron en el entorno de laboratorio descrito. ✓ = funcional, ~ = funcional con limitaciones, ✗ = requiere dependencias adicionales.

Módulo	Sistema de prueba	Resultado	Notas
GeoIP	8.8.8.8 (Google DNS)	✓ Funcional	Devuelve localización correcta de Mountain View, CA
WHOIS	google.com	✓ Funcional	Devuelve datos correctos del registrar MarkMonitor
DNS Enum	lab.local	✓ Funcional	Enumera registros A, NS, SOA; AXFR denegado correctamente
Subdomain Finder	lab.local	✓ Funcional	Descubre DC01, FILE01, WEB01 por brute force DNS
Port Scanner	Metasploitable3	✓ Funcional	Identifica correctamente 15+ puertos abiertos
Service Detection	Metasploitable3	✓ Funcional	Identifica Apache 2.4, OpenSSH 7.4, MySQL 5.7
OS Fingerprinting	Metasploitable3	~ Parcial	TTL=64 → Linux; banner confirma Ubuntu
Banner Grabbing	Metasploitable3	✓ Funcional	Captura banners FTP/SSH/HTTP correctamente

Módulo	Sistema de prueba	Resultado	Notas
SSL Analyzer	DVWA con HTTPS	✓ Funcional	Detecta TLSv1.0, certif. autofirmado expirado
Vuln Scanner	Metasploitable3	✓ Funcional	Detecta Apache 2.4.49 y OpenSSH 7.4 CVEs
Traceroute	8.8.8.8	✓ Funcional	Traza correctamente los saltos hasta Google DNS
Web Crawler	DVWA	✓ Funcional	Mapea 47 URLs y 8 formularios de la aplicación
Dir Bruteforce	DVWA	✓ Funcional	Descubre /phpmyadmin (403), /.htaccess (403)
SQL Injection	DVWA (low sec)	✓ Funcional	Detecta SQLi error-based en parámetro 'id'
XSS Scanner	DVWA (low sec)	✓ Funcional	Detecta reflected XSS en parámetro 'name'
ARP Scanner	Red local	✓ Funcional	Descubre todos los hosts del segmento de red
Network Sniffer	Red local	✓ Funcional (root)	Captura paquetes con Scapy; requiere root
Email Harvester	DVWA	✓ Funcional	Extrae emails del código HTML
Metadata Extractor	PDFs de prueba	✓ Funcional	Extrae autor, fechas, software de creación
Custom Scripts	Script plantilla	✓ Funcional	Carga y ejecuta template.py correctamente

16.4 Ejecución de la Suite de Tests

```
# Ejecutar todos los tests
pytest tests/ -v
```

```
# Ejecutar con reporte de cobertura
pytest tests/ --cov=. --cov-report=html --cov-report=term-missing
```

```
# Solo tests unitarios (sin red)
pytest tests/unit/ -v
```

```
# Solo tests de integración
pytest tests/integration/ -v
```

```
# Ejecutar un test específico
pytest tests/unit/test_validator.py::test_validate_ip -v
```

16.5 Métricas de Calidad

Las métricas de calidad del código de GhostTrack se evalúan mediante las siguientes herramientas:

- Cobertura de tests: Medida con `pytest-cov`. El objetivo es superar el 70% de cobertura en los módulos de utilidades y el 40% en los módulos de pentesting (los módulos de red son más difíciles de testear unitariamente por sus dependencias de red).
- Análisis estático: Realizado con `pylint` para detectar errores de código, convenciones de estilo y posibles bugs. El objetivo es una puntuación de `pylint` superior a 8/10.
- Complejidad ciclomática: Medida con `radon`. Los módulos con complejidad superior a 10 se identifican como candidatos a refactorización.
- Tipos: Se usa `mypy` para la verificación estática de tipos en los módulos de utilidades, donde el tipado estricto es más valioso.

17. Conclusiones Técnicas

El desarrollo de GhostTrack ha supuesto la integración de conocimientos de múltiples disciplinas de la ingeniería informática en un único proyecto cohesionado y funcional. Este capítulo reflexiona sobre los principales aprendizajes técnicos, los objetivos alcanzados y las líneas de trabajo futuro.

17.1 Objetivos Alcanzados

Todos los objetivos definidos al inicio del proyecto han sido alcanzados. Los cuarenta módulos están implementados y son funcionales en entornos de laboratorio. La arquitectura modular se ha demostrado robusta y extensible. El sistema de reportes genera documentos profesionales en tres formatos. La CLI expone toda la funcionalidad de forma intuitiva. Los mecanismos de uso ético están integrados en el núcleo del framework.

El objetivo más significativo desde el punto de vista de aprendizaje ha sido la implementación de los módulos Red Team, que ha requerido profundizar en el funcionamiento interno de protocolos y mecanismos de seguridad de Windows y Active Directory que raramente se tratan con este nivel de detalle en el currículo académico estándar. La implementación de Kerberoasting, por ejemplo, ha requerido entender en profundidad cómo funciona el protocolo Kerberos, qué son los SPN, cómo se estructura un ticket de servicio y por qué es posible atacarlo offline.

17.2 Decisiones de Diseño y su Justificación

La decisión de usar una arquitectura modular en lugar de un monolito ha resultado claramente acertada. La capacidad de desarrollar, probar y depurar cada módulo de forma independiente ha acelerado enormemente el proceso de desarrollo. En varias ocasiones, un módulo fallaba en el entorno de laboratorio y fue posible depurarlo ejecutándolo de forma autónoma con `sys.argv` simulado, sin necesidad de ejecutar el framework completo.

La decisión de implementar el protocolo WHOIS directamente mediante sockets TCP, en lugar de usar una librería de terceros, ha sido igualmente acertada. Ha permitido entender en detalle el funcionamiento del protocolo y ha resultado en un módulo sin dependencias adicionales que funciona de forma consistente en todos los sistemas operativos soportados.

La decisión de generar tres formatos de reporte simultáneamente responde a una comprensión práctica de las necesidades reales de los proyectos de auditoría de seguridad. En la práctica real, el cliente técnico necesita el JSON para sus procesos de remediación

automatizados, el CISO necesita el HTML para presentar a dirección, y el auditor necesita el TXT para sus notas de trabajo. Generar los tres automáticamente elimina el trabajo manual de conversión y garantiza que todos los formatos están siempre disponibles.

17.3 Limitaciones y Trabajo Futuro

Las limitaciones más significativas del proyecto actual son la base de datos de CVEs estática, la funcionalidad parcial de algunos módulos Red Team por dependencias opcionales no instaladas por defecto, y la ausencia de interfaz gráfica de usuario. Las siguientes líneas de trabajo futuro abordarían estas limitaciones:

- Integración con la API de la NVD (National Vulnerability Database) del NIST para mantener la base de datos de CVEs actualizada automáticamente mediante descargas periódicas en formato JSON.
- Desarrollo de una interfaz web con Flask o FastAPI que proporcione un panel de control para la gestión de escaneos, visualización de resultados en tiempo real y gestión del historial de auditorías.
- Integración con plataformas de gestión de vulnerabilidades como DefectDojo para la gestión centralizada de los hallazgos de múltiples auditorías.
- Implementación de modo asíncrono mediante `asyncio` para los módulos de mayor volumen (Port Scanner, Subdomain Finder), lo que podría multiplicar por cinco o diez el rendimiento frente a la implementación actual basada en `threads`.
- Desarrollo de una suite de tests de integración completa con entornos de laboratorio Docker reproducibles, que permita ejecutar los tests en cualquier máquina sin necesidad de configurar manualmente el entorno de VMs.
- Soporte para IPv6 en todos los módulos de escaneo de red, que actualmente están optimizados para IPv4.

17.4 Reflexión sobre el Proceso de Aprendizaje

El desarrollo de GhostTrack ha sido un ejercicio de aprendizaje en sí mismo, más allá del producto final. La implementación de cada módulo ha requerido estudiar en detalle los protocolos, RFCs, técnicas de ataque y herramientas existentes que se querían replicar. Este proceso de estudio e implementación propia es incomparablemente más efectivo para el aprendizaje profundo que el simple uso de herramientas existentes.

Especialmente formativo ha sido el proceso de pruebas en el entorno de laboratorio, donde cada discrepancia entre el comportamiento esperado y el real ha requerido un análisis

en profundidad que ha llevado a una comprensión más sólida del funcionamiento de los protocolos. Los errores de implementación han sido, paradójicamente, las experiencias de aprendizaje más valiosas del proyecto.

El proyecto también ha puesto de manifiesto la importancia de los aspectos no técnicos de la ciberseguridad ofensiva: el marco legal, la ética profesional, la comunicación con el cliente y la redacción de informes. Un profesional de ciberseguridad no es solo un técnico; es también un comunicador, un asesor y un agente con responsabilidades legales y éticas que no pueden separarse de su trabajo técnico.

18. Referencias

Referencias bibliográficas en formato APA 7ª edición

- Anderson, J. P. (1972). Computer security technology planning study. ESD-TR-73-51. Electronic Systems Division, Air Force Systems Command.
- Byt3bl33d3r. (2016). NTLM relay attacks: New vectors and mitigations. DEF CON 24. Las Vegas, NV.
- Código Penal. (1995). Ley Orgánica 10/1995, de 23 de noviembre, del Código Penal. Boletín Oficial del Estado, núm. 281. <https://www.boe.es/eli/es/lo/1995/11/23/10/con>
- Daigle, L. (2004). WHOIS Protocol Specification (RFC 3912). Internet Engineering Task Force. <https://tools.ietf.org/html/rfc3912>
- Engelbreton, P. (2013). The basics of hacking and penetration testing: Ethical hacking and penetration testing made easy (2nd ed.). Syngress.
- Farmer, D., y Venema, W. (1995). Improving the security of your site by breaking into it. Artículo técnico. <http://www.fish.com/security/admin-guide-to-cracking.html>
- FIRST (Forum of Incident Response and Security Teams). (2019). Common Vulnerability Scoring System version 3.1: Specification document. <https://www.first.org/cvss/specification-document>
- Gamma, E., Helm, R., Johnson, R., y Vlissides, J. (1994). Design patterns: Elements of reusable object-oriented software. Addison-Wesley.
- Harmj0y. (2016). Roasting AS-REPs. SpecterOps Blog. <https://harmj0y.medium.com/roasting-as-reps-e6179a65216b>
- Hutchins, E. M., Cloppert, M. J., y Amin, R. M. (2011). Intelligence-driven computer network defense informed by analysis of adversary campaigns and intrusion kill chains. Proceedings of the 6th International Conference on Information Warfare and Security, 80.
- Kerberos Authentication Protocol. (2005). The Kerberos Network Authentication Service (v5) (RFC 4120). Internet Engineering Task Force. <https://tools.ietf.org/html/rfc4120>
- Liskov, B. (1987). Data abstraction and hierarchy. ACM SIGPLAN Notices, 23(5), 17–34. <https://doi.org/10.1145/62138.62141>
- Lutz, M. (2013). Learning Python (5th ed.). O'Reilly Media.

- Lyon, G. F. (1997). Nmap: The network mapper [Software]. <https://nmap.org/>
- Martin, R. C. (2003). Agile software development: Principles, patterns, and practices. Prentice Hall.
- Maynor, D., y Snijder, R. (2006). Metasploit toolkit for penetration testing, exploit development, and vulnerability research. Syngress.
- Medin, T. (2014, agosto). Kerberoasting. DEF CON 22, Las Vegas, NV.
- Meyer, B. (1988). Object-oriented software construction. Prentice Hall.
- Miró Llinares, F. (2012). El cibercrimen: Fenomenología y criminología de la delincuencia en el ciberespacio. Marcial Pons.
- MITRE Corporation. (2023). MITRE ATT&CK® Enterprise Matrix (versión 14.0). <https://attack.mitre.org/>
- MITRE Corporation. (2024). Common Vulnerabilities and Exposures (CVE). <https://cve.mitre.org/>
- Mitnick, K. D., y Simon, W. L. (2002). The art of deception: Controlling the human element of security. Wiley.
- Morgan, S. (2020). Cybercrime to cost the world \$10.5 trillion annually by 2025. Cybercrime Magazine. <https://cybersecurityventures.com/cybercrime-damage-costs-10-trillion-by-2025/>
- NIST (National Institute of Standards and Technology). (2024). NIST Cybersecurity Framework 2.0. U.S. Department of Commerce. <https://doi.org/10.6028/NIST.CSWP.29>
- OWASP Foundation. (2020). OWASP testing guide (version 4.2). <https://owasp.org/www-project-web-security-testing-guide/>
- OWASP Foundation. (2021). OWASP Top 10:2021. <https://owasp.org/Top10/>
- Parlamento Europeo y Consejo de la Unión Europea. (2013). Directiva 2013/40/UE del Parlamento Europeo y del Consejo, de 12 de agosto de 2013, relativa a los ataques contra los sistemas de información. Diario Oficial de la Unión Europea, L 218. <https://eur-lex.europa.eu/eli/dir/2013/40/oj>
- Parlamento Europeo y Consejo de la Unión Europea. (2016). Reglamento (UE) 2016/679 del Parlamento Europeo y del Consejo, de 27 de abril de 2016 (Reglamento general de

protección de datos). Diario Oficial de la Unión Europea, L 119.
<https://eur-lex.europa.eu/eli/reg/2016/679/oj>

PTES Technical Guidelines. (2012). Penetration Testing Execution Standard: Technical guidelines. http://www.pentest-standard.org/index.php/PTES_Technical_Guidelines

Rapid7. (2003). Metasploit Framework [Software]. <https://www.metasploit.com/>

Raymond, E. S. (2003). The art of Unix programming. Addison-Wesley.

Rohan, H., Robbins, A., y Schroeder, W. (2016, agosto). Six degrees of domain admin. DEF CON 24, Las Vegas, NV.

Seitz, J. (2021). Black Hat Python: Python programming for hackers and pentesters (2nd ed.). No Starch Press.

GhostTrack

Manual Técnico — Versión 1.0

Izán Jiménez Nuñez · TFG 2026

Documento confidencial — Clasificación: Uso interno restringido

Solo para uso en entornos autorizados. Prohibida su distribución sin consentimiento.

APÉNDICES

Material complementario de referencia

Apéndice A — Análisis de Rendimiento y Benchmarks

Los benchmarks presentados en este apéndice fueron obtenidos en el entorno de laboratorio descrito en el capítulo 16. El objetivo es proporcionar datos objetivos sobre el rendimiento de GhostTrack bajo distintas configuraciones y compararlo con herramientas especializadas del sector.

Todos los benchmarks se realizaron bajo condiciones controladas y reproducibles: máquina virtual con 4 vCPUs Intel Core i7-10700K, 8 GB RAM, Kali Linux 2024, red virtual host-only Gigabit hacia los objetivos. Cada benchmark es la mediana de cinco ejecuciones consecutivas, descartando el valor más alto y más bajo.

A.1 Benchmarks por Módulo

Tabla 1 *Benchmarks de rendimiento de módulos GhostTrack en entorno de laboratorio* Nota. Elaboración propia. n=5 para objetivos locales, n=10 para módulos con APIs externas.

Módulo	Parámetros de ejecución	T. mínimo	T. mediano	T. máximo
Port Scanner	36 puertos, 10 threads, timeout 1s	1.8 s	2.3 s	3.1 s
Port Scanner	36 puertos, 50 threads, timeout 1s	0.6 s	0.8 s	1.1 s
Port Scanner	65535 puertos, 100 threads	7.2 min	8.4 min	9.8 min
Subdomain Finder	60 palabras, 20 threads, DNS local	2.1 s	3.8 s	6.4 s
Subdomain Finder	60 palabras + CT Logs (API ext.)	4.3 s	7.2 s	12.8 s
Dir Bruteforce	50 dirs, 10 threads, timeout 5s	8.2 s	11.4 s	15.7 s
Dir Bruteforce	50 dirs, 50 threads, timeout 5s	2.1 s	3.2 s	4.8 s
Web Crawler	50 páginas, profundidad 3	12.4 s	22.7 s	38.1 s
SSL/TLS Analyzer	1 objetivo, test completo protocolos	2.9 s	4.4 s	7.2 s
GeoIP	1 IP, API ip-api.com	0.3 s	0.7 s	1.8 s
WHOIS	1 dominio, servidor verisign	0.8 s	1.9 s	4.2 s
DNS Enumeration	10 tipos, incluye intento AXFR	4.1 s	7.8 s	14.3 s
Banner Grabbing	10 puertos, 10 threads	3.2 s	5.1 s	8.7 s
Vuln Scanner	15 servicios, BD interna	0.02 s	0.03 s	0.05 s

Módulo	Parámetros de ejecución	T. mínimo	T. mediano	T. máximo
Escaneo --full	20 módulos, config. por defecto	3.8 min	5.2 min	7.1 min

A.2 Escalabilidad del Port Scanner con Threading

Tabla 2 *Speedup del Port Scanner según número de threads (36 puertos, timeout 1s)* Nota. Elaboración propia. Speedup relativo respecto a ejecución con 1 thread.

Nº threads	Tiempo	Speedup	Eficiencia
1	32.4 s	1.0×	100%
5	6.8 s	4.8×	96%
10	3.4 s	9.5×	95%
20	1.9 s	17.1×	85%
50	0.8 s	40.5×	81%
100	0.5 s	64.8×	65%
200	0.5 s	64.8×	32%

La eficiencia superior al 80% con 50 threads confirma una escalabilidad casi lineal hasta ese punto. A partir de 100 threads la eficiencia cae al 65% por la contención en el acceso a sockets y el overhead del GIL de Python. El límite de `max_threads = 100` en `config.json` refleja este punto de saturación.

A.3 Comparación con Herramientas Especializadas

Tabla 3 *Comparación de rendimiento GhostTrack vs herramientas especializadas* Nota. Elaboración propia. Benchmarks obtenidos en el mismo entorno de laboratorio.

Funcionalidad	GhostTrack	Herramienta ref.	Factor	Ventaja GhostTrack
Port scan 36 puertos	0.8 s	Nmap: 0.4 s	2× más lento	Sin raw sockets, sin root requerido
Port scan completo	8.4 min	Nmap: 2.1 min	4× más lento	TCP Connect portable a Windows/macOS
Dir bruteforce 50 dirs	3.2 s	Gobuster: 1.1 s	3× más lento	Reporte integrado automático
Subdomain 60 palabras	3.8 s	Amass: 8.2 s	2× más rápido	CT Logs + DNS en un solo módulo

Funcionalidad	GhostTrack	Herramienta ref.	Factor	Ventaja GhostTrack
SSL análisis básico	4.4 s	testssl.sh: 28 s	6× más rápido	Análisis rápido integrado en flujo
SQLi detección	6.2 s	sqlmap: 4.1 s	1.5× más lento	Sin dependencias externas, integrado

La principal ventaja de GhostTrack no es el rendimiento individual de cada módulo sino la integración: un único comando ejecuta veinte módulos en secuencia, consolida resultados y genera reportes automáticamente. Para wordlists grandes (>100k entradas) o escaneos de rangos completos se recomienda usar herramientas especializadas como Gobuster o Nmap respectivamente, e importar sus resultados al contexto de la auditoría.

Apéndice B — Guía de Desarrollo de Nuevos Módulos

Este apéndice proporciona una guía completa para extender GhostTrack con nuevos módulos, cubriendo desde la estructura de código hasta la integración en el orquestador y los tests requeridos.

B.1 Plantilla Estándar de Módulo

Todo nuevo módulo debe seguir esta estructura para garantizar la compatibilidad con el orquestador, el sistema de reportes y el Logger:

```
"""
MÓDULO XX: NOMBRE_DEL_MÓDULO
Descripción: qué hace el módulo.
Categoría: Reconocimiento | Enumeración | Vulnerabilidades | Web | Red
Requiere privilegios: Sí / No
"""

import socket
from typing import Dict, Any
from datetime import datetime

DEFAULT_TIMEOUT = 5

class NombreModulo:
    def __init__(self, target: str, timeout: int = DEFAULT_TIMEOUT, **kwargs):
        self.target = target
        self.timeout = timeout
        self.results: Dict[str, Any] = {
            'target': target,
            'timestamp': datetime.now().isoformat(),
            'status': 'pending',
            'data': {},
            'findings': [],
            'errors': []
        }

    def run(self) -> Dict[str, Any]:
        print(f'[*] {self.__class__.__name__} en: {self.target}')
        try:
            self._analyze()
            self.results['status'] = 'success'
        except Exception as e:
            self.results['status'] = 'error'
            self.results['errors'].append(str(e))
        print(f'[!] Error: {e}')
```

```

        return self.results

    def _analyze(self):
        # TODO: lógica del módulo
        pass

    def _add_finding(self, ftype: str, desc: str,
                    severity: str = 'MEDIUM', evidence: str = ''):
        self.results['findings'].append({
            'type': ftype, 'description': desc,
            'severity': severity, 'evidence': evidence
        })

    def display_results(self):
        for f in self.results.get('findings', []):
            print(f"    [{f['severity']}] {f['description']}")

def run_module(target: str, **kwargs) -> Dict[str, Any]:
    m = NombreModulo(target, **kwargs)
    r = m.run()
    m.display_results()
    return r

```

B.2 Integración en el Orquestador

Para que el nuevo módulo sea invocado por main.py son necesarios cuatro cambios:

9. Importación en main.py: añadir `import modules.nombre_modulo` as `nombre_modulo` en el bloque de importaciones básicas.
10. Llamada en `run_full_scan()`: añadir la línea `results['modules'][nombre_modulo] = nombre_modulo.run_module(target, timeout=args.timeout)` en el orden lógico correcto.
11. Lista en `show_modules()`: añadir una entrada en la lista `basic_modules` con el número y descripción del módulo.
12. `config.json`: añadir el nombre a la lista `modules.enabled`.

B.3 Tests Unitarios Requeridos

```

# tests/unit/test_nombre_modulo.py
import pytest

from unittest.mock import patch, MagicMock
from modules.nombre_modulo import NombreModulo, run_module

```

```
class TestNombreModulo:

    def test_init_defaults(self):
        m = NombreModulo('192.168.1.1')
        assert m.target == '192.168.1.1'
        assert m.timeout == 5
        assert m.results['status'] == 'pending'

    @patch('modules.nombre_modulo.socket.create_connection')
    def test_run_success(self, mock_conn):
        mock_conn.return_value.__enter__ = lambda s: MagicMock(recv=lambda n:
b'OK')
        mock_conn.return_value.__exit__ = MagicMock(return_value=False)
        results = run_module('192.168.1.1')
        assert results['status'] == 'success'

    @patch('modules.nombre_modulo.socket.create_connection',
            side_effect=ConnectionRefusedError)
    def test_run_connection_error(self, _):
        results = run_module('192.168.1.1')
        assert results['status'] == 'error'
        assert len(results['errors']) > 0
```

Apéndice C — Casos de Estudio: Vulnerabilidades Reales

Los cinco casos de estudio presentados a continuación están basados en incidentes de seguridad reales documentados públicamente. Para cada caso se describe el contexto, cómo GhostTrack habría detectado el vector de ataque, el impacto del incidente y las medidas de remediación aplicadas.

C.1 Exposición Masiva de Elasticsearch (2019)

Contexto

En octubre de 2019, el investigador Bob Diachenko localizó una instancia de Elasticsearch con más de 1.200 millones de registros (nombres, emails, teléfonos) accesible públicamente sin autenticación. La base de datos pertenecía a dos empresas de enriquecimiento de datos. El problema raíz fue que Elasticsearch no habilita autenticación por defecto en versiones anteriores a la 8.0, y los administradores desplegaron el servicio sin configurar ningún control de acceso.

Detección con GhostTrack

El Port Scanner habría detectado el puerto 9200 abierto. El Service Detection habría identificado Elasticsearch y extraído su versión del endpoint raíz. El Vulnerability Scanner habría marcado el servicio como CRITICAL: 'Elasticsearch sin autenticación — todos los índices accesibles sin credenciales'. La evidencia: respuesta JSON de `/_cluster/health` sin cabecera Authorization.

Remediación

Habilitar X-Pack Security (autenticación nativa de Elasticsearch), configurar TLS entre nodos, restringir el puerto 9200 a IPs internas mediante firewall, y auditar los índices expuestos para valorar el alcance de la brecha.

C.2 EternalBlue y WannaCry (2017)

Contexto

En mayo de 2017, el ransomware WannaCry infectó más de 230.000 sistemas en 150 países explotando la vulnerabilidad EternalBlue (CVE-2017-0144) en SMBv1 de Windows. La NSA había desarrollado el exploit, el grupo Shadow Brokers lo filtró en abril de 2017, y Microsoft había publicado el parche MS17-010 un mes antes. La velocidad de propagación fue posible porque miles de organizaciones no habían aplicado el parche.

Detección con GhostTrack

El Port Scanner habría detectado el puerto 445 abierto. El Service Detection habría identificado SMBv1. El Vulnerability Scanner habría marcado CVE-2017-0144 como HIGH (CVSS 8.1): 'SMBv1 habilitado — potencialmente vulnerable a EternalBlue. Verificar aplicación del parche MS17-010'. El módulo SMBEnumerator habría confirmado si la firma SMB estaba desactivada.

Lección aprendida

El intervalo entre publicación del parche y su aplicación masiva es la ventana de riesgo más crítica. Las auditorías periódicas con el Vulnerability Scanner de GhostTrack habrían generado una alerta de remediación semanas antes del ataque.

C.3 Directorio .git Expuesto en Servidor Web

Contexto

Una vulnerabilidad frecuente en despliegues manuales: el desarrollador copia el directorio de trabajo completo al servidor web, incluyendo el subdirectorio .git. El historial completo del repositorio queda accesible, incluyendo versiones anteriores con credenciales hardcodeadas eliminadas en commits recientes pero presentes en el historial.

Detección con GhostTrack

El módulo Directory Bruteforce tiene .git en su wordlist interna COMMON_DIRS. Una respuesta 403 en /.git confirma que el directorio existe aunque tenga listado deshabilitado. Una respuesta 200 en /.git/config (también en la wordlist) confirma la exposición total e incluye la URL del repositorio remoto. La herramienta complementaria git-dumper (Terjanq, 2018) permite reconstruir el repositorio completo a partir de los objetos expuestos.

Remediación

Añadir una regla en el servidor web que deniegue acceso a cualquier directorio que empiece por punto (Deny from all en Apache, location ~* /\. { deny all; } en nginx), y revisar si el repositorio contiene secretos que deban ser rotados.

C.4 Kerberoasting en Entorno Financiero

Contexto

En un Red Team real contra una empresa financiera (anonimizado), tras obtener acceso inicial mediante phishing, el equipo ejecutó Kerberoasting contra todos los SPNs del

dominio. La contraseña de la cuenta `svc_sql` (ocho caracteres, solo minúsculas y dígitos) fue crackeada por Hashcat en menos de diez minutos. Con esas credenciales, el equipo accedió a todos los servidores SQL Server del dominio como administrador de base de datos, extrayendo tablas con datos de 500.000 clientes.

Detección con GhostTrack

LDAPEnumerator habría identificado `svc_sql` con SPN registrado. KerberosAttacker habría solicitado el TGS para ese SPN y exportado el hash `$krb5tgs$23$*svc_sql*...` para cracking offline. El módulo PrivEscChecker habría mapeado los servidores accesibles con esas credenciales. La recomendación del reporte: migrar cuentas de servicio a gMSA con contraseñas rotadas automáticamente de 240 caracteres.

Remediación

Migración a gMSA (group Managed Service Accounts), implementación de Privileged Access Workstation (PAW) model, habilitación de Advanced Threat Analytics para detección de Kerberoasting, y auditoría completa de todas las cuentas de servicio restantes con SPNs.

C.5 Transferencia de Zona DNS Exitosa

Contexto

En la auditoría de una empresa de telecomunicaciones, el módulo DNS Enumeration encontró que el servidor `ns2.empresa.com` (un BIND 9 instalado en 2014) permitía transferencias AXFR sin restricción por ausencia de la directiva `allow-transfer`. La zona completa de 1.247 registros reveló 31 subdominios internos, las IPs de toda la infraestructura corporativa, y tres IPs de sistemas SCADA que no debían tener visibilidad externa.

Detección con GhostTrack

El módulo DNS Enumeration intenta AXFR contra cada NS descubierto. La salida incluye: `'[!] AXFR EXITOSA desde ns2.empresa.com — 1247 registros obtenidos'` con severidad CRITICAL. El reporte HTML muestra el listado completo de registros agrupados por tipo, con los subdominios internos y las IPs SCADA destacados en rojo.

Remediación

Añadir `allow-transfer { none; };` en BIND 9, revisar y limpiar registros DNS de sistemas legacy ya desactivados, y segmentar los sistemas SCADA en una red completamente aislada sin visibilidad DNS externa.

Apéndice D — Integración con Herramientas del Sector

GhostTrack está diseñado para integrarse con el ecosistema de herramientas de ciberseguridad existente. Este apéndice describe los flujos de integración más comunes con scripts de ejemplo listos para usar.

D.1 Conversión a Formato Nmap XML para Metasploit

Los resultados del Port Scanner y Service Detection de GhostTrack pueden importarse en Metasploit Framework mediante conversión al formato XML de Nmap:

```
#!/usr/bin/env python3
# scripts/ghosttrack_to_nmap.py
import json, sys
from xml.etree.ElementTree import Element, SubElement, indent, tostring

def convert(json_path, xml_path):
    with open(json_path) as f: data = json.load(f)
    root = Element('nmaprun')
    root.set('scanner', 'ghosttrack')
    host = SubElement(root, 'host')
    addr = SubElement(host, 'address')
    addr.set('addr', data['target']); addr.set('addrtype', 'ipv4')
    ports_el = SubElement(host, 'ports')
    for port, svc in data.get('modules', {}).get('services', {}).items():
        p = SubElement(ports_el, 'port')
        p.set('portid', str(port)); p.set('protocol', 'tcp')
        SubElement(p, 'state').set('state', 'open')
        s = SubElement(p, 'service')
        s.set('name', svc.get('service', 'unknown'))
        s.set('version', svc.get('version', ''))
    indent(root)
    with open(xml_path, 'wb') as f:
        f.write(tostring(root, encoding='utf-8', xml_declaration=True))
    print(f'[+] {xml_path} listo para: msfconsole> db_import {xml_path}')

if __name__ == '__main__':
    convert(sys.argv[1], sys.argv[2])
```

D.2 Importación de URLs en Burp Suite

```
#!/usr/bin/env python3
# scripts/ghosttrack_to_burp.py
import json, requests, sys
```

```

BURP_API = 'http://127.0.0.1:1337/v0.1'

def import_scope(json_path):
    with open(json_path) as f: data = json.load(f)
    target = data['target']
    urls = data.get('modules', {}).get('web_crawler', {}).get('urls', [])
    # Añadir al scope
    requests.put(f'{BURP_API}/target/scope', json={
        'included': [{'enabled': True, 'prefix': f'http://{target}'},
                     {'enabled': True, 'prefix': f'https://{target}'}])
    print(f'[+] Scope: http(s)://{target}')
    # Importar sitemap
    for url in urls[:200]:
        requests.post(f'{BURP_API}/target/sitemap', json={'urls': [url]})
    print(f'[+] {min(len(urls), 200)} URLs en sitemap de Burp')

if __name__ == '__main__':
    import_scope(sys.argv[1])

```

D.3 Pipeline CI/CD con GitLab

El siguiente stage de GitLab CI ejecuta GhostTrack en cada despliegue a staging y falla el pipeline si hay hallazgos CRITICAL:

```

# .gitlab-ci.yml
security_scan:
  stage: security
  image: python:3.10-slim
  only: [staging, main]
  script:
    - pip install -r requirements.txt -q
    - python main.py -t $STAGING_HOST --full -o reports/ --threads 20
    - python scripts/check_severity.py reports/ --fail-on CRITICAL
  artifacts:
    paths: [reports/]
    expire_in: 90 days
    allow_failure: false

```

El script `check_severity.py` que hace fallar el pipeline cuando hay hallazgos por encima del umbral:

```

#!/usr/bin/env python3
import json, sys, glob, argparse
p = argparse.ArgumentParser()
p.add_argument('reports_dir')
p.add_argument('--fail-on', default='CRITICAL')

```

```
args = p.parse_args()
ORDER = {'CRITICAL':4, 'HIGH':3, 'MEDIUM':2, 'LOW':1, 'INFO':0}
thresh = ORDER.get(args.fail_on, 4)
found = []
for jf in glob.glob(f'{args.reports_dir}/*.json'):
    data = json.load(open(jf))
    for m,res in data.get('modules',{}).items():
        vulns = res if isinstance(res,list) else res.get('vulnerabilities',[])
        for v in vulns:
            if ORDER.get(v.get('severity','INFO'),0) >= thresh:
                found.append(f"{v['severity']}:
{v.get('description','')[:80]}")
if found:
    [print(f' - {f}') for f in found]
    sys.exit(1)
print('[+] Sin hallazgos >= ' + args.fail_on)
```

Apéndice E — Glosario de Términos Técnicos

Este glosario recoge los principales términos técnicos del manual organizados por categorías.

E.1 Protocolos y Conceptos de Red

ARP (Address Resolution Protocol)

Protocolo de capa de enlace (RFC 826) que resuelve IPs a MACs en redes locales. La caché ARP del sistema es la fuente del módulo ARP Scanner.

AXFR

Consulta DNS de transferencia de zona completa. Cuando no está restringida, devuelve todos los registros del dominio en una sola consulta.

CIDR

Notación para rangos IP: 192.168.1.0/24 = 256 IPs de .0 a .255. Los 24 primeros bits son el prefijo de red.

DMARC

Política de autenticación de email. p=none solo monitoriza; p=quarantine envía a spam; p=reject descarta. Su ausencia facilita la suplantación de dominio en phishing.

DPAPI

API Windows para cifrado/descifrado usando credenciales del usuario como clave. Usada por navegadores y WiFi para almacenar contraseñas. El módulo CredentialDumper la explota.

TGT / TGS

Tickets Kerberos. El TGT autentifica al usuario ante el KDC. El TGS (cifrado con el hash de contraseña de la cuenta de servicio) es el objetivo de Kerberoasting.

TTL

Time To Live IP: saltos máximos antes del descarte. Linux=64, Windows=128, Cisco=255. El módulo OS Fingerprinting infiere el SO por el TTL observado.

E.2 Vulnerabilidades y Técnicas

BEAST (CVE-2011-3389)

Vulnerabilidad TLSv1.0 con CBC que permite descifrar tráfico MITM. Detectado por SSL/TLS Analyzer cuando TLSv1.0 está habilitado.

CVE / CVSS

CVE: identificador único de vulnerabilidades (CVE-año-número). CVSS: puntuación 0.0-10.0 de severidad con rangos Low/Medium/High/Critical.

DLL Hijacking

Colocar una DLL maliciosa en una ruta buscada antes que la ubicación legítima. Posible cuando directorios de aplicaciones privilegiadas son escribibles por usuarios normales.

EternalBlue (CVE-2017-0144)

RCE sin autenticación en SMBv1. Vector de WannaCry (2017). Detectado por el Vulnerability Scanner cuando SMBv1 está activo.

Kerberoasting

Solicitar TGS para cuentas con SPN y atacarlos offline. Posible porque cualquier usuario autenticado puede pedir TGS para cualquier SPN del dominio.

Living-off-the-Land

Usar herramientas legítimas del SO (PowerShell, WMI, certutil) para ejecutar código malicioso, evitando la detección por AV que busca ejecutables propios del atacante.

POODLE (CVE-2014-3566)

Padding oracle en SSLv3 que permite descifrar tráfico cifrado. Detectado por SSL/TLS Analyzer cuando SSLv3 está habilitado.

Process Hollowing

Crear proceso legítimo en estado suspendido, vaciar su código y reemplazarlo con payload malicioso. El proceso tiene apariencia legítima pero ejecuta código del atacante.

Unquoted Service Path

Ruta de servicio Windows con espacios y sin comillas. Permite colocar ejecutable malicioso en ruta intermedia que Windows ejecuta en lugar del servicio real al reiniciarlo.

Zero-Day

Vulnerabilidad desconocida para el fabricante o sin parche disponible en el momento de su explotación. No existe defensa directa hasta que se publica el parche.

E.3 Herramientas Referenciadas***BloodHound***

Herramienta de análisis de AD que modela relaciones como grafo y encuentra rutas de ataque con Dijkstra sobre Neo4j. Replicada parcialmente por BloodHoundCollector.

Hashcat

Cracker de hashes que aprovecha GPU. Compatible con NTLM, bcrypt, Kerberos etype 23 (Kerberoasting) y +300 algoritmos más.

Impacket

Clases Python para protocolos Windows (SMB, Kerberos, NTLM, LDAP). Dependencia opcional de los módulos Red Team avanzados de GhostTrack.

Nmap

Network Mapper (Lyon, 1997). Referencia en escaneo de puertos con SYN Scan, OS fingerprinting y scripting NSE.

Responder

Envenena LLMNR/NBT-NS para capturar hashes NTLM. Complementa al módulo NTLMRelayAttacker de GhostTrack.

sqlmap

Detección y explotación automática de SQL Injection (Damele y Stampar, 2009). Complementa al módulo SQLInjectionTester para la fase de explotación.

Apéndice F — Marco Legal y Ético Ampliado

Este apéndice complementa el capítulo 4 con un análisis más detallado de las implicaciones legales específicas del desarrollo y uso de frameworks de pentesting en el contexto español y europeo.

F.1 Elementos Esenciales del Contrato de Auditoría

El contrato de auditoría es el documento legal que diferencia el pentesting del acceso ilícito. Debe incluir como mínimo:

- Identificación de las partes con poder de representación legal de ambas organizaciones.
- Alcance exacto: listado de sistemas, rangos IP, dominios y aplicaciones autorizados. Todo sistema no listado está fuera del alcance.
- Técnicas permitidas: qué tipos de pruebas están autorizadas (escaneo, explotación, ingeniería social, acceso físico) y cuáles excluidas.
- Período de autorización: fechas de inicio y fin. La autorización no tiene validez indefinida.
- Procedimiento de escalado: qué hacer si se descubre un compromiso previo o si una prueba causa una interrupción no planificada.
- Cláusulas de confidencialidad: las vulnerabilidades descubiertas no pueden divulgarse más allá de los destinatarios del contrato.
- Responsabilidad civil: delimitación de responsabilidad por daños accidentales y recomendación de seguro de RC profesional.

F.2 La Doctrina del Daño Mínimo

En pentesting profesional existe el principio informal del 'daño mínimo': obtener la evidencia suficiente para documentar la vulnerabilidad usando el menor nivel de intrusión necesario. Si una SQL Injection puede demostrarse con un error de sintaxis SQL en la respuesta, no es ético ni necesario extraer la base de datos completa. GhostTrack aplica este principio a nivel arquitectural: el módulo SQL Injection detecta y documenta la vulnerabilidad sin explotar su impacto real.

F.3 Divulgación Responsable

La divulgación responsable (responsible disclosure) es el proceso de comunicar una vulnerabilidad descubierta al propietario del sistema antes de hacerla pública. El proceso estándar: notificación privada con detalles técnicos suficientes para reproducir y corregir la

vulnerabilidad, período de espera razonable (90 días según Google Project Zero) y divulgación pública al finalizar el plazo independientemente de si existe parche.

Los programas de bug bounty en plataformas como HackerOne y Bugcrowd permiten aplicar legalmente las habilidades de pentesting en sistemas reales con autorización explícita y potencial compensación económica. Son especialmente relevantes para estudiantes que quieran ganar experiencia práctica con sistemas reales.

F.4 Responsabilidad del Publicador de Herramientas

El Convenio de Budapest (Consejo de Europa, 2001), ratificado por España, establece en su artículo 6 que los estados deben tipificar la producción y posesión de herramientas diseñadas principalmente para cometer ciberdelitos, con una excepción explícita para herramientas de investigación y defensa legítima. La publicación de GhostTrack en el contexto de un TFG académico, con disclaimer explícito de uso ético, restricción contractual a entornos autorizados y propósito documentado de investigación, se sitúa claramente dentro de la excepción de investigación legítima.

Apéndice G — Roadmap: GhostTrack v2.0 y v3.0

Este apéndice describe las líneas de desarrollo futuro planificadas para GhostTrack, organizadas en dos releases principales.

G.1 GhostTrack v2.0 — Interfaz Web

La versión 2.0 transformará GhostTrack de una herramienta CLI en una plataforma web de gestión de auditorías construida con FastAPI (backend) y React (frontend):

- Dashboard con historial de auditorías, estadísticas de hallazgos por severidad y tendencias temporales.
- Ejecución de escaneos desde el navegador con seguimiento en tiempo real del progreso mediante WebSockets.
- Integración nativa con la API de la NVD para mantener la base de datos de CVEs actualizada semanalmente.
- API REST completa para integraciones CI/CD y herramientas de terceros (DefectDojo, Jira, ServiceNow).
- Sistema de usuarios con roles (administrador, auditor, cliente de solo lectura) para entornos multi-usuario.
- Modo asíncrono (asyncio + aiohttp/aiohttp) para Port Scanner y Subdomain Finder, estimando un speedup de 5-10×.
- Soporte IPv6 completo en todos los módulos de red.

G.2 GhostTrack v3.0 — Inteligencia Artificial

La versión 3.0 incorporará modelos de IA para automatizar el análisis y la correlación de hallazgos:

- Correlación automática de hallazgos: modelos de grafos que identifican relaciones entre hallazgos de distintos módulos que juntos representan una ruta de ataque (por ejemplo, servicio con credenciales por defecto + regla de firewall permisiva + cuenta de servicio con Kerberoasting = ruta directa a Domain Admin).
- Generación de narrativa de ataque: dado el conjunto de hallazgos, el sistema genera automáticamente un escenario narrativo de cómo un atacante encadenaría los hallazgos para comprometer el objetivo.
- Recomendaciones de remediación priorizadas por impacto y esfuerzo de implementación.

- Detección de anomalías: aprendizaje de líneas de base de configuración 'normal' para una organización y alertas cuando los resultados de una auditoría periódica difieren de la línea de base.
- Integración con LLMs para generación automática de los capítulos ejecutivos de los informes en lenguaje natural.

Apéndice H — Análisis Comparativo de Metodologías de Pentesting

Las metodologías de pentesting proporcionan marcos estructurados que guían al auditor a través del proceso de evaluación de seguridad, garantizando la cobertura de todos los vectores relevantes y la reproducibilidad de los resultados. Este apéndice analiza en profundidad las cuatro metodologías de referencia en las que se basa GhostTrack, comparando su alcance, estructura y enfoque.

H.1 PTES — Penetration Testing Execution Standard

Historia y origen

El PTES fue creado en 2010 por un grupo de profesionales de la seguridad como intento de estandarizar la industria del pentesting, que hasta ese momento carecía de un marco común ampliamente aceptado. Sus creadores —entre los que se encuentran nombres reconocidos de la comunidad de seguridad como Chris Nickerson y David Kennedy— partieron de la premisa de que la mayoría de los servicios de pentesting en el mercado tenían calidades muy dispares, y que la estandarización del proceso beneficiaría tanto a los proveedores de servicios (con un marco de trabajo claro) como a los clientes (con expectativas claras de qué incluye un pentest completo).

El estándar es un documento vivo mantenido de forma colaborativa y publicado en <http://www.pentest-standard.org/>. Aunque su última actualización formal data de 2012, los principios que establece siguen siendo válidos y son ampliamente referenciados en la industria.

Las siete fases del PTES

El PTES estructura el proceso de pentesting en siete fases secuenciales, aunque en la práctica frecuentemente se produce cierto solapamiento e iteración entre ellas:

- **Pre-Engagement Interactions:** Definición del alcance, establecimiento de las reglas de enfrentamiento (Rules of Engagement), firma del contrato de autorización, definición de los puntos de contacto de emergencia y establecimiento de los canales de comunicación durante el ejercicio. Esta fase es puramente administrativa pero es la más importante desde el punto de vista legal: sin ella, todo lo que sigue sería potencialmente ilícito.
- **Intelligence Gathering:** Recopilación de toda la información disponible sobre el objetivo usando fuentes abiertas (OSINT) y técnicas activas de baja intrusión. Corresponde a los módulos GeolP, WHOIS, DNS Enumeration, Subdomain Finder, Email Harvester y Traceroute de GhostTrack.

- **Threat Modeling:** Análisis de qué activos del objetivo son más valiosos y qué tipos de atacantes podrían estar interesados en comprometerlos. En este contexto, el threat modeling permite priorizar qué áreas de la auditoría merecen más atención: un banco debería priorizar la seguridad de su sistema de transacciones; una empresa de salud, la protección de datos de pacientes.

- **Vulnerability Analysis:** Identificación de vulnerabilidades en los sistemas objetivo mediante técnicas de escaneo activo. Corresponde a los módulos Port Scanner, Service Detection, OS Fingerprinting, Banner Grabbing, SSL/TLS Analyzer, Vulnerability Scanner, Web Crawler, Directory Bruteforce, SQL Injection y XSS Scanner de GhostTrack.

- **Exploitation:** Intento de explotación activa de las vulnerabilidades identificadas para demostrar su impacto real. Esta fase va más allá de la funcionalidad de detección de GhostTrack y requiere herramientas adicionales como Metasploit Framework o sqlmap para la explotación real.

- **Post-Exploitation:** Una vez conseguido el acceso, determinar el impacto real del compromiso: qué datos son accesibles, qué sistemas adicionales pueden comprometerse, qué persistencia puede establecerse. Corresponde a los módulos Red Team de GhostTrack.

- **Reporting:** Documentación de todos los hallazgos con evidencias, impacto y recomendaciones de remediación. El sistema de reportes de GhostTrack automatiza la generación de los reportes técnicos, aunque el reporte ejecutivo (orientado a la dirección no técnica) requiere redacción manual.

H.2 OWASP Testing Guide v4.2

Estructura y categorías

La Guía de Pruebas de OWASP es específica para la evaluación de seguridad de aplicaciones web. La versión 4.2, publicada en 2020, organiza las pruebas en doce categorías principales. Cada categoría contiene múltiples casos de prueba con un código único (OTG-CATEGORIA-NNN), una descripción del objetivo de la prueba, las técnicas recomendadas, los criterios de éxito y las referencias a vulnerabilidades relacionadas. La guía completa contiene más de 100 casos de prueba individuales.

GhostTrack implementa de forma directa o parcial los siguientes casos de prueba de OWASP:

- **OTG-INFO-003 (Enumerate Application Entry Points):** El módulo Web Crawler implementa el descubrimiento de todos los endpoints y formularios de la aplicación.

- OTG-CONFIG-004 (Enumerate Infrastructure and Application Admin Interfaces): El módulo Directory Bruteforce implementa el descubrimiento de interfaces administrativas ocultas.
- OTG-CONFIG-006 (Test HTTP Methods): El módulo Web Crawler registra los métodos HTTP permitidos por cada endpoint.
- OTG-CRYPST-001 (Testing for Weak SSL/TLS Ciphers): El módulo SSL/TLS Analyzer implementa la detección de versiones de protocolo débiles y vulnerabilidades como POODLE y BEAST.
- OTG-INPVAL-001 (Testing for Reflected Cross Site Scripting): El módulo XSS Scanner implementa la detección de XSS reflejado mediante inyección de payloads y verificación de reflexión.
- OTG-INPVAL-005 (Testing for SQL Injection): El módulo SQL Injection Scanner implementa la detección de Error-Based SQL Injection.

H.3 MITRE ATT&CK: Profundidad Técnica

La matriz Enterprise en detalle

La matriz ATT&CK Enterprise version 14 (MITRE Corporation, 2023) contiene 14 tácticas, 196 técnicas y 411 sub-técnicas. La siguiente tabla muestra la cobertura de GhostTrack por táctica:

Tabla 4 Cobertura de GhostTrack por táctica MITRE ATT&CK Enterprise v14 Nota. Elaboración propia basada en MITRE Corporation (2023). n/a = táctica cubierta parcialmente o de forma indirecta.

Táctica	ID	Cobertura GhostTrack	Módulos principales
Reconnaissance	TA0043	Alta — 5/10 técnicas	GeoIP, WHOIS, DNS Enum, Subdomain Finder, Email Harvester
Resource Development	TA0042	Baja — 1/7 técnicas	PayloadGenerator (parcial)
Initial Access	TA0001	Media — 3/9 técnicas	WiFiAttacker, SocialEngineering, PayloadGenerator
Execution	TA0002	Media — 4/13 técnicas	Custom Scripts, PayloadGenerator, ProcessInjector
Persistence	TA0003	Alta — 5/19 técnicas	PersistenceChecker, DLLHijacking, Custom Scripts
Privilege Escalation	TA0004	Alta — 6/13 técnicas	PrivEscChecker, TokenManipulator, DLLHijacking

Táctica	ID	Cobertura GhostTrack	Módulos principales
Defense Evasion	TA0005	Alta — 8/42 técnicas	AVEvasion, FirewallEvasion, ProcessInjector, TokenManipulator
Credential Access	TA0006	Alta — 6/17 técnicas	KerberosAttacker, NTLMRelayAttacker, CredentialDumper
Discovery	TA0007	Alta — 9/31 técnicas	SMBEnum, LDAPEnum, BloodHound, PortScanner, ServiceDetection
Lateral Movement	TA0008	Alta — 4/9 técnicas	LateralMovement, PassTheHash, WinRM, DCOM
Collection	TA0009	Media — 2/17 técnicas	NetworkSniffer, CredentialDumper
Command and Control	TA0011	Alta — 5/16 técnicas	C2Communication (HTTPS, DNS, ICMP, DGA)
Exfiltration	TA0010	Alta — 4/9 técnicas	DataExfiltration (LSB, DNS, Cloud)
Impact	TA0040	Baja — 0/13 técnicas	No implementado (fuera del alcance)

La táctica Impact (TA0040) no está implementada deliberadamente. Las técnicas de Impact incluyen cifrado de datos para ransomware, destrucción de datos y sabotaje de sistemas, que están completamente fuera del alcance de un framework de pentesting ético y de uso académico.

H.4 Comparación de Metodologías

Tabla 5 Comparación de metodologías de referencia de GhostTrackNota. Elaboración propia.

Aspecto	PTES	OWASP TG	MITRE ATT&CK	NIST CSF
Enfoque	Proceso completo de pentesting	Aplicaciones web	Tácticas y técnicas del atacante	Gestión del riesgo
Público objetivo	Proveedores de pentesting	Evaluadores web	Red Team / Threat Intel	CISO / Gestión
Granularidad	Alta (fases + técnicas)	Muy alta (100+ tests)	Muy alta (196 técnicas)	Baja (5 funciones)
Actualización	2012 (sin actualizar)	2020 (v4.2)	Frecuente (v14, 2023)	2024 ()
Uso en GhostTrack	Estructura del proceso	Módulos web	Mapeo de módulos RT	Marco general

Apéndice I — Análisis del Código Fuente

Este apéndice proporciona métricas objetivas del código fuente de GhostTrack, incluyendo líneas de código por módulo, complejidad ciclomática de las funciones más críticas, y un análisis de la distribución de la lógica entre los diferentes componentes del sistema.

I.1 Métricas de Líneas de Código

Tabla 6 Métricas de líneas de código por fichero (LOC = líneas de código, excluye comentarios y blancos) Nota. Elaboración propia. Medido con radon cc y cloc.

Fichero	LOC totales	LOC código	LOC comentarios	Clases	Funciones
main.py	553	412	87	1	4
modules/geoip.py	98	71	18	0	5
modules/whois_lookup.py	187	141	31	1	3
modules/dns_enum.py	198	152	28	1	4
modules/subdomain_finder.py	247	189	38	1	5
modules/port_scanner.py	228	171	42	1	7
modules/service_detection.py	162	121	27	1	4
modules/os_fingerprint.py	143	108	22	1	5
modules/banner_grabbing.py	156	118	24	1	4
modules/ssl_analyzer.py	213	164	34	1	5
modules/vuln_scanner.py	198	147	38	1	4
modules/traceroute.py	112	84	19	1	3
modules/web_crawler.py	218	165	38	1	6
modules/directory_bruteforce.py	187	143	28	1	3
modules/sql_injection.py	142	108	24	1	3
modules/xss_scanner.py	98	74	16	1	2
modules/arp_scanner.py	78	58	14	1	2
modules/network_sniffer.py	67	49	13	1	2
modules/email_harvester.py	89	67	16	1	3
modules/metadata_extractor.py	112	84	21	1	3
modules/custom_scripts.py	98	74	18	1	3

Fichero	LOC totales	LOC código	LOC comentarios	Clases	Funciones
modules/redteam/complete_redteam.py	1847	1423	289	20	42
utils/colors.py	78	58	14	1	5
utils/logger.py	89	67	16	1	8
utils/validator.py	112	84	21	1	9
utils/reporter.py	198	149	35	1	5
TOTAL	~5.773	~4.382	~955	~38	~165

El fichero `modules/redteam/complete_redteam.py` es el más extenso del proyecto con ~1.847 líneas totales, lo que refleja la complejidad de las veinte clases Red Team implementadas en un único fichero. En versiones futuras este fichero podría dividirse en módulos individuales siguiendo el mismo patrón que los módulos básicos.

I.2 Complejidad Ciclomática

La complejidad ciclomática (McCabe, 1976) mide el número de caminos linealmente independientes a través de un método o función. Un valor de 1-5 indica código simple y fácil de testear; 6-10 indica complejidad moderada; 10+ indica alta complejidad y necesidad de refactorización. Las siguientes funciones de GhostTrack tienen la mayor complejidad ciclomática:

Tabla 7 Funciones de mayor complejidad ciclomática en GhostTrack Nota. Elaboración propia. Medido con radon cc -a. Umbral de refactorización: CC > 10.

Función / Método	Módulo	CC	Evaluación
<code>_parse_whois()</code>	<code>whois_lookup.py</code>	9	Moderada — múltiples regex por campo
<code>determine_os()</code>	<code>os_fingerprint.py</code>	7	Aceptable — lógica de votación
<code>run_full_scan()</code>	<code>main.py</code>	12	Alta — debería dividirse en submétodos
<code>scan()</code>	<code>port_scanner.py</code>	6	Aceptable — lógica de threading sencilla
<code>_crawl_recursive()</code>	<code>web_crawler.py</code>	8	Moderada — múltiples condiciones de parada

Función / Método	Módulo	CC	Evaluación
test_url()	sql_injection.py	7	Aceptable — bucle doble payload/parámetro
check_vulnerabilities()	ssl_analyzer.py	9	Moderada — múltiples checks independientes
check_misconfigurations()	vuln_scanner.py	8	Aceptable — múltiples condiciones de puerto
run_full_redteam_assessment()	complete_redteam.py	18	Muy alta — debe dividirse

Las dos funciones con $CC > 10$ (`run_full_scan()` en `main.py` con $CC=12$ y `run_full_redteam_assessment()` en `complete_redteam.py` con $CC=18$) son candidatas prioritarias a refactorización en la versión 2.1. La estrategia de refactorización sería extraer la invocación de cada módulo a un método privado dedicado (`_run_geoiip()`, `_run_whois()`, etc.), reduciendo el CC de la función principal a 1-2.

I.3 Análisis de Dependencias entre Módulos

El grafo de dependencias entre módulos es relevante para entender el acoplamiento del sistema y para planificar cambios sin efectos secundarios no deseados. En GhostTrack el grafo de dependencias es deliberadamente simple gracias a la arquitectura modular:

- `main.py` depende de todos los módulos (directo) y de `utils/` (directo).
- Todos los módulos básicos son hojas del grafo: no dependen de ningún otro módulo de GhostTrack.
- `complete_redteam.py` es también una hoja: sus clases no importan módulos del directorio `modules/`.
- `utils/` es el único paquete del que múltiples módulos podrían depender, pero en la implementación actual los módulos no importan `utils/` directamente; el logging y la presentación se hacen con `print()` y sin usar el `Logger` ni el `Reporter`.

Esta estructura de grafo en forma de estrella (`main.py` en el centro, módulos como hojas) es la ideal para un sistema modular: minimiza el acoplamiento y garantiza que cualquier módulo puede modificarse, reemplazarse o eliminarse sin afectar a los demás. La única excepción es si un módulo usara los resultados de otro como entrada (`Service Detection` usa los puertos de `Port Scanner`, `Vulnerability Scanner` usa los servicios de `Service Detection`), lo que se gestiona pasando los resultados explícitamente a través del orquestador en lugar de mediante importaciones directas entre módulos.

Apéndice J — Configuración Avanzada del Framework

Este apéndice describe todas las opciones de configuración disponibles en GhostTrack, incluyendo el fichero `config.json`, las variables de entorno del fichero `.env` y las opciones de configuración avanzada que no están expuestas directamente en la CLI.

J.1 Referencia Completa de `config.json`

El fichero `config.json` en la raíz del proyecto controla el comportamiento del framework sin necesidad de modificar el código fuente. A continuación se describe cada parámetro disponible:

```
{
  "application": {
    "name": "GhostTrack",           // Nombre del framework
    "version": "2.0",              // Versión actual
    "author": "Izán Jiménez Nuñez", // Autor
    "description": "...",          // Descripción para los reportes
  },

  "scanning": {
    "default_threads": 10,         // Threads por defecto en módulos multi-thread
    "default_timeout": 5,          // Timeout de red en segundos
    "max_threads": 100,            // Límite máximo de threads
    "max_timeout": 30,             // Timeout máximo permitido
    "user_agent": "GhostTrack/2.0 Security Scanner" // UA para peticiones
    HTTP
  },

  "ports": {
    // Listas de puertos para diferentes perfiles de escaneo
    "common": [21,22,23,25,53,80,110,143,443,445,3306,3389,5432,8080],
    "web": [80,443,8080,8443,8888],
    "database": [1433,1521,3306,5432,6379,27017],
    "remote": [22,23,3389,5900]
  },

  "wordlists": {
    "directories": "data/wordlists/directories.txt",
    "subdomains": "data/wordlists/subdomains.txt",
    "passwords": "data/wordlists/passwords.txt"
  },

  "output": {
```

```

    "reports_dir": "reports",          // Directorio de salida de reportes
    "logs_dir":    "logs",             // Directorio de ficheros de log
    "format":      ["html","json","txt"], // Formatos a generar
    "save_raw_data": true              // Guardar datos crudos en reportes
  },

  "logging": {
    "level": "INFO",                  // Nivel mínimo de log
    (DEBUG/INFO/WARNING/ERROR)
    "file_rotation": true,            // Rotar ficheros de log por sesión
    "max_file_size": 10485760,        // Tamaño máximo del log (10 MB)
    "backup_count": 5                  // Número de logs históricos a mantener
  },

  "modules": {
    "enabled": ["geoip","whois",...], // Módulos activos en --full
    "disabled": []                    // Módulos desactivados
  },

  "redteam": {
    "enabled": true,                  // Habilitar módulos Red Team
    "require_confirmation": true      // Pedir confirmación extra antes de RT
  },

  "api_keys": {
    "shodan": "",                    // API key de Shodan
    "virustotal": "",                // API key de VirusTotal
    "censys": "",                    // API key de Censys
    "securitytrails": ""             // API key de SecurityTrails
  },

  "legal": {
    "disclaimer_required": true,      // Mostrar disclaimer en cada ejecución
    "log_all_scans": true,            // Registrar todas las operaciones en log
    "ethical_use_only": true          // Marker de uso ético (informativo)
  }
}

```

J.2 Variables de Entorno (.env)

El fichero `.env` almacena credenciales y configuraciones sensibles que no deben incluirse en el repositorio de código:

```

# API Keys para módulos opcionales
SHODAN_API_KEY=tu_clave_shodan
VIRUSTOTAL_API_KEY=tu_clave_vt

```

```

CENSYS_API_KEY=tu_clave_censys
CENSYS_API_SECRET=tu_secreto_censys
SECURITYTRAILS_API_KEY=tu_clave_st

# Proxy (para auditorías que requieren enrutar a través de proxy)
HTTP_PROXY=http://proxy.corporativo:8080
HTTPS_PROXY=http://proxy.corporativo:8080
NO_PROXY=localhost,127.0.0.1

# Configuración avanzada de logging
LOG_LEVEL=DEBUG # DEBUG para diagnóstico, INFO para uso normal

# Credenciales para módulos autenticados (Red Team)
DOMAIN_USER=usuario@corp.local
DOMAIN_PASS=contraseña
DOMAIN_HASH=aad3b435b51404eeaad3b435b51404ee:31d6cfe0... # NTLM hash

```

J.3 Configuración de Wordlists Personalizadas

Las wordlists internas de GhostTrack pueden complementarse con wordlists externas de mayor cobertura. El fichero config.json permite especificar rutas a wordlists personalizadas que reemplazarán las internas:

```

# Descarga de SecLists (recomendado para auditorías en profundidad)
git clone https://github.com/danielmiessler/SecLists.git data/seclists/

# Actualizar config.json para usar SecLists
# "wordlists": {
#   "directories": "data/seclists/Discovery/Web-Content/common.txt",
#   "subdomains":
# "data/seclists/Discovery/DNS/subdomains-top1million-5000.txt"
# }

# Ejecutar con wordlist grande
python main.py -t objetivo.com -m directory_bruteforce --threads 50

```

Apéndice K — Preguntas Frecuentes (FAQ)

Este apéndice recoge las preguntas más frecuentes sobre el uso, configuración y alcance de GhostTrack, organizadas por categorías.

K.1 Instalación y Requisitos

¿GhostTrack funciona en Windows?

Sí, GhostTrack es compatible con Windows 10 y Windows 11. Sin embargo, algunos módulos tienen limitaciones: el módulo Network Sniffer requiere Npcap instalado (el equivalente Windows de libpcap) para que Scapy pueda capturar paquetes, y los módulos que usan comandos del sistema (Traceroute, DNS Enumeration) detectan automáticamente si se están ejecutando en Windows y usan los comandos equivalentes (tracert, nslookup).

Para una experiencia óptima se recomienda Linux, específicamente Kali Linux, que incluye la mayoría de las dependencias del sistema preinstaladas. En Windows, es posible usar WSL2 (Windows Subsystem for Linux) para ejecutar GhostTrack en un entorno Linux dentro de Windows, lo que proporciona compatibilidad completa.

¿Qué versión de Python es necesaria?

GhostTrack requiere Python 3.8 como mínimo. Se recomienda usar Python 3.10 o superior para mejor rendimiento y compatibilidad con las últimas versiones de las dependencias. La verificación de la versión se realiza automáticamente al inicio de la ejecución; si la versión es inferior a 3.8 el programa informa del error y termina limpiamente.

¿Puedo instalar GhostTrack sin conexión a Internet?

La instalación de dependencias mediante pip requiere conexión a Internet. Sin embargo, si se dispone de los paquetes en formato .whl o en un índice de paquetes privado (como PyPI Mirror o Nexus), es posible instalar las dependencias sin conectividad externa usando el flag `--find-links` de pip.

Durante la ejecución, los módulos GeolIP, WHOIS, Subdomain Finder (parte de CT Logs) y Certificate Transparency requieren conectividad a Internet con los servidores de las APIs correspondientes. Los módulos de red (Port Scanner, Service Detection, Banner Grabbing, etc.) solo requieren conectividad con el objetivo auditado.

K.2 Uso y Funcionamiento

¿Qué diferencia hay entre --full y --full --redteam?

El flag `--full` ejecuta los veinte módulos básicos de GhostTrack, cubriendo las fases de reconocimiento, enumeración y análisis de vulnerabilidades. El flag `--redteam` ejecuta los veinte módulos Red Team adicionales, orientados a Active Directory y post-explotación. Usar ambos flags simultáneamente ejecuta los cuarenta módulos en secuencia, lo que constituye el escaneo más completo disponible. El tiempo de ejecución con `--full --redteam` es la suma del tiempo de cada bloque individualmente.

¿Por qué el Port Scanner no detecta todos los puertos abiertos que Nmap sí detecta?

Si Nmap detecta puertos que GhostTrack no detecta, la causa más probable es el timeout configurado. GhostTrack usa por defecto un timeout de 1 segundo por puerto; si el servidor tiene latencia alta o está detrás de una conexión VPN lenta, puede necesitar un timeout mayor. Prueba con `python main.py -t objetivo --full --timeout 5` para aumentar el timeout a 5 segundos. Si el problema persiste, puede ser que Nmap esté usando SYN Scan (que solo envía el paquete SYN sin esperar la conexión completa, más rápido pero requiere root) mientras que GhostTrack usa TCP Connect Scan (conexión completa, más lento pero sin root).

¿Cómo puedo añadir mis propias wordlists al Directory Bruteforce?

Hay dos formas. La primera es la opción de script personalizado: crea un script Python usando el módulo Custom Scripts que invoque el DirectoryBruteforcer con tu wordlist como parámetro. La segunda es actualizar el fichero `config.json` para apuntar a tu wordlist personalizada en el campo `wordlists.directories`, de modo que el módulo Directory Bruteforce la use automáticamente en escaneos `--full`.

¿GhostTrack guarda datos de los objetivos escaneados en servidores externos?

No. GhostTrack es una herramienta completamente offline: todos los resultados se almacenan localmente en el directorio `reports/` y en los ficheros de log del directorio `logs/`. Los únicos datos que se envían a servidores externos son las consultas de reconocimiento (petición a `ip-api.com` con la IP objetivo para GeoIP, consulta WHOIS al servidor del registrador, consulta a `crt.sh` con el dominio para CT Logs). Estos servidores son servicios públicos cuyas peticiones no identifican al auditor más allá de su IP de origen.

K.3 Seguridad y Legalidad

¿Puedo usar GhostTrack contra mi propio servidor en Internet?

Sí, siempre que el servidor sea de tu propiedad o tengas autorización escrita del propietario. Si el servidor está alojado en un proveedor cloud (AWS, Azure, GCP, OVH, Hetzner, etc.), algunos proveedores tienen políticas que requieren notificación previa antes de realizar pruebas de penetración contra sus sistemas aunque sean tuyos. Verifica la política de pentesting de tu proveedor antes de ejecutar GhostTrack contra servidores en cloud.

¿Qué pasa si GhostTrack accidentalmente escanea un sistema fuera del alcance?

El módulo Validator valida el formato del target antes de iniciar cualquier escaneo, pero no puede verificar si una IP pertenece al alcance autorizado; esa responsabilidad recae en el auditor. Si por error se escanea un sistema fuera del alcance, es importante documentarlo inmediatamente, informar al cliente y, si se generaron alertas en los sistemas del tercero afectado, coordinar con el cliente para aclarar la situación. Los logs de GhostTrack con timestamps precisos son evidencia de que el escaneo fue un error no intencionado y no una acción maliciosa deliberada.

¿Cuánto tiempo debo conservar los logs y reportes de una auditoría?

No existe un período de retención legalmente obligatorio específico para logs de pentesting en España. Sin embargo, se recomienda conservarlos durante al menos el período de prescripción de las infracciones penales relacionadas con el acceso no autorizado a sistemas (5 años según el artículo 131 del Código Penal para delitos con penas de hasta 3 años). Durante este período, los logs y reportes son la única evidencia disponible para demostrar que las actividades se realizaron dentro del alcance autorizado en caso de una disputa o investigación posterior.

Apéndice L — Análisis de Riesgos y Mitigaciones por Módulo

Este apéndice sintetiza, para cada uno de los cuarenta módulos de GhostTrack, el riesgo que la vulnerabilidad detectada representa para una organización, la facilidad de explotación estimada, la disponibilidad de exploit público y las medidas de mitigación recomendadas. Esta información está diseñada para facilitar la priorización del plan de remediación por parte del equipo de seguridad del cliente.

L.1 Módulos Básicos — Análisis de Riesgo

Tabla 8 *Análisis de riesgo y mitigación — Módulos básicos de GhostTrack* Nota. Elaboración propia.

Facilidad de explotación: Alta = explotable sin conocimientos especializados; Media = requiere conocimientos de seguridad; Baja = requiere expertise avanzado.

Módulo	Vulnerabilidad tipo detectada	Riesgo máx.	Facilidad explot.	Mitigación principal
GeoIP	Exposición de infraestructura	INFO	—	Usar CDN / anycast para ocultar IPs reales
WHOIS	Datos de contacto expuestos	LOW	—	Habilitar privacidad WHOIS en el registrador
DNS Enum	Zone transfer / registros sensibles	HIGH	Alta	Restringir AXFR a IPs de NS secundarios
Subdomain Finder	Subdominios de dev/staging expuestos	HIGH	Media	Eliminar/proteger subdominios no productivos
Port Scanner	Servicios innecesarios expuestos	MEDIUM	Alta	Firewall: principio de mínimo privilegio
Service Detection	Versiones con CVEs conocidos	CRITICAL	Alta	Actualizar software, ocultar versiones en banners
OS Fingerprinting	SO identificado, CVEs aplicables	MEDIUM	Media	Normalizar TTL, suprimir banners de SO
Banner Grabbing	Información de versión expuesta	MEDIUM	Alta	server_tokens off (nginx), ServerTokens Prod (Apache)
SSL/TLS Analyzer	Protocolos débiles / cert expirado	HIGH	Media	TLSv1.2+ mínimo, renovar certificados, HSTS
Vulnerability Scanner	CVEs de alta severidad	CRITICAL	Alta	Parcheo inmediato, WAF como mitigación temporal
Traceroute	Topología de red interna expuesta	LOW	—	Filtrar ICMP/UDP de salida en el perímetro
Web Crawler	Endpoints y formularios mapeados	MEDIUM	Media	Robots.txt, autenticación en paneles admin

Módulo	Vulnerabilidad tipo detectada	Riesgo máx.	Facilidad explot.	Mitigación principal
Dir Bruteforce	Directorios admin accesibles	HIGH	Alta	Eliminar recursos legacy, autenticación por IP
SQL Injection	Inyección SQL en parámetros	CRITICAL	Alta	Prepared statements, WAF, validación de entrada
XSS Scanner	Cross-Site Scripting reflejado	HIGH	Alta	HTML encoding, CSP headers, validación de entrada
ARP Scanner	Inventario de red local	INFO	—	Segmentación de red, 802.1X port authentication
Network Sniffer	Credenciales en texto claro	CRITICAL	Media	Cifrar todos los protocolos (TLS, SSH, VPN)
Email Harvester	Emails para phishing / brute force	MEDIUM	—	No publicar emails directos, DMARC p=reject
Metadata Extractor	Metadatos con info interna	MEDIUM	—	Limpiar metadatos antes de publicar documentos
Custom Scripts	Depende del script ejecutado	VARIABLE	—	Revisar scripts antes de ejecutar

L.2 Módulos Red Team — Análisis de Riesgo

Tabla 9 *Análisis de riesgo y mitigación — Módulos Red Team de GhostTrack* Nota. Elaboración propia. Impacto: indica el peor escenario si la técnica tiene éxito en un entorno sin protecciones específicas.

Módulo	Técnica	Impacto si éxito	Contramedida principal
SMBEnumerator	Null session / shares enum	Inventario completo de AD	SMB Signing req. + RestrictNullSessAccess=1
LDAPEnumerator	LDAP anónimo / auth básica	Mapa completo de AD + SPNs	LDAP Signing + Channel Binding obligatorios
KerberosAttacker	Kerberoasting / AS-REP Roasting	Credenciales de cuentas de servicio	gMSA + contraseñas largas + auditoría SPNs
NTLMRelayAttacker	NTLM relay a SMB/LDAP	Ejecución remota / creación de cuentas	SMB Signing req. + deshabilitar LLMNR/NBT-NS
BloodHoundCollector	Recolección de datos AD	Mapa completo de rutas de ataque	Tier model + Protected Users group
PrivEscChecker	Unquoted paths / token abuse	Escalada a SYSTEM/root	Auditoría de permisos + Credential Guard
LateralMovement	Pass-the-Hash / WinRM	Acceso a múltiples sistemas	Segmentación + MFA + EDR comportamental

Módulo	Técnica	Impacto si éxito	Contramedida principal
CredentialDumper	LSASS dump / SAM dump	Todos los hashes del sistema	Credential Guard + LAPS + Protected Users
TokenManipulator	Token impersonation	Ejecución como SYSTEM	Deshabilitar SeImpersonatePrivilege donde posible
ProcessInjector	DLL/Process Hollowing	Evasión de AV, persistencia	EDR con memory scanning + Exploit Protection
DLLHijacking	DLL search order hijacking	Persistencia con privilegios elevados	Permisos estrictos en directorios de aplicaciones
PersistenceChecker	Registry Run / Tasks / WMI	Acceso persistente post-reboot	Monitorización de autoruns + WMI subscriptions
FirewallEvasion	DNS tunneling / HTTPS C2	Canal C2 indetectable	DNS filtering + TLS inspection + proxy web
AVEvasion	AMSI bypass / sleep obfuscation	Ejecución de payload sin detección	EDR comportamental + threat hunting proactivo
PayloadGenerator	Meterpreter / Empire agent	Control total del sistema	AppLocker/WDAC + application whitelisting
C2Communication	HTTPS/DNS/ICMP C2	Comunicación persistente C2	Behavioral analysis + UEBA + anomaly detection
DataExfiltration	LSB steganography / DNS exfil	Fuga silenciosa de datos	DLP + CASB + DNS monitoring + data classification
WiFiAttacker	WPA2 handshake capture	Acceso a red corporativa WiFi	WPA3 + 802.11w + red WiFi segregada
BluetoothScanner	BlueBorne / KNOB	RCE en dispositivos Bluetooth	Deshabilitar BT, actualizar firmware
SocialEngineering	Spear phishing / cloning	Initial access con credenciales reales	Formación usuarios + MFA + sandbox de email

Apéndice M — Entornos de Laboratorio Recomendados

Este apéndice describe los entornos de laboratorio más adecuados para practicar con GhostTrack de forma segura y legal, cubriendo tanto plataformas online accesibles sin infraestructura propia como entornos locales virtualizados para pruebas más avanzadas.

M.1 Plataformas Online para Práctica Legal

HackTheBox (HTB)

HackTheBox es una plataforma online que ofrece máquinas virtuales intencionalmente vulnerables accesibles mediante VPN. Las máquinas cubren una amplia variedad de sistemas operativos, servicios y vectores de ataque. GhostTrack puede usarse directamente contra las máquinas de HTB como herramienta de reconocimiento inicial y análisis de vulnerabilidades, con el resultado de que los hallazgos del escaneo --full proporcionarán el punto de partida para la explotación manual. La plataforma tiene dos modalidades: Machines (retos individuales) y Pro Labs (entornos empresariales completos con múltiples máquinas y relaciones entre ellas, similar a una auditoría real).

Para usar GhostTrack con HTB: conectar a la VPN de HTB descargando el fichero .ovpn y ejecutando `openvpn htb.ovpn`; verificar la conectividad con ping a la IP de la máquina objetivo; ejecutar `python main.py -t IP_MAQUINA --full`. La IP de cada máquina es visible en el panel de HackTheBox una vez que se activa.

TryHackMe

TryHackMe es una plataforma similar a HackTheBox pero con enfoque más pedagógico: cada reto está acompañado de guías paso a paso que explican los conceptos y técnicas necesarios para resolverlo. Es especialmente adecuada para quienes están aprendiendo pentesting y quieren entender el porqué de cada técnica, no solo el cómo. GhostTrack puede usarse como herramienta complementaria a las herramientas sugeridas en los retos de TryHackMe para comparar resultados y desarrollar intuición sobre qué herramienta usar en cada contexto.

VulnHub

VulnHub es una colección gratuita de máquinas virtuales intencionalmente vulnerables que se descargan y ejecutan localmente (no requieren VPN ni suscripción). La ventaja es la privacidad total y la posibilidad de pausar, restaurar y modificar las máquinas. Machines relevantes para practicar con GhostTrack: Metasploitable2 y Metasploitable3 (múltiples

servicios vulnerables), DVWA (Damn Vulnerable Web Application, ideal para módulos web), y OWASP WebGoat.

Pentesterlab

PentesterLab es una plataforma orientada específicamente a la evaluación de seguridad web. Sus ejercicios cubren en profundidad las vulnerabilidades del OWASP Top 10, incluyendo variantes avanzadas de SQL Injection, XSS, XXE, SSRF y lógica de negocio. Los módulos web de GhostTrack (Web Crawler, Directory Bruteforce, SQL Injection, XSS Scanner) pueden usarse como punto de partida en los ejercicios de PentesterLab.

M.2 Entorno Local Virtualizado con VirtualBox

Para las pruebas más avanzadas, especialmente los módulos Red Team que requieren un entorno de Active Directory completo, es necesario un entorno local virtualizado. La siguiente configuración fue la usada en el desarrollo y validación de GhostTrack:

Hardware recomendado

- CPU: 4+ cores con soporte de virtualización (Intel VT-x o AMD-V habilitado en BIOS).
- RAM: 16 GB mínimo (8 GB para el sistema anfitrión + 8 GB para las VMs).
- Almacenamiento: 100 GB de espacio libre en SSD para las imágenes de VM.

Configuración de red en VirtualBox

Todas las VMs del laboratorio deben estar en una red 'Host-Only' de VirtualBox (o 'Internal Network' para mayor aislamiento), completamente separada del acceso a Internet del sistema anfitrión. Esta configuración garantiza que ningún tráfico de las pruebas sale a Internet accidentalmente. Los pasos son: VirtualBox > Archivo > Administrador de Red Anfitrión > Crear (por ejemplo, 192.168.100.0/24); asignar todas las VMs del laboratorio a este adaptador.

VMs del laboratorio completo

- Kali Linux 2024 (auditor): Sistema desde el que se ejecuta GhostTrack. Descarga disponible en <https://www.kali.org/get-kali/>. Configurar en la red Host-Only y asignarle IP estática 192.168.100.10.
- Metasploitable3 (objetivo vulnerable): Máquina Ubuntu con múltiples servicios vulnerables instalados deliberadamente. Disponible en <https://github.com/rapid7/metasploitable3>. IP estática 192.168.100.20.

- DVWA en Apache (objetivo web): Docker o VM con DVWA para pruebas de módulos web. `docker run -d -p 80:80 vulnerables/web-dvwa`. IP 192.168.100.25.
- Windows Server 2019 con AD (controlador de dominio): Para módulos Red Team. Requiere licencia de evaluación gratuita de 180 días disponible en Microsoft Evaluation Center. Configurar como DC del dominio lab.local con IP 192.168.100.30.
- Windows 10 cliente (víctima AD): Un equipo unido al dominio lab.local para simular un usuario comprometido. IP 192.168.100.40.

Configuración del dominio de laboratorio

Una vez instalado Windows Server 2019, configurar Active Directory Domain Services (AD DS) con el nombre de dominio lab.local mediante el Asistente de Configuración de AD DS. Crear los siguientes objetos para las pruebas:

```
# PowerShell - Configuración del laboratorio AD
# Crear OUs
New-ADOrganizationalUnit -Name 'Users' -Path 'DC=lab,DC=local'
New-ADOrganizationalUnit -Name 'Servers' -Path 'DC=lab,DC=local'
New-ADOrganizationalUnit -Name 'ServiceAccounts' -Path 'DC=lab,DC=local'

# Crear usuarios de prueba
New-ADUser -Name 'jgarcia' -AccountPassword (ConvertTo-SecureString
'Password1!' -AsPlainText -Force) -Enabled $true
New-ADUser -Name 'mlopez' -AccountPassword (ConvertTo-SecureString
'Summer2023!' -AsPlainText -Force) -Enabled $true

# Crear cuenta de servicio CON SPN (vulnerable a Kerberoasting)
New-ADUser -Name 'svc_sql' -AccountPassword (ConvertTo-SecureString 'sql123'
-AsPlainText -Force) -Enabled $true
setspn -A MSSQLSvc/sqlserver01.lab.local:1433 lab\svc_sql

# Crear cuenta SIN preauth Kerberos (vulnerable a AS-REP Roasting)
New-ADUser -Name 'svc_web' -AccountPassword (ConvertTo-SecureString 'web456'
-AsPlainText -Force) -Enabled $true
Set-ADAccountControl -Identity svc_web -DoesNotRequirePreAuth $true

# Verificar configuraciones vulnerables
Get-ADUser -Filter {ServicePrincipalName -ne '$null'} -Properties
ServicePrincipalName
Get-ADUser -Filter {DoesNotRequirePreAuth -eq $true} -Properties
DoesNotRequirePreAuth
```

M.3 Laboratorio con Docker Compose

Para un setup más ligero y reproducible, se puede levantar un entorno de pruebas para los módulos web usando Docker Compose sin necesidad de VMs completas:

```
# docker-compose.yml — Entorno de laboratorio web
version: '3.8'
services:
  dvwa:
    image: vulnerables/web-dvwa
    ports: ['80:80']
    networks: [lab]

  webgoat:
    image: webgoat/webgoat
    ports: ['8080:8080']
    networks: [lab]

  juice-shop:
    image: bkimminich/juice-shop
    ports: ['3000:3000']
    networks: [lab]

  elasticsearch:
    image: elasticsearch:7.17.0
    environment:
      - discovery.type=single-node
      - xpack.security.enabled=false # Intencionalmente inseguro
    ports: ['9200:9200']
    networks: [lab]

networks:
  lab:
    driver: bridge
    ipam:
      config: [{subnet: '172.20.0.0/24'}]

# Levantar el entorno
docker-compose up -d
# GhostTrack contra los servicios
python main.py -t 172.20.0.1 --full --threads 20
```

Apéndice N — Módulos Básicos: Análisis Técnico Adicional

Este apéndice complementa la descripción del capítulo 7 con información técnica adicional sobre los módulos que por su complejidad merecen un análisis más detallado: el funcionamiento interno de las expresiones regulares de parsing, los algoritmos de análisis y las decisiones de diseño que no pudieron explicarse en extenso en el cuerpo principal del manual.

N.1 Análisis Profundo del Port Scanner

N.1.1 TCP Three-Way Handshake y `socket.connect_ex()`

El módulo PortScanner usa `socket.connect_ex()` en lugar de `socket.connect()` para sondear cada puerto. La diferencia es importante: `socket.connect()` lanza una excepción `ConnectionRefusedError` cuando el puerto está cerrado, requiriendo un bloque `try/except` para cada intento; `socket.connect_ex()` devuelve el código de error del sistema operativo sin lanzar excepción (0 = éxito, `errno.ECONNREFUSED` = rechazado, `errno.ETIMEDOUT` = timeout). Esto hace el código más limpio y potencialmente más eficiente porque evita la sobrecarga de la gestión de excepciones en el happy path de los puertos cerrados.

El three-way handshake completo que realiza TCP Connect Scan consta de tres segmentos: el cliente envía SYN al servidor; si el puerto está abierto el servidor responde con SYN-ACK; el cliente envía ACK completando la conexión. Inmediatamente después, el cliente envía RST para cerrar la conexión ya que no necesita comunicarse realmente con el servicio. Esta secuencia completa es visible en los logs del servidor objetivo, lo que hace el TCP Connect Scan detectable por sistemas IDS/IPS. La alternativa más sigilosa es el SYN Scan (Half-Open Scan), donde el cliente envía SYN pero al recibir SYN-ACK en lugar de completar el handshake con ACK envía RST, evitando que la conexión se establezca completamente y aparezca en los logs de conexiones del servidor.

N.1.2 Manejo de ICMP Unreachable en UDP Scan

El escaneo UDP es fundamentalmente distinto al TCP por la naturaleza sin conexión del protocolo. Cuando se envía un datagrama UDP a un puerto cerrado, el sistema operativo del objetivo genera un mensaje ICMP de tipo 3 (Destination Unreachable) con código 3 (Port Unreachable), que indica explícitamente que no hay ningún proceso escuchando en ese puerto. Cuando el puerto está abierto, el servicio puede generar una respuesta UDP (como DNS, que responde a cualquier consulta) o simplemente no generar ninguna respuesta si el protocolo de aplicación no lo requiere.

La ambigüedad surge porque la ausencia de respuesta puede indicar tanto 'puerto abierto y el servicio no genera respuesta a este datagrama' como 'el mensaje ICMP Port Unreachable fue filtrado por un firewall'. Para distinguir entre ambos casos se necesitaría saber si hay un firewall filtrando ICMP, lo que requiere técnicas adicionales fuera del alcance del módulo actual.

N.2 Análisis Profundo del Web Crawler

N.2.1 Normalización de URLs

El Web Crawler debe manejar correctamente las diferentes formas en que pueden aparecer los enlaces en HTML. La función `urljoin()` de `urllib.parse` resuelve URLs relativas a partir de la URL base de la página actual, manejando correctamente los casos más comunes: `href='./pagina.html'` (relativo al directorio padre), `href='/ruta/pagina.html'` (relativo a la raíz del servidor), `href='pagina.html'` (relativo al directorio actual) y `href='https://otro.dominio.com/pagina.html'` (URL absoluta). Sin esta normalización, los enlaces relativos producirían URLs malformadas que no pueden resolverse.

Un caso especial importante son los enlaces con anchors (`#`): `href='pagina.html#seccion1'` es esencialmente la misma URL que `href='pagina.html'` desde el punto de vista del servidor (el anchor es procesado por el navegador del cliente, no por el servidor). El crawler filtra los anchors comparando solo el componente de ruta de la URL (scheme + netloc + path + query) sin el fragment, evitando visitar la misma página varias veces por distintos anchors.

N.2.2 Manejo de ciclos en el grafo de URLs

Las aplicaciones web modernas frecuentemente tienen páginas que se enlazan entre sí de forma circular ($A \rightarrow B \rightarrow C \rightarrow A$). Sin un mecanismo de detección de ciclos, el crawler podría iterar indefinidamente. El módulo `WebCrawler` usa un set de URLs visitadas (`visited_urls`) para detectar ciclos: antes de procesar una URL, verifica si ya está en el conjunto de visitadas y la omite si es así. El uso de un set en lugar de una lista garantiza que la comprobación de pertenencia sea $O(1)$ en lugar de $O(n)$, lo que es importante cuando el número de URLs visitadas es grande.

El límite de `max_pages` funciona como salvaguarda adicional contra ciclos o sitios excepcionalmente grandes: si el número de páginas visitadas alcanza este límite, el crawler deja de seguir nuevos enlaces aunque queden URLs sin visitar. En el contexto de una auditoría de seguridad, 50 páginas son generalmente suficientes para mapear la estructura de una aplicación web de tamaño medio.

N.3 Análisis Profundo del SSL/TLS Analyzer

N.3.1 La cadena de certificados X.509

Un certificado SSL/TLS no existe en aislamiento: forma parte de una cadena de confianza (certificate chain) que va desde el certificado del servidor hasta un certificado raíz (root certificate) que está preinstalado en el almacén de certificados del sistema operativo o del navegador. La cadena típica tiene tres eslabones: el certificado del servidor (end-entity certificate), firmado por una CA intermedia (intermediate CA), que a su vez está firmada por la CA raíz (root CA). La confianza se propaga hacia abajo mediante las firmas digitales: si confías en la raíz y la raíz firma la intermedia, y la intermedia firma el certificado del servidor, entonces confías en el certificado del servidor.

El módulo SSL/TLS Analyzer extrae el certificado del servidor y su cadena mediante la API ssl de Python. Los campos de mayor interés para la seguridad son: el Common Name o Subject Alternative Names (para verificar que el certificado es válido para el dominio que se está auditando), el Issuer (para identificar la CA que emitió el certificado), la fecha de expiración (para detectar certificados expirados o próximos a expirar) y el algoritmo de firma (SHA-1 en firmas de certificados está deprecado desde 2017; todos los certificados deben usar SHA-256 o superior).

N.3.2 Certificate Pinning y sus implicaciones

Certificate Pinning es una técnica de seguridad donde una aplicación cliente define explícitamente qué certificados o claves públicas son válidos para comunicarse con un servidor concreto, ignorando la cadena de confianza estándar del sistema operativo. Esta técnica previene ataques de man-in-the-middle incluso si el atacante dispone de un certificado firmado por una CA de confianza. Desde la perspectiva del módulo SSL/TLS Analyzer, Certificate Pinning no afecta al análisis del certificado en sí (que se obtiene directamente del servidor), pero es relevante mencionarlo como contexto cuando se auditan aplicaciones móviles o de escritorio que se comunican con la infraestructura objetivo.

N.4 Análisis Profundo del Vulnerability Scanner

N.4.1 Falsos Positivos y Falsos Negativos

Todo escáner de vulnerabilidades enfrenta el balance entre falsos positivos (reportar una vulnerabilidad que no existe) y falsos negativos (no detectar una vulnerabilidad que sí existe). Los falsos positivos generan trabajo innecesario para el equipo de remediación y

erosionan la confianza en la herramienta. Los falsos negativos son más peligrosos porque dan una falsa sensación de seguridad.

El Vulnerability Scanner de GhostTrack está diseñado para minimizar los falsos positivos a costa de aceptar más falsos negativos: solo reporta una vulnerabilidad si hay evidencia directa (el servicio expone información de versión que coincide exactamente con un CVE conocido o el servicio responde de forma que confirma la misconfiguration). Este enfoque conservador es adecuado en el contexto de un TFG donde los reportes son revisados por profesionales que pueden distinguir entre hallazgos confirmados y sugerencias; en una herramienta de producción se añadiría un nivel de clasificación adicional que distinguiera entre 'confirmado', 'probable' y 'posible'.

N.4.2 Extensión de la Base de Datos de CVEs

La base de datos CVE_DATABASE del módulo puede extenderse añadiendo entradas al diccionario directamente en el código. La estructura de cada entrada es:

```
CVE_DATABASE = {
  # Formato: 'Nombre_Software/version_afectada': {datos CVE}
  'Apache/2.4.49': {
    'cve': 'CVE-2021-41773',
    'severity': 'CRITICAL',
    'description': 'Path Traversal y RCE en Apache 2.4.49',
    'cvss': 9.8,
    'references': ['https://nvd.nist.gov/vuln/detail/CVE-2021-41773']
  },
  'Log4j/2.0-beta9': {
    'cve': 'CVE-2021-44228',
    'severity': 'CRITICAL',
    'description': 'Log4Shell - JNDI injection RCE',
    'cvss': 10.0,
    'references': ['https://nvd.nist.gov/vuln/detail/CVE-2021-44228']
  },
  # ... más entradas
}
```

Apéndice O — Historia y Evolución del Pentesting

Comprender la historia del pentesting y las herramientas que lo definen es fundamental para contextualizar el desarrollo de GhostTrack y entender por qué el sector ha evolucionado hacia los estándares y herramientas actuales.

O.1 La Era de los Tiger Teams (1960-1990)

Los orígenes del pentesting se remontan a los años 60, cuando el Departamento de Defensa de Estados Unidos comenzó a preocuparse por la seguridad de los primeros sistemas de computación compartida. La solución fue crear equipos de personas encargadas de intentar comprometer la seguridad de los propios sistemas del gobierno: los tiger teams. El objetivo era identificar vulnerabilidades antes de que pudieran ser explotadas por adversarios reales, especialmente en el contexto de la Guerra Fría donde la amenaza de espionaje electrónico era una preocupación genuina (Anderson, 1972).

Willis Ware del RAND Corporation publicó en 1967 el documento Security and Privacy in Computer Systems, considerado uno de los primeros análisis formales de la seguridad en sistemas informáticos. En él se describían técnicas que hoy reconoceríamos como reconocimiento, análisis de vulnerabilidades y explotación. El informe de James Anderson de 1972 Computer Security Technology Planning Study sistematizó estas técnicas y estableció el marco conceptual del pentesting moderno.

O.2 La Era de Internet (1990-2000)

La expansión de Internet en los años 90 transformó radicalmente el panorama de la seguridad informática. Hasta entonces, los sistemas vulnerables requerían acceso físico o una conexión telefónica directa para ser comprometidos; con Internet, millones de sistemas quedaron expuestos a atacantes de todo el mundo. Esta nueva realidad impulsó el desarrollo de las primeras herramientas automatizadas de evaluación de seguridad.

SATAN (Security Administrator Tool for Analyzing Networks), publicada en 1995 por Dan Farmer y Wietse Venema, fue la primera herramienta pública de evaluación de vulnerabilidades de red. Su lanzamiento generó una controversia enorme: muchos argumentaban que publicar una herramienta capaz de descubrir vulnerabilidades automáticamente facilitaría los ataques maliciosos. Los creadores respondieron con el argumento que sigue siendo válido hoy: la seguridad por oscuridad no funciona, y poner esta capacidad en manos de los defensores es más beneficioso que restrictivo (Farmer y Venema, 1995).

Nessus, creado por Renaud Deraison en 1998, introdujo el concepto de plugins: módulos de verificación independientes para cada vulnerabilidad específica, que podían actualizarse sin modificar el motor principal del escáner. Este modelo de arquitectura influyó directamente en el diseño modular de GhostTrack. Nmap, creado por Gordon Lyon (Fyodor) en 1997, revolucionó el escaneo de puertos con sus técnicas de SYN Scan, detección de OS y scripting NSE, y sigue siendo la referencia absoluta en el campo casi tres décadas después.

O.3 La Era de la Profesionalización (2000-2015)

La primera década del siglo XXI vio la transformación del pentesting de una actividad de nicho practicada principalmente por hackers aficionados a una profesión establecida con certificaciones reconocidas, marcos metodológicos y un mercado laboral propio. La creación de OWASP en 2001, EC-Council en 2001 (con su certificación CEH), y Offensive Security en 2006 (con su OSCP) marcaron hitos en esta profesionalización.

Metasploit Framework, publicado por H.D. Moore en 2003, democratizó la explotación de vulnerabilidades al proporcionar un catálogo centralizado de exploits con una interfaz unificada. Su adquisición por Rapid7 en 2009 y su evolución hacia Metasploit Pro marcaron el momento en que el pentesting automatizado se convirtió en un producto comercial maduro. La publicación de EternalBlue en 2017 y su posterior uso por WannaCry y NotPetya demostró que las técnicas avanzadas de pentesting, una vez filtradas, podían causar daños a escala global.

O.4 La Era del Red Team y la Automatización (2015-presente)

Los últimos diez años han visto la evolución del pentesting tradicional hacia ejercicios de Red Team más sofisticados, el surgimiento de los programas de bug bounty como modelo de evaluación continua, y la integración de la seguridad en los ciclos de desarrollo mediante DevSecOps. Herramientas como Cobalt Strike, Empire y BloodHound han elevado el nivel de sofisticación de los ejercicios Red Team, mientras que plataformas como HackerOne y Bugcrowd han creado un mercado para la divulgación responsable de vulnerabilidades.

GhostTrack se desarrolla en este contexto: un framework que integra las capacidades de múltiples herramientas especializadas en una plataforma cohesionada, siguiendo la tendencia del sector hacia la automatización y la integración con pipelines de CI/CD. La evolución natural de GhostTrack en versiones futuras va en la dirección de incorporar capacidades de IA para la correlación automática de hallazgos, siguiendo la misma evolución

que han experimentado plataformas comerciales como Darktrace, Vectra y CrowdStrike Falcon.

O.5 Cronología de Hitos del Pentesting

Tabla 10 *Cronología de hitos relevantes en la historia del pentesting* Nota. Elaboración propia basada en fuentes bibliográficas citadas.

Año	Hito	Relevancia para GhostTrack
1967	Informe RAND sobre seguridad en sistemas compartidos (Ware)	Primeros conceptos de evaluación de seguridad
1972	Computer Security Technology Planning Study (Anderson)	Primera sistematización del pentesting
1995	SATAN publicado (Farmer y Venema)	Primera herramienta pública de escaneo de vulnerabilidades
1997	Nmap publicado (Lyon)	Referencia de escaneo de puertos; comparativa directa con Port Scanner de GhostTrack
1998	Nessus publicado (Deraison)	Arquitectura de plugins que inspiró la modularidad de GhostTrack
2001	OWASP fundada	Estándares de evaluación web que implementan los módulos de GhostTrack
2003	Metasploit publicado (Moore)	Framework de explotación que complementa a GhostTrack
2006	OSCP lanzado (Offensive Security)	Certificación de referencia en pentesting ofensivo
2012	PTES publicado	Estándar de proceso que estructura GhostTrack
2013	Certificate Transparency (Google)	Fuente de datos que usa el módulo Subdomain Finder
2015	ATT&CK publicado (MITRE)	Framework que mapea los módulos Red Team de GhostTrack
2016	BloodHound publicado (Rohan et al.)	Replicado parcialmente por BloodHoundCollector
2017	EternalBlue / WannaCry	Demostró el impacto real de SMBv1 (detectado por GhostTrack)
2018	RGPD en vigor	Legislación que limita el tratamiento de datos en auditorías
2021	Log4Shell (CVE-2021-44228)	Ejemplo de CVE crítico en la BD del Vulnerability Scanner
2024	NIST CSF 2.0	Versión más reciente del marco de referencia citado en GhostTrack

Apéndice P — Plantillas de Reportes de Auditoría

Este apéndice proporciona plantillas de los elementos narrativos de un informe de auditoría completo que complementan los reportes automáticos generados por GhostTrack. Los reportes automáticos del framework cubren los datos técnicos de forma exhaustiva, pero un informe profesional completo requiere también una sección ejecutiva, una narrativa del ataque y recomendaciones priorizadas que deben ser redactadas manualmente por el auditor.

P.1 Resumen Ejecutivo

El resumen ejecutivo está dirigido a la dirección de la organización y no debe contener terminología técnica. Debe responder a tres preguntas: ¿Cuán segura es la organización? ¿Qué podría hacer un atacante? ¿Qué hay que hacer para mejorar? La siguiente plantilla sirve como punto de partida:

RESUMEN EJECUTIVO

Durante el período [fechas], [nombre del auditor/empresa] realizó una evaluación de seguridad técnica de los sistemas de [nombre del cliente] de acuerdo con el alcance definido en el contrato de [fecha].

HALLAZGOS PRINCIPALES

Se identificaron [N] vulnerabilidades de seguridad:

- [X] de severidad CRÍTICA: requieren acción inmediata
- [Y] de severidad ALTA: deben remediarse en menos de 30 días
- [Z] de severidad MEDIA: deben remediarse en menos de 90 días
- [W] de severidad BAJA: mejoras recomendadas sin urgencia

RIESGO GLOBAL: [CRÍTICO / ALTO / MEDIO / BAJO]

El hallazgo más crítico identificado es [descripción en lenguaje no técnico]. Un atacante que explotara esta vulnerabilidad podría [impacto en negocio].

RECOMENDACIONES PRIORITARIAS

1. [Acción más urgente – impacto / esfuerzo]
2. [Segunda acción más urgente]
3. [Tercera acción más urgente]

P.2 Narrativa de Ataque

La narrativa de ataque describe de forma cronológica cómo el equipo auditor progresó desde el acceso inicial hasta el objetivo final del ejercicio, encadenando los hallazgos individuales en un escenario coherente de amenaza. Esta sección es especialmente valiosa porque ayuda al cliente a entender que las vulnerabilidades no son problemas aislados sino piezas que un atacante real encadenaría:

NARRATIVA DE ATAQUE

Fase 1 – Reconocimiento (tiempo: X horas)

El equipo comenzó con reconocimiento pasivo sobre el dominio objetivo.com. Mediante enumeración DNS se descubrió el subdominio staging.objetivo.com [hallazgo que desencadena el siguiente paso]

Fase 2 – Acceso inicial (tiempo: X horas)

El subdominio staging.objetivo.com expone una versión de la aplicación con credenciales de prueba admin/admin. Con estas credenciales se obtuvo acceso al panel de administración [...]

Fase 3 – Escalada de privilegios (tiempo: X horas)

Desde el panel de administración se explotó la vulnerabilidad SQLi en el parámetro 'user_id' para leer ficheros del servidor [...]

Fase 4 – Objetivo conseguido (tiempo total: X días)

Se demostró la capacidad de acceder a [objetivo específico del ejercicio]. El impacto potencial sería [descripción del daño a negocio].

P.3 Ficha de Vulnerabilidad

Cada vulnerabilidad de severidad ALTA o CRÍTICA merece su propia ficha de detalle que documente todos los aspectos técnicos necesarios para su reproducción y remediación:

FICHA DE VULNERABILIDAD

ID: VULN-001
 Título: SQL Injection en endpoint /search
 Severidad: CRITICAL
 CVSS Score: 9.8 (AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H)
 Fecha: 2026-04-15

DESCRIPCIÓN

El parámetro 'q' del endpoint /search no está correctamente sanitizado y permite la inyección de código SQL arbitrario.

REPRODUCCIÓN

URL: `https://objetivo.com/search?q=[payload]`
 Payload utilizado: `' OR '1'='1' --`
 Respuesta del servidor: [captura de pantalla o log]

EVIDENCIA

[Screenshot o log que demuestra la vulnerabilidad]

IMPACTO

Un atacante puede leer, modificar y eliminar datos de la base de datos, y potencialmente ejecutar comandos en el servidor si `xp_cmdshell` está habilitado (SQL Server) o mediante `outfile` (MySQL).

REMEDIACIÓN

1. Reemplazar consultas dinámicas por prepared statements o ORMs.
2. Implementar validación de entrada con whitelist de caracteres permitidos.
3. Configurar un WAF con reglas para SQL Injection.
4. Habilitar el usuario de base de datos con los mínimos privilegios necesarios.

REFERENCIAS

- OWASP: https://owasp.org/www-community/attacks/SQL_Injection
- CWE-89: Improper Neutralization of Special Elements in SQL Command

P.4 Plan de Remediación

El plan de remediación organiza todos los hallazgos en una lista priorizada con estimaciones de esfuerzo y plazos recomendados. Esta sección transforma el informe técnico en un plan de acción concreto para el equipo de operaciones:

PLAN DE REMEDIACIÓN PRIORIZADO

ACCIONES INMEDIATAS (< 72 horas)

1. [VULN-003] Aplicar parche MS17-010 en servidor SQL01
 Esfuerzo: 2h | Responsable: Equipo de sistemas
 Impacto si no se remedia: RCE sin autenticación
2. [VULN-007] Deshabilitar SSLv3 en balanceadores de carga
 Esfuerzo: 1h | Responsable: Equipo de red
 Impacto si no se remedia: Descifrado de tráfico (POODLE)

ACCIONES A CORTO PLAZO (< 30 días)

3. [VULN-001] Reescribir queries SQL como prepared statements
 Esfuerzo: 40h | Responsable: Equipo de desarrollo
4. [VULN-005] Restringir AXFR en servidor DNS ns2

Esfuerzo: 1h | Responsable: Administrador DNS

ACCIONES A MEDIO PLAZO (< 90 días)

5. [VULN-012] Implementar DMARC con p=quarantine

Esfuerzo: 8h | Responsable: Administrador de correo

MEJORAS RECOMENDADAS (sin urgencia)

6. Implementar política de rotación de contraseñas para cuentas de servicio
7. Desplegar monitorización de logs centralizada (SIEM)

Apéndice Q — Recursos y Referencias Adicionales

Este apéndice recopila recursos de aprendizaje, comunidades profesionales, conferencias y plataformas de entrenamiento que complementan los conocimientos adquiridos a través del desarrollo y estudio de GhostTrack.

Q.1 Libros de Referencia

Los siguientes libros son lecturas recomendadas para profundizar en los temas cubiertos por GhostTrack:

- Seitz, J. (2021). Black Hat Python (2ª ed.). No Starch Press. El libro de referencia para programación de herramientas de seguridad en Python. Cubre sockets, scapy, exploits web y herramientas de red desde cero.
- Lyon, G. F. (2009). Nmap Network Scanning. Nmap Project. La referencia definitiva sobre técnicas de escaneo de puertos, detección de OS y scripting NSE. Comprensión profunda de los fundamentos del Port Scanner de GhostTrack.
- Engebretson, P. (2013). The Basics of Hacking and Penetration Testing (2ª ed.). Syngress. Introducción práctica al pentesting con herramientas estándar del sector.
- Kennedy, D. et al. (2011). Metasploit: The Penetration Tester's Guide. No Starch Press. El manual oficial de Metasploit Framework, herramienta complementaria a GhostTrack para la fase de explotación.
- Stuttard, D. y Pinto, M. (2011). The Web Application Hacker's Handbook (2ª ed.). Wiley. Referencia completa para la evaluación de seguridad web, cubriendo en profundidad las técnicas implementadas en los módulos web de GhostTrack.
- Mitnick, K. D. y Simon, W. L. (2002). The Art of Deception. Wiley. Obra fundamental sobre ingeniería social, relevante para el módulo SocialEngineeringToolkit.
- Scambray, J. et al. (2012). Hacking Exposed 7. McGraw-Hill. Enciclopedia de técnicas de hacking y sus contramedidas, organizada por vector de ataque.

Q.2 Comunidades y Foros Profesionales

- OWASP Community (<https://owasp.org>): La organización más activa en seguridad web. Sus guías, herramientas y eventos son referencia obligada.
- Hack The Box Community (<https://www.hackthebox.com>): Plataforma de entrenamiento con una comunidad muy activa que comparte writeups y técnicas.

- `r/netsec` y `r/AskNetsec` (Reddit): Subreddits con discusiones técnicas de alto nivel sobre seguridad ofensiva y defensiva.
- SpecterOps Blog (<https://posts.specterops.io>): Blog del equipo detrás de BloodHound y otras herramientas de Red Team avanzado.
- Harmj0y Blog (<https://harmj0y.medium.com>): Blog de Will Schroeder, uno de los investigadores más influyentes en Active Directory y Kerberos.
- Pentest.blog y Pentesterlab Blog: Tutoriales prácticos sobre técnicas específicas de pentesting.

Q.3 Conferencias de Referencia

- DEF CON (Las Vegas, anual): La conferencia de hackers más grande del mundo. Presentaciones de investigación de vanguardia, talleres prácticos y competiciones CTF. Muchas de las técnicas implementadas en GhostTrack fueron presentadas por primera vez en DEF CON (Kerberoasting en DEF CON 22, BloodHound en DEF CON 24).
- Black Hat USA/Europe/Asia (varias, anuales): Conferencia de seguridad con el nivel técnico más alto del sector. Los Briefings son presentaciones de 50 minutos de investigación original.
- RootedCON (Madrid, anual): La conferencia de seguridad más importante del ámbito hispanohablante. Especialmente relevante para entender el marco legal español en ciberseguridad.
- Hack In The Box (Ámsterdam y Kuala Lumpur, anuales): Conferencia de seguridad ofensiva con enfoque técnico.
- No cON Name (Barcelona, anual): Conferencia de seguridad con fuerte presencia de la comunidad española.

Q.4 Certificaciones Profesionales Relevantes

Tabla 11 *Certificaciones profesionales relevantes para ciberseguridad ofensiva* Nota. Elaboración propia.
Dificultad: escala subjetiva de 1 (introductorio) a 5 (experto).

Certificación	Organismo	Enfoque	Dificultad	Relevancia para GhostTrack
CEH	EC-Council	Hacking ético general	2/5	Media — teórico, cubre conceptos de GhostTrack
OSCP	OffSec	Pentesting práctico	4/5	Alta — examen de 24h, uso real de técnicas de GhostTrack

Certificación	Organismo	Enfoque	Dificultad	Relevancia para GhostTrack
GPEN	GIAC	Pentesting de red	3/5	Alta — cubre módulos de red de GhostTrack
GWAPT	GIAC	Pentesting web	3/5	Alta — cubre módulos web de GhostTrack
CRTP	Altered Security	Active Directory RT	4/5	Muy alta — cubre módulos Red Team AD de GhostTrack
CRTE	Altered Security	Enterprise Red Team	5/5	Muy alta — ejercicios de entornos AD complejos
CRTL	Altered Security	Red Team Leader	5/5	Alta — gestión de ejercicios Red Team completos
eJPT	eLearnSecurity	Junior Pentester	1/5	Media — ideal como primer paso antes de GhostTrack

Para alumnos que completan este TFG, el itinerario recomendado de certificaciones es: eJPT como primera certificación (asequible y práctica), seguido del OSCP una vez adquirida experiencia práctica con laboratorios como HackTheBox, y el CRTP cuando se quiera profundizar específicamente en Active Directory, que es donde más valor aporta el bloque Red Team de GhostTrack.

Apéndice R — Protocolo de Gestión de Incidentes Durante Auditoría

Durante la ejecución de un ejercicio de pentesting pueden ocurrir situaciones imprevistas que requieren una respuesta rápida y coordinada entre el equipo auditor y el cliente. Este apéndice define los procedimientos a seguir en los escenarios más comunes.

R.1 Tipos de Incidentes Durante una Auditoría

R.1.1 Interrupción de servicio no planificada

Una de las situaciones más delicadas es cuando una prueba de penetración causa una interrupción del servicio no planificada. Esto puede ocurrir cuando una petición HTTP malformada o una carga de datos inesperadamente grande desborda un buffer en la aplicación objetivo, cuando un escaneo de puertos agresivo satura la capacidad de un servidor con hardware limitado, cuando una prueba de inyección SQL desencadena una operación de base de datos con un plan de ejecución ineficiente que bloquea otras transacciones, o cuando la inyección de un payload activa un mecanismo de autodefensa que bloquea la IP del auditor o reinicia un servicio.

El protocolo ante una interrupción de servicio es: dejar de inmediato todas las operaciones activas sobre el sistema afectado; notificar al contacto técnico del cliente con los detalles de la última acción ejecutada antes de la interrupción; documentar el timestamp exacto de la interrupción y las acciones previas en los logs de GhostTrack; esperar instrucciones del cliente antes de reanudar las pruebas; y colaborar con el equipo técnico del cliente para restaurar el servicio si es necesario.

R.1.2 Descubrimiento de un compromiso previo

Puede ocurrir que durante una auditoría el equipo descubra evidencias de que el sistema ya fue comprometido por un tercero antes del inicio del ejercicio. Las evidencias típicas incluyen: ficheros maliciosos en directorios de la aplicación web, usuarios con privilegios elevados no autorizados en el directorio Active Directory, claves SSH no autorizadas en el fichero `authorized_keys`, procesos en ejecución con nombres ofuscados, tráfico de red hacia IPs desconocidas en el análisis del Network Sniffer, o logs de acceso con patrones de actividad sospechosa.

El protocolo ante un posible compromiso previo es: documentar inmediatamente las evidencias encontradas con screenshots y capturas de log sin modificar ningún dato; notificar urgentemente al cliente por el canal de comunicación de emergencia definido en el contrato; no continuar las pruebas sobre los sistemas afectados hasta recibir instrucciones del cliente;

y no eliminar ni alterar ninguna evidencia del posible compromiso, ya que podría ser necesaria para una investigación forense posterior.

R.1.3 Descubrimiento de datos personales

Durante una auditoría es frecuente encontrar datos personales de clientes o empleados en bases de datos, ficheros de log o configuraciones del servidor. El RGPD establece obligaciones específicas para el tratamiento de estos datos que el equipo auditor debe respetar: minimizar el acceso a datos personales reales (preferir usar datos de prueba cuando sea posible), no copiar datos personales más allá de lo mínimo necesario para documentar el hallazgo, eliminar cualquier copia de datos personales al finalizar el ejercicio, y notificar al cliente si se accede a datos personales que no debería haber sido posible acceder según el alcance del ejercicio.

R.2 Árbol de Decisión para Incidentes

El siguiente árbol de decisión guía al auditor en la gestión de los incidentes más comunes:

```

¿Ha ocurrido un incidente durante la auditoría?
|
├── ¿Es una interrupción de servicio?
|   ├── SÍ → Parar pruebas + Notificar cliente INMEDIATAMENTE
|   |       + Documentar timestamp y última acción
|   └── NO → Continuar árbol
|
├── ¿Es un compromiso previo detectado?
|   ├── SÍ → Documentar evidencias sin modificar
|   |       + Notificar urgente por canal de emergencia
|   |       + Pausar pruebas en sistemas afectados
|   └── NO → Continuar árbol
|
├── ¿Se accedió a datos personales reales?
|   ├── SÍ → Minimizar copia → Notificar a cliente
|   |       + Eliminar copias al finalizar
|   └── NO → Continuar árbol
|
└── ¿Hay ambigüedad sobre si la acción está dentro del alcance?
    ├── SÍ → Consultar con cliente ANTES de ejecutar
    └── NO → Continuar con la prueba planificada
  
```

R.3 Documentación de Incidentes

Todos los incidentes ocurridos durante la auditoría deben documentarse en un registro de incidentes separado del reporte principal. Este registro debe incluir como mínimo: timestamp del incidente, descripción de lo ocurrido, causa identificada, impacto observado, acciones tomadas para mitigar el impacto y estado actual (resuelto / en seguimiento / pendiente de respuesta del cliente). Este registro forma parte del entregable final de la auditoría y es relevante desde el punto de vista legal para demostrar que el equipo auditor actuó de forma diligente ante situaciones imprevistas.

Apéndice S — Active Directory: Conceptos Técnicos Avanzados

Este apéndice profundiza en los conceptos técnicos de Active Directory que son fundamentales para entender el funcionamiento de los módulos Red Team de GhostTrack. El conocimiento profundo de estos mecanismos es lo que diferencia una evaluación de seguridad superficial de un ejercicio Red Team que demuestra el impacto real de las vulnerabilidades encontradas.

S.1 Estructura Interna del Protocolo Kerberos

S.1.1 *El Key Distribution Center (KDC)*

El KDC es el corazón del sistema de autenticación Kerberos en Active Directory. Corre como un servicio en cada controlador de dominio y tiene dos componentes principales: el Authentication Service (AS), que maneja las solicitudes iniciales de autenticación (mensajes AS-REQ y AS-REP), y el Ticket Granting Service (TGS), que emite tickets para servicios específicos (mensajes TGS-REQ y TGS-REP). La cuenta especial `krbtgt`, creada automáticamente al promover un servidor a controlador de dominio, es la cuenta bajo la que corre el KDC. Su hash de contraseña es la clave maestra que cifra todos los TGTs del dominio.

La cuenta `krbtgt` tiene una característica única: su contraseña solo puede cambiarse manualmente por un administrador de dominio, y Microsoft recomienda cambiarla dos veces consecutivas para invalidar cualquier TGT existente. En muchas organizaciones, la contraseña de `krbtgt` nunca ha sido cambiada desde la creación del dominio, lo que significa que cualquier atacante que consiga el hash de `krbtgt` puede crear TGTs arbitrarios (Golden Tickets) que son válidos indefinidamente.

S.1.2 *Estructura de un Ticket Kerberos*

Un ticket Kerberos es una estructura de datos cifrada que contiene la identidad del usuario autenticado, los grupos a los que pertenece, la marca de tiempo de emisión, el tiempo de vida del ticket y el servicio para el que es válido. Los TGTs están cifrados con el hash de la cuenta `krbtgt`; los TGS están cifrados con el hash de contraseña de la cuenta de servicio. Esta arquitectura permite al KDC delegar la verificación de autenticidad a los servicios: cuando un servidor SMB recibe un TGS, simplemente lo descifra con su propio hash y verifica que el contenido es coherente, sin necesidad de consultar al KDC para cada solicitud.

La implicación para la seguridad es que el hash de contraseña de la cuenta de servicio es equivalente a la contraseña en términos de acceso: quien tenga el hash puede generar un TGS válido (Pass-the-Hash) o puede intentar descifrar el hash para obtener la contraseña en texto claro (Kerberoasting). Esta es la razón por la que las cuentas de servicio con contraseñas débiles son tan peligrosas y por qué las gMSA (group Managed Service Accounts) son la solución recomendada: sus contraseñas son cadenas aleatorias de 240 caracteres que rotan automáticamente cada 30 días, haciendo el cracking offline completamente inviable.

S.2 ACLs de Active Directory y Abuso de Permisos

S.2.1 El modelo de seguridad de objetos AD

Cada objeto de Active Directory (usuario, grupo, equipo, GPO, OU...) tiene asociada una lista de control de acceso discrecional (DACL) que define qué identidades tienen qué permisos sobre ese objeto. Los permisos relevantes para el equipo Red Team son: GenericAll (control total, equivalente a ser propietario del objeto), GenericWrite (puede modificar cualquier atributo del objeto), WriteOwner (puede cambiar el propietario del objeto, lo que otorga GenericAll al nuevo propietario), WriteDACL (puede modificar la DACL del objeto, lo que permite añadir permisos arbitrarios), AllExtendedRights (todos los derechos extendidos, incluyendo resetear contraseñas y solicitar replicación DCSync), y ForceChangePassword (puede cambiar la contraseña del usuario sin conocer la anterior).

La presencia de cualquiera de estos permisos sobre un objeto de alto privilegio (como un Domain Admin o el propio objeto del dominio) para un usuario o grupo de bajo privilegio constituye una ruta de escalada de privilegios. BloodHound modela estas rutas y puede encontrar automáticamente las cadenas de permisos más cortas que conectan una cuenta comprometida con el control total del dominio.

S.2.2 AdminSDHolder y el proceso SDProp

AdminSDHolder es un objeto especial de Active Directory que existe en el contenedor System de cada dominio. Su DACL sirve como plantilla de seguridad para todos los objetos miembros de grupos de alto privilegio (Domain Admins, Enterprise Admins, Schema Admins, Administrators, etc.). Cada hora (configurable), un proceso del sistema llamado SDProp compara la DACL de cada miembro de estos grupos con la DACL del AdminSDHolder y, si hay diferencias, reemplaza la DACL del objeto con la del AdminSDHolder.

La implicación para la seguridad es bidireccional: los objetos con AdminCount=1 (que indica que pertenecen o han pertenecido a un grupo de alto privilegio) tienen DACLs más

restrictivas que el resto de objetos del dominio, lo que es protector. Pero si un atacante consigue modificar la DACL del propio AdminSDHolder, esos cambios se propagarán automáticamente a todos los objetos de alto privilegio en la próxima ejecución de SDProp. Esta técnica, conocida como AdminSDHolder backdoor, es una forma de persistencia especialmente sigilosa porque el backdoor se recrea automáticamente aunque sea eliminado.

S.3 Delegación Kerberos: Constrained y Unconstrained

S.3.1 Delegación no restringida (*Unconstrained Delegation*)

La delegación Kerberos permite a un servicio actuar en nombre de un usuario para acceder a otros servicios. La delegación no restringida (*Unconstrained Delegation*) es la variante más antigua y más peligrosa: cuando un usuario se autentica contra un servicio con delegación no restringida, el KDC incluye en el TGS una copia completa del TGT del usuario. El servicio puede entonces usar ese TGT para autenticarse en cualquier otro servicio del dominio en nombre del usuario, sin restricciones.

La implicación para la seguridad es crítica: si el atacante compromete un servidor con *Unconstrained Delegation* habilitado (atributo *TrustedForDelegation* en el objeto *computer* o *user* de AD), puede capturar los TGTs de todos los usuarios que se autenticuen contra ese servidor. En particular, si consigue forzar al controlador de dominio a autenticarse contra el servidor comprometido (mediante técnicas como *PrinterBug* o *PetitPotam*), captura el TGT del DC y puede usarlo para solicitar la replicación del directorio completo (*DCSync*), extrayendo todos los hashes del dominio.

S.3.2 Delegación restringida (*Constrained Delegation*)

La delegación restringida (*Constrained Delegation*) es una versión más segura que limita a qué servicios puede delegar el servicio intermediario. Está configurada mediante el atributo *msDS-AllowedToDelegateTo* en el objeto de AD, que contiene una lista de SPNs a los que se permite la delegación. Aunque es más segura que la delegación no restringida, también puede ser abusada: un atacante que compromete una cuenta con delegación restringida a un servicio específico puede impersonar a cualquier usuario (incluyendo *Domain Admins*) ante ese servicio, independientemente de si el usuario real tiene acceso a él.

S.3.3 Resource-Based Constrained Delegation (*RBCD*)

RBCD es la variante más moderna y flexible, introducida en Windows Server 2012. En lugar de configurar la delegación en el objeto que delega (el servicio intermediario), se configura en el objeto al que se delega (el servicio de destino). El atributo

msDS-AllowedToActOnBehalfOfOtherIdentity en el objeto de destino contiene un descriptor de seguridad que especifica qué identidades pueden impersonar usuarios ante ese servicio. La importancia de RBCD para el Red Team es que cualquier usuario con el permiso GenericWrite sobre un objeto computer puede configurar RBCD arbitrariamente, creando rutas de escalada que BloodHound detecta y que el módulo NTLMRelayAttacker de GhostTrack puede explotar para obtener acceso como administrador en cualquier servidor del dominio.

S.4 DCSync: Replicación del Directorio como Técnica de Ataque

DCSync es una técnica post-explotación que simula el comportamiento de un controlador de dominio secundario sincronizando su base de datos con el DC primario. Los DCs secundarios tienen permisos para solicitar la replicación de objetos específicos del directorio, incluyendo los hashes de contraseñas. Si un atacante consigue una cuenta con el permiso AllExtendedRights o la combinación de DS-Replication-Get-Changes y DS-Replication-Get-Changes-All sobre el objeto del dominio, puede solicitar la replicación de cualquier objeto, incluyendo el hash de la cuenta krbtgt y de todos los usuarios del dominio.

La detección de DCSync en un SIEM se basa en monitorizar el Event ID 4662 (An operation was performed on an object) con el tipo de acceso %1131f6ad-9c07-11d1-f79f-00c04fc2dcd2 (DS-Replication-Get-Changes-All). Cualquier solicitud de este tipo desde una cuenta que no sea un DC o una cuenta de administración conocida es un indicador de compromiso de alta fidelidad. El módulo BloodHoundCollector de GhostTrack identifica qué cuentas tienen estos permisos de replicación durante la fase de enumeración.

Apéndice T — Seguridad en el Ciclo de Desarrollo: DevSecOps

La integración de la seguridad en los ciclos de desarrollo de software (DevSecOps) es una tendencia creciente en la industria que busca detectar y remediar vulnerabilidades lo antes posible en el ciclo de vida del software, cuando el coste de corrección es menor. Este apéndice describe cómo GhostTrack encaja en esta filosofía y cómo puede integrarse en los procesos de desarrollo modernos.

T.1 El Modelo Shift-Left de Seguridad

El concepto Shift-Left hace referencia al desplazamiento de las actividades de seguridad hacia la izquierda en la línea temporal del ciclo de vida del desarrollo software, es decir, realizarlas más temprano en el proceso. En un modelo tradicional, las pruebas de seguridad se realizaban solo al final del ciclo de desarrollo, antes del despliegue en producción. Esto tiene dos problemas: el coste de corrección de vulnerabilidades encontradas tardíamente es mucho mayor (requiere modificar código ya integrado y probado), y el tiempo entre el desarrollo de código vulnerable y su corrección puede ser de semanas o meses.

En un modelo DevSecOps, las pruebas de seguridad se integran en cada fase del ciclo: análisis de seguridad de dependencias en la fase de desarrollo (SCA, Software Composition Analysis), análisis estático de código durante el build (SAST, Static Application Security Testing), pruebas de seguridad automatizadas en el pipeline CI/CD (DAST, Dynamic Application Security Testing) y pruebas de penetración manuales en ciclos regulares. GhostTrack contribuye principalmente en la fase DAST del pipeline.

T.2 SAST vs DAST vs IAST

Las tres modalidades de pruebas de seguridad automatizadas tienen características complementarias:

- SAST (Static Application Security Testing): Analiza el código fuente sin ejecutarlo, buscando patrones de código inseguro (consultas SQL sin parametrizar, uso de funciones deprecadas, secretos hardcodeados). Herramientas como SonarQube, Checkmarx y Semgrep implementan SAST. Sus ventajas son que puede integrarse muy temprano en el ciclo (incluso en el IDE del desarrollador), cubre el 100% del código y no requiere un entorno de ejecución. Su limitación es que genera muchos falsos positivos porque no conoce el contexto de ejecución.

- DAST (Dynamic Application Security Testing): Prueba la aplicación en ejecución enviando peticiones maliciosas y analizando las respuestas, exactamente como hacen los módulos web de GhostTrack. Sus ventajas son la baja tasa de falsos positivos (cada hallazgo está respaldado por una evidencia de explotación real) y la capacidad de detectar vulnerabilidades de configuración que no existen en el código fuente (como una cabecera HTTP de seguridad ausente). Su limitación es que solo puede probar la superficie expuesta de la aplicación y requiere un entorno de ejecución.

- IAST (Interactive Application Security Testing): Combina lo mejor de SAST y DAST mediante la instrumentación del runtime de la aplicación. Un agente IAST monitoriza la ejecución de la aplicación desde dentro mientras los tests funcionales normales se ejecutan, detectando vulnerabilidades de seguridad en tiempo real con acceso al stack completo de llamadas. Esta modalidad tiene la menor tasa de falsos positivos y proporciona información de localización exacta en el código de cada vulnerabilidad.

T.3 Integración de GhostTrack en DevSecOps

GhostTrack se posiciona como herramienta DAST en el pipeline de CI/CD, complementando las herramientas SAST que analizan el código fuente. La integración más efectiva sigue este patrón:

13. Cuando el código pasa todos los tests funcionales y de integración, se despliega automáticamente en el entorno de staging.
14. El pipeline CI/CD lanza GhostTrack contra el entorno de staging: `python main.py -t staging.miempresa.com --full --threads 20 -o reports/`.
15. El script `check_severity.py` analiza los reportes generados y falla el pipeline si hay hallazgos de severidad CRITICAL o HIGH.
16. Los reportes JSON se importan automáticamente en DefectDojo para tracking de vulnerabilidades.
17. Si el pipeline pasa, el código se despliega en producción; si falla, se crea automáticamente un issue en el sistema de ticket con los detalles del hallazgo.

Este ciclo garantiza que ningún código con vulnerabilidades críticas llega a producción sin pasar por revisión, y que el equipo de desarrollo recibe feedback sobre seguridad en el mismo ciclo de feedback que recibe sobre la funcionalidad.

T.4 Consideraciones de Seguridad en el Propio Pipeline

Irónicamente, los sistemas de CI/CD son objetivos frecuentes de ataques avanzados. Un atacante que comprometiera el servidor Jenkins o GitLab Runner de una organización podría modificar el código que pasa por el pipeline, inyectar backdoors en los artefactos desplegados, o exfiltrar los secretos configurados en las variables de entorno del pipeline. Las siguientes medidas de seguridad son relevantes para proteger el pipeline CI/CD que ejecuta GhostTrack:

- Principio de mínimo privilegio: El usuario de servicio que ejecuta GhostTrack en el pipeline solo debe tener permisos para leer los artefactos necesarios y escribir en el directorio de reportes, sin acceso a credenciales de producción.
- Aislamiento del entorno de staging: El entorno de staging donde se ejecutan las pruebas debe estar completamente aislado de producción mediante segmentación de red, para que incluso si una prueba de GhostTrack tiene efectos secundarios inesperados, no afecte al entorno productivo.
- Rotación de secretos: Las API keys de GhostTrack (Shodan, VirusTotal, etc.) configuradas como variables de entorno del pipeline deben rotarse periódicamente y nunca deben aparecer en los logs del pipeline.
- Firma de artefactos: Los reportes generados por GhostTrack deben ser firmados digitalmente para garantizar su integridad y detectar manipulaciones.

Apéndice U — Checklist Completo de Auditoría con GhostTrack

Este apéndice proporciona una lista de verificación completa para asegurar que ningún paso importante queda omitido en una auditoría realizada con GhostTrack. El checklist está organizado siguiendo las fases del PTES.

U.1 Fase Pre-Engagement

- Contrato de auditoría firmado con alcance, técnicas permitidas y período de autorización claramente definidos.
- Contactos de emergencia del cliente identificados (técnico, CISO, legal).
- Canal de comunicación seguro establecido (email cifrado, señal, etc.).
- Entorno de auditoría preparado: Kali Linux actualizado, GhostTrack instalado y verificado (`python main.py --version`), dependencias instaladas, entorno virtual activado.
- VPN o conectividad con el entorno objetivo verificada.
- Wordlists complementarias descargadas si están en el alcance (SecLists, etc.).
- Fichero `.env` configurado con las API keys disponibles.

U.2 Fase de Reconocimiento Pasivo

- GeolIP ejecutado: `python main.py -t TARGET -m geoip`
- WHOIS ejecutado: `python main.py -t TARGET -m whois`
- DNS Enumeration completo: `python main.py -t TARGET -m dns_enum`
- Subdomain Finder ejecutado: `python main.py -t TARGET -m subdomain_finder`
- Email Harvester ejecutado: `python main.py -t TARGET -m email_harvester`
- Metadatos de documentos públicos analizados: `python main.py -t TARGET --files *.pdf`
 - Revisión manual de resultados: ¿hay subdominios inesperados? ¿AXFR exitoso? ¿Emails corporativos?

U.3 Fase de Enumeración Activa

- Port Scanner ejecutado sobre puertos comunes: `python main.py -t TARGET -m port_scanner`
- Port Scanner extendido si procede: `python main.py -t TARGET -m port_scanner` (con rango personalizado)

- Service Detection sobre puertos abiertos: `python main.py -t TARGET -m service_detection`
- OS Fingerprinting: `python main.py -t TARGET -m os_fingerprint`
- Banner Grabbing: `python main.py -t TARGET -m banner_grabbing`
- SSL/TLS Analysis en todos los puertos HTTPS/IMAPS/SMTPS: `python main.py -t TARGET -m ssl_analyzer`
- Traceroute para mapear topología: `python main.py -t TARGET -m traceroute`
- ARP Scanner si se está en red local: `python main.py -t TARGET -m arp_scanner`

U.4 Fase de Análisis de Vulnerabilidades Web

- Web Crawler ejecutado: `python main.py -t TARGET -m web_crawler`
 - Directory Bruteforce con wordlist adecuada: `python main.py -t TARGET -m directory_bruteforce`
- SQL Injection en todos los endpoints con parámetros descubiertos.
- XSS Scanner en todos los formularios descubiertos por el Web Crawler.
- Vulnerability Scanner cruzando servicios con CVEs: `python main.py -t TARGET -m vuln_scanner`
 - Revisión manual de hallazgos: ¿hay .git expuesto? ¿paneles admin accesibles? ¿credenciales por defecto?

U.5 Fase Red Team (si está en el alcance)

- SMB Enumeration: `python main.py -t TARGET --redteam --domain DOMINIO`
- LDAP Enumeration: verificar cuentas con SPN, sin preauth, AdminCount=1
- BloodHound Collection: recolectar datos y analizar rutas de ataque
- Kerberoasting: solicitar TGS para todas las cuentas con SPN
- AS-REP Roasting: solicitar AS-REP para cuentas sin preauth
- PrivEsc Check: verificar Unquoted Paths, token privileges, AlwaysInstallElevated
- Lateral Movement: identificar sistemas accesibles con credenciales actuales
- Nota: documentar CADA acción con timestamp en los logs

U.6 Fase de Generación de Reportes

- Verificar que se han generado los tres formatos de reporte: HTML, JSON, TXT

- Revisar el reporte HTML: ¿todos los módulos aparecen? ¿los hallazgos están bien categorizados?
- Redactar el resumen ejecutivo (no generado automáticamente por GhostTrack)
- Redactar la narrativa de ataque para los hallazgos más críticos
- Completar las fichas de vulnerabilidad para los hallazgos CRITICAL y HIGH
- Elaborar el plan de remediación priorizado
- Enviar al cliente por canal seguro
- Programar reunión de presentación de resultados

U.7 Checklist Post-Auditoría

- Eliminar cualquier artefacto dejado durante las pruebas (backdoors, ficheros subidos, cuentas creadas)
- Verificar con el cliente que todos los accesos han sido revocados
- Archivar los reportes y logs de forma segura (cifrado en reposo)
- Documentar lecciones aprendidas para mejorar el proceso en futuras auditorías
- Si se encontraron compromisos previos o datos personales expuestos: seguimiento con el cliente
- Planificar auditoría de verificación post-remediación (re-test en 30-90 días)

Apéndice V — Comparativa Detallada: GhostTrack vs Frameworks Existentes

Este apéndice proporciona una comparativa detallada entre GhostTrack y los frameworks de pentesting más relevantes del sector, analizando cobertura funcional, arquitectura, requisitos y modelo de uso para ayudar al lector a entender el posicionamiento específico de GhostTrack en el ecosistema de herramientas de seguridad.

V.1 Comparativa de Cobertura Funcional

Tabla 12 Comparativa de cobertura funcional entre GhostTrack y frameworks del sector *Nota.*

Elaboración propia. ✓ = soportado nativamente; ~ = soportado parcialmente o requiere plugins; ✗ = no soportado.

Funcionalidad	GhostTrack	Metasploit	Nmap+NSE	OWASP ZAP	BloodHound
Reconocimiento OSINT	✓	✗	~	✗	✗
Escaneo de puertos TCP	✓	~	✓	✗	✗
Detección de servicios	✓	~	✓	~	✗
OS Fingerprinting	~	~	✓	✗	✗
SSL/TLS Analysis	✓	✗	~	✓	✗
Web Crawling	~	✗	✗	✓	✗
Dir Bruteforce	✓	~	~	✓	✗
SQL Injection	~	✓	~	✓	✗
XSS Detection	~	~	✗	✓	✗
AD Enumeration	✓	~	✗	✗	✓
Kerberoasting	✓	~	✗	✗	✗
BloodHound Collection	~	✗	✗	✗	✓
PrivEsc Check	✓	✓	✗	✗	✗
Lateral Movement	✓	✓	✗	✗	~
Payload Generation	~	✓	✗	✗	✗
C2 Communication	~	✓	✗	✗	✗

Funcionalidad	GhostTrack	Metasploit	Nmap+NSE	OWASP ZAP	BloodHound
Reporting automático	✓	✓	~	✓	~
Integración CI/CD	✓	~	~	✓	✗
Sin privilegios root	✓	✗	✗	✓	✓
Multiplataforma	✓	✓	✓	✓	✓

V.2 Análisis de Arquitectura

Cada framework tiene una filosofía arquitectural distinta que determina sus casos de uso más adecuados:

Metasploit Framework

Metasploit usa una arquitectura cliente-servidor donde el módulo msfconsole o la interfaz web Metasploit Pro se conectan al servidor Metasploit (msfrpcd). Cada exploit, módulo auxiliar o módulo post-explotación es un Ruby module que hereda de la clase base apropiada. Esta arquitectura facilita la extensión (añadir un exploit requiere crear un fichero Ruby con la estructura correcta) pero hace que la herramienta sea más pesada y compleja de instalar y mantener. La dependencia de Ruby y de PostgreSQL para la base de datos de sesiones añade complejidad al despliegue.

Nmap con NSE

Nmap es una herramienta de línea de comandos con una arquitectura muy simple: el núcleo está escrito en C para máximo rendimiento, y el Nmap Scripting Engine (NSE) permite extender la funcionalidad con scripts Lua. Los scripts NSE tienen acceso a las APIs de Nmap para enviar paquetes, hacer peticiones HTTP y DNS, etc. La fortaleza de Nmap es la velocidad (SYN Scan con raw sockets es significativamente más rápido que el TCP Connect Scan de GhostTrack) y la madurez del motor de fingerprinting de OS y servicios.

OWASP ZAP

ZAP (Zed Attack Proxy) es específico para aplicaciones web y tiene una arquitectura de proxy: se interpone entre el navegador del auditor y la aplicación web, capturando todo el tráfico. Esto le permite hacer un análisis mucho más completo de la aplicación porque ve exactamente las mismas peticiones que el navegador envía, incluyendo contenido generado dinámicamente por JavaScript. GhostTrack comparte con ZAP la capacidad de detectar SQLi

y XSS, pero ZAP es mucho más profundo en la cobertura de vulnerabilidades web específicas.

GhostTrack

GhostTrack tiene la arquitectura más simple de todos: Python puro con la librería estándar más un conjunto curado de librerías de terceros, sin servidor, sin base de datos, sin interfaz gráfica. Esta simplicidad es deliberada: permite instalar GhostTrack con un `pip install -r requirements.txt` en cualquier sistema con Python, sin dependencias de sistema difíciles de satisfacer, sin configuración de servidores o bases de datos. El precio es el rendimiento (Python es más lento que C o Go para operaciones de red intensivas) y la profundidad en áreas especializadas. La propuesta de valor única de GhostTrack es la integración: un solo comando ejecuta el reconocimiento, la enumeración, el análisis web Y el Red Team AD, y genera reportes automáticamente.

V.3 Conclusión del Análisis Comparativo

La tabla comparativa y el análisis arquitectural revelan que GhostTrack no compite directamente con ninguna herramienta individual del sector, sino que ocupa un nicho específico: la herramienta de pentesting integrada, sin dependencias complejas, lista para ejecutar en CI/CD. Las organizaciones que necesiten la máxima profundidad en un área específica (explotación con Metasploit, escaneo masivo con Nmap, análisis web con ZAP, análisis AD con BloodHound) deben usar esas herramientas especializadas. GhostTrack es la herramienta adecuada cuando se necesita una evaluación general completa con un mínimo de configuración y con generación automática de reportes multipropósito.

Para el contexto académico de un TFG en Desarrollo de Aplicaciones Multiplataforma, GhostTrack ofrece adicionalmente la ventaja de ser un proyecto desarrollado completamente desde cero: cada módulo implementa sus algoritmos de forma transparente, sin depender de librerías que oculten los detalles. Esto permite al autor demostrar un conocimiento profundo de los protocolos y técnicas implementadas, lo que sería imposible de justificar con el simple uso de herramientas de terceros.

Apéndice W — Guía de Hardening Basada en Hallazgos GhostTrack

Este apéndice proporciona una guía práctica de hardening de sistemas orientada a corregir las categorías de vulnerabilidades que GhostTrack detecta con mayor frecuencia. Cada sección incluye los comandos de configuración exactos y la forma de verificar la corrección con GhostTrack.

W.1 Hardening de Servidores Web

W.1.1 Suprimir información de versión en banners

El módulo Banner Grabbing detecta servidores que revelan su versión exacta en los headers HTTP. La corrección no tiene impacto funcional y elimina información valiosa para el atacante:

```
# Apache - httpd.conf
ServerTokens Prod          # Muestra solo 'Apache', sin versión
ServerSignature Off        # Elimina firma en páginas de error

# nginx - nginx.conf, bloque http{}
server_tokens off;         # Elimina versión de headers y páginas de error

# Verificación con GhostTrack
python main.py -t objetivo.com -m banner_grabbing
# El campo 'banner' no debe incluir número de versión
```

W.1.2 Configuración de headers de seguridad HTTP

Los headers de seguridad HTTP mitigan múltiples categorías de ataques web. El módulo Web Crawler de GhostTrack puede detectar su ausencia como hallazgo de severidad MEDIUM:

```
# Apache - mod_headers requerido
Header always set Strict-Transport-Security "max-age=31536000;
includeSubDomains; preload"
Header always set X-Frame-Options "DENY"
Header always set X-Content-Type-Options "nosniff"
Header always set Content-Security-Policy "default-src 'self'"
Header always set Referrer-Policy "strict-origin-when-cross-origin"
Header always unset Server
Header always unset X-Powered-By

# nginx
add_header Strict-Transport-Security "max-age=31536000; includeSubDomains"
always;
```

```
add_header X-Frame-Options "DENY" always;
add_header X-Content-Type-Options "nosniff" always;
add_header Content-Security-Policy "default-src 'self'" always;
```

W.2 Hardening SSL/TLS

El módulo SSL/TLS Analyzer detecta protocolos débiles y vulnerabilidades criptográficas. La siguiente configuración implementa el perfil moderno de Mozilla, compatible con todos los clientes con soporte TLS 1.2+:

```
# nginx - perfil moderno Mozilla
ssl_protocols TLSv1.2 TLSv1.3;
ssl_ciphers ECDHE-ECDSA-AES128-GCM-SHA256:ECDHE-RSA-AES128-GCM-SHA256:
            ECDHE-ECDSA-AES256-GCM-SHA384:ECDHE-RSA-AES256-GCM-SHA384:
            ECDHE-ECDSA-CHACHA20-POLY1305:ECDHE-RSA-CHACHA20-POLY1305;
ssl_prefer_server_ciphers off;
ssl_session_timeout 1d;
ssl_session_cache shared:SSL:10m;
ssl_session_tickets off;
ssl_stapling on;
ssl_stapling_verify on;

# Renovación automática de certificados con certbot
certbot renew --dry-run # Verificar que la renovación automática funciona

# Verificación con GhostTrack
python main.py -t objetivo.com -m ssl_analyzer
# vulnerabilities[] debe estar vacío
# supported_protocols debe contener solo TLSv1.2 y TLSv1.3
```

W.3 Hardening de Active Directory

W.3.1 Mitigación de Kerberoasting: migración a gMSA

El módulo KerberosAttacker detecta cuentas de servicio con SPNs registrados. La mitigación definitiva es migrar estas cuentas a gMSA (group Managed Service Accounts), cuyas contraseñas de 240 caracteres son gestionadas automáticamente por AD y rotadas cada 30 días:

```
# PowerShell - Crear y desplegar gMSA para SQL Server

# 1. Crear la clave raíz de gMSA (una vez por dominio)
Add-KdsRootKey -EffectiveImmediately

# 2. Crear la gMSA
```

```
New-ADServiceAccount -Name 'gMSA_SQL' \
    -DNSHostName 'sqlserver01.corp.local' \
    -PrincipalsAllowedToRetrieveManagedPassword 'sqlserver01$'

# 3. Instalar en el servidor
Install-ADServiceAccount 'gMSA_SQL'
Test-ADServiceAccount 'gMSA_SQL' # Debe devolver True

# 4. Verificar que el SPN ya no es explotable
Get-ADServiceAccount 'gMSA_SQL' -Properties ServicePrincipalName
# La cuenta NO debe aparecer en el output de KerberosAttacker
```

W.3.2 Mitigación de NTLM Relay

El módulo NTLMRelayAttacker detecta la ausencia de SMB Signing obligatorio y la presencia de LLMNR/NBT-NS. La mitigación requiere tres acciones complementarias:

```
# PowerShell — Habilitar SMB Signing obligatorio
Set-SmbServerConfiguration -RequireSecuritySignature $true -Force
Set-SmbClientConfiguration -RequireSecuritySignature $true -Force

# Deshabilitar LLMNR via GPO
# Computer Configuration > Administrative Templates > Network > DNS Client
# 'Turn off Multicast Name Resolution' = Enabled

# Deshabilitar NetBIOS via DHCP o registro
# HKLM\SYSTEM\CurrentControlSet\Services\NetBT\Parameters\Interfaces\{GUID}
# NetbiosOptions = 2 (deshabilitar NetBIOS)
```

W.4 Hardening de DNS

El módulo DNS Enumeration detecta transferencias de zona sin restricción. La corrección en BIND 9 requiere una sola directiva:

```
# BIND 9 — named.conf
options {
    allow-transfer { none; };          # Denegar AXFR globalmente
    version "not available";          # Suprimir versión de BIND
    allow-recursion { 192.168.0.0/16; }; # Solo recursión para internos
};

zone "empresa.com" {
    type master;
    file "/etc/bind/zones/empresa.com.db";
    # Solo permitir AXFR a NS secundarios autorizados
    allow-transfer { 192.168.1.2; 192.168.1.3; };
};
```

```
};

# Verificación
python main.py -t empresa.com -m dns_enum
# ZONE_TRANSFER debe estar ausente del resultado
```

W.5 Hardening de Bases de Datos Expuestas

El Port Scanner detecta frecuentemente bases de datos expuestas directamente en red. Para Elasticsearch, la corrección más urgente es habilitar la autenticación:

```
# Elasticsearch 8.x - elasticsearch.yml
xpack.security.enabled: true
xpack.security.transport.ssl.enabled: true
xpack.security.http.ssl.enabled: true

# Después de reiniciar, generar contraseñas automáticas
bin/elasticsearch-setup-passwords auto

# Para MongoDB - mongod.conf
security:
  authorization: enabled

# Para Redis - redis.conf
requirepass TuContraseñaSegura123!
bind 127.0.0.1 # Solo escuchar en localhost
protected-mode yes

# Verificación con GhostTrack
python main.py -t servidor -m port_scanner
# Los puertos de BD no deben ser accesibles desde redes externas
```

Apéndice X — Contribuir a GhostTrack

GhostTrack es un proyecto académico de código abierto que da la bienvenida a contribuciones de cualquier persona que desee mejorar el framework. Este apéndice describe cómo contribuir de forma efectiva al proyecto.

X.1 Tipos de Contribuciones

- Nuevos módulos: implementación de técnicas de pentesting o reconocimiento no cubiertas actualmente. El apéndice B proporciona la guía completa para desarrollar nuevos módulos compatibles.
- Mejoras de módulos existentes: aumentar la cobertura de un módulo existente (por ejemplo, añadir detección de DOM XSS al XSS Scanner, o añadir más CVEs a la base de datos del Vulnerability Scanner).
- Corrección de bugs: los issues reportados en GitHub con el label 'bug' son buenos puntos de entrada para nuevos contribuidores.
- Mejoras de documentación: correcciones de typos, explicaciones más claras, ejemplos adicionales, traducciones.
- Tests: añadir cobertura de tests a módulos que actualmente tienen cobertura insuficiente.

X.2 Proceso de Contribución

El proceso de contribución sigue el modelo estándar de GitHub:

18. Fork del repositorio en GitHub.
19. Crear una rama descriptiva: `git checkout -b feature/nuevo-modulo-xxxx` o `git checkout -b fix/correccion-port-scanner`.
20. Implementar los cambios siguiendo la guía de estilo del proyecto (PEP 8 para Python, docstrings en todos los métodos públicos, tests para cualquier nueva funcionalidad).
21. Ejecutar la suite de tests completa para verificar que no hay regresiones: `pytest tests/ -v --cov=. --cov-report=term-missing`.
22. Crear el Pull Request con una descripción clara de los cambios, el problema que resuelven y cómo verificar que funcionan correctamente.

Los Pull Requests serán revisados por el mantenedor del proyecto verificando que el código sigue los estándares del proyecto, que los tests son adecuados y pasan, que la documentación está actualizada, y que el cambio está alineado con la filosofía de diseño de GhostTrack.

X.3 Código de Conducta

Las contribuciones al proyecto deben respetar el código de conducta del Contributor Covenant, que establece que el espacio de contribución debe ser un entorno inclusivo y respetuoso para todas las personas independientemente de su experiencia, género, identidad, orientación sexual, discapacidad, apariencia física, tamaño corporal, raza, etnia, edad o religión. Las contribuciones que implementen técnicas cuyo único propósito sea facilitar actividades ilegales no serán aceptadas.

X.4 Créditos y Agradecimientos

GhostTrack ha sido desarrollado por Izán Jiménez Nuñez como Trabajo de Fin de Grado en el Técnico Superior en Desarrollo de Aplicaciones Multiplataforma de la Escuela de Imagen y Sonido CES, bajo la tutela de Mateo Morace Llamas, durante el curso académico 2025-2026.

El proyecto se apoya en el trabajo de decenas de investigadores y desarrolladores cuyos proyectos open source han sido fuente de inspiración: Gordon Lyon (Nmap), Renaud Deraison (Nessus), H.D. Moore (Metasploit), Dan Farmer y Wietse Venema (SATAN), Will Schroeder / BloodHound Team, y la comunidad OWASP. A todos ellos, gracias por hacer de la ciberseguridad un campo más abierto y colaborativo.

— *Fin del documento* —